DIGITAL COMMUNICATIONS

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

DEPARTMENT OF INFORMATION AND COMPUTER ENGINEERING

PROJECT IN MATLAB 2023

STUDENT DETAILS

NAME: ATHANASIOU VASILEIOS EVANGELOS
STUDENT ID: 19390005
SEMESTER: 8th
STUDENT STATUS : UNDERGRADUATE
STUDY PROGRAM : UNIWA
LABORATORY DEPARTMENT : DEPARTMENT 2A TUESDAY 15:25 – 16:30
LABORATORY INSTRUCTOR : MICHAILIDES EMMANOUIL
DELIVERY DATE : 14/6/2023

STUDENT PHOTO:

CONTENTS

EXERCISE 1

Sine wave signal. Create a function that generates a sinusoidal signal. Arguments of the function to consider the frequency of the signal, its sampling frequency, its amplitude, its phase and its time duration. Outputs of the function to be the sinusoidal signal and the time axis on which it was defined, e.g. [x,t]=mine_sin(f0,fs,A,ph,T1).

Code in Matlab

Main program

```
f 0 = 5
fs = 1000
A = 5
ph = pi
T 1 = 2
[ x , t ] = mine _ sin ( f 0 , fs , A , ph , T 1 )
plot ( t , x )
xlabel ( 'Time ( sec )' );
ylabel ( 'Amplitude ( Volt )' );
title ( ' x ( t ) = 5* sin (2* pi *5* t +π)' );
```

Code of the mine _ sin function

```
function [ x , t ] = mine _ sin ( f 0 , fs , A , ph , T 1 )

  Ts = 1/ fs ;
  Start = 0;
  Step = Ts ;
  End = T 1- Ts ;
  t = Start : Step : End ;
  x = A * sin (2* pi * f 0* t + ph );

end
```

Explanation of the commands used

Commands	Results	Comments
>> f 0 = 5	f0 = 5	The frequency of the signal.
>> fs = 1000	fs = 1000	The sampling frequency.
>> A = 5	A = 5	The width of the signal.
>> ph = pi	ph = 3.1416	The phase of the signal
>> T 1 = 2	T1 =	The time duration of the signal

	2	
>> [x , t] = mine _ sin (f 0 , fs , A , ph , T 1) Ts = 1/ fs Start = 0 Step = Ts End = T 1- Ts t = Start : Step : End ; x = A * sin (2* pi * f 0* t + ph);	Ts = 1.0000 e -03 Start = 0 Step = 1.0000 e -03 End = 1.9990 x = Columns 1 through 5 0.0000 -0 .1571 -0.3140 -0.4705 -0.6267 Columns 6 through 10 -0.7822 -0.9369 -1.0907 -1.2434 -1.3950 ... Columns 1.991 through 1.995 1.5451 1.3950 1.2434 1.0907 0.9369 Columns 1,996 through 2,000 0.7822 0.6267 0.4705 0.3140 0.1571 t = Columns 1 through 5 0 0.0010 0.0020 0.0030 0.0040 Columns 6 through 10 0.0050 0.0060 0.0070 0.0080 0.0090 … Columns 1.991 through 1.995 1.9900 1.9910 1.9920 1.9930 1.9940 Columns 1,996 through 2,000	Call the function mine _ sin (f 0, fs , A , ph , T 1) and return the sine signal, as the time axis on which it was set. Ts = 1/ fs →The sampling period Start = 0 →The starting time of the signal Step = Ts →The pitch of the signal End = T1-T s →The time when the signal completes a period x = A * sin (2* pi * f 0* t + ph) →The sinusoidal signal t = Start : Step : End →The time axis defined by the signal

	1.9950 1.9960 1.9970 1.9980 1.9990	
>> plot(t, x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('x(t) = 5*sin(2*pi*5*t+ π)');	Figure 1.1	Plotting the graph of the signal in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis .

Graphic representations

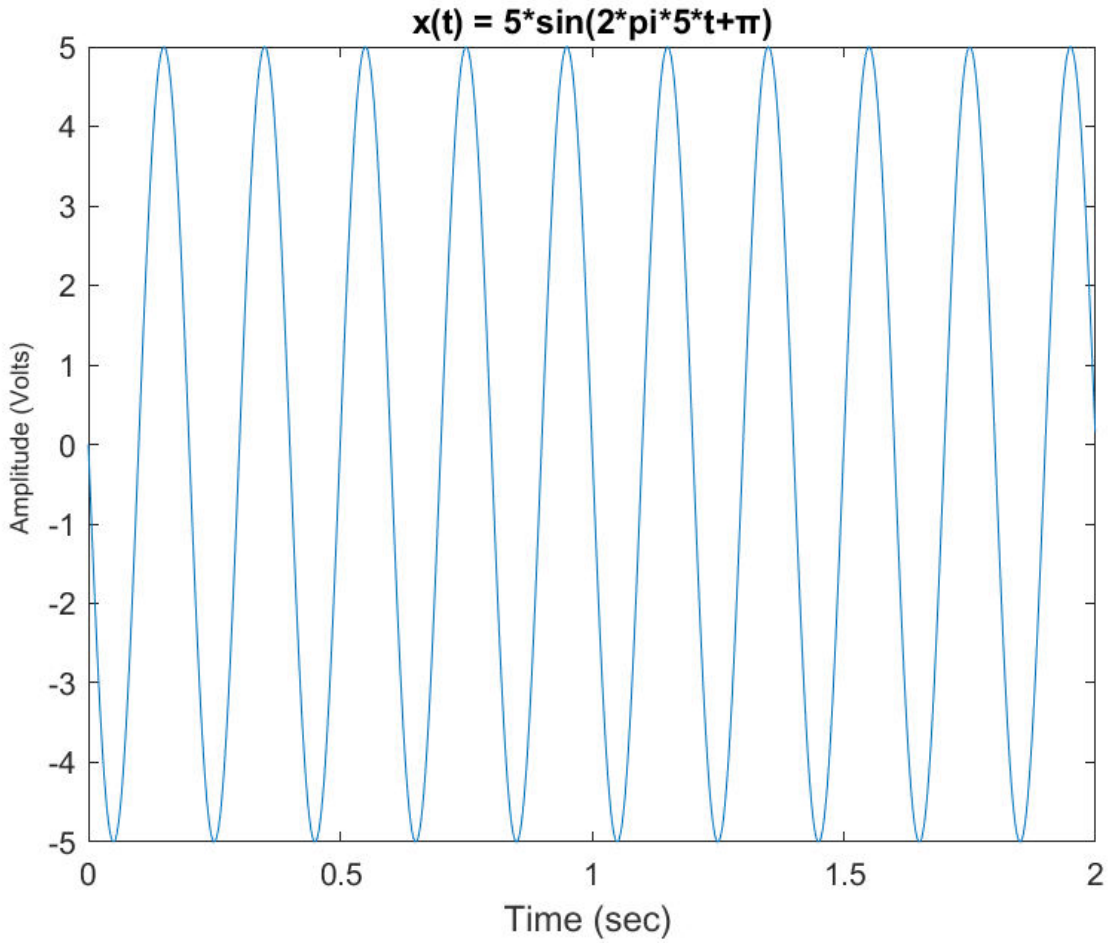

Figure 1.1. The sinusoidal signal

EXERCISE 2

Spectrum function. Create a function that calculates the amplitude spectrum of a sinusoidal signal using the fft() function. Arguments of the function take the signal and its period.

Code in Matlab

Main program

```
f 0 = 5
fs = 1000
A = 5
ph = pi
T1 = 2
[x, t] = mine_sin ( f0 , fs, A, ph , T1)
T = 1/f0
[ ph , f ] = phase_sin ( x , T )
plot( f, abs( ph ))
xlabel ( ' Field Frequencies (Hz)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Spectrum of width x(t) = 5*(2*pi*5*t+ π )' );
```

Code her function phase_sin

```
function [ ph , f ] = phase_ sin ( x , T )

dt = T/100;
BW = 1/dt;
N = length(x);
df = BW/N;
Start = -BW/2;
Step = df ;
End = BW/2-df;
f = Start:Step :End ;
ph = fft (x);
ph = fftshift(ph)/N;

end
```

Explanation of the commands used

Commands	Results	Comments
>> f0 = 5	f0 = 5	The frequency of the signal.
>> fs = 1000	fs = 1000	The sampling frequency.
>> A = 5	A = 5	The width of the signal.
>> ph = pi	ph = 3.1416	The phase of the signal
>> T1 = 2	T1 =	The time duration of the signal

	2	
>> [x, t] = mine_sin (f0 , fs, A, ph , T1) Ts = 1/fs Start = 0 Step = Ts End = T1-Ts t = Start:Step :End ; x = A*sin(2*pi*f0* t+ph); Call the function mine _ sin (f 0, fs , A , ph , T 1) and return the sine signal, as the time axis on which it was set. Ts = 1/ fs →The sampling period Start = 0 →The starting time of the signal Step = Ts →The pitch of the signal End = T1-T s →The time when the signal completes a period x = A * sin (2* pi * f 0* t + ph) →The sinusoidal signal t = Start : Step : End →The time axis defined by the signal	Ts = 1.0000e-03 Start = 0 Step = 1.0000e-03 End = 1.9990 x = Columns 1 through 5 0.0000 -0.1571 -0.3140 -0.4705 -0.6267 Columns 6 through 10 -0.7822 -0.9369 -1.0907 -1.2434 -1.3950 ... Columns 1.991 through 1.995 1.5451 1.3950 1.2434 1.0907 0.9369 Columns 1,996 through 2,000 0.7822 0.6267 0.4705 0.3140 0.1571 t = Columns 1 through 5 0 0.0010 0.0020 0.0030 0.0040 Columns 6 through 10 0.0050 0.0060 0.0070 0.0080 0.0090 … Columns 1.991 through 1.995 1.9900 1.9910 1.9920 1.9930 1.9940 Columns 1,996 through 2,000	

	1.9950 1.9960 1.9970 1.9980 1.9990	
`>> T = 1/f0`	T = 0.2000	The signal period

`>> [ ph , f ] = phase_sin ( x , T )`
 	
`dt = T/100`
`BW = 1/dt`
`N = length(x)`
`df = BW/N`
`Start = -BW/2`
`Step = df`
`End = BW/2-df`
`f = Start:Step :End ;`
`ph = fft (x);`
`ph = fftshift ( ph )/N;`
	dt =
	
	0.0020
	
	BW =
	
	500
	
	N =
	
	2000
	
	df =
	
	0.2500
	
	Start =
	
	-250
	
	Step =
	
	0.2500
	
	End =
	
	249.7500
	
	ph =
	
	Columns 1 through 6
	
	0.0000 + 0.0000i 0.0000 + 0.0000i 0.0000 + 0.0000i - 0.0000 + 0.0000i - 0.0000 + 0.0000i -0.0000 + 0.0000i
	
	Columns 7 through 12
	
	-0.0000 + 0.0000i - 0.0000 + 0.0000i - 0.0000 + 0.0000i -0.0000 - 0.0000i - 0.0000 - 0.0000i -0.0000 - 0.0000i …
	
	Columns 1.993 through 1.998
	
	-0.0000 - 0.0000i - 0.0000 - 0.0000i - 0.0000 - 0.0000i -0.0000 - 0.0000i - 0.0000 - 0.0000i -0.0000 - 0.0000i
	
	Columns 1,999 through 2,000
		Call the function phase _ sin (x , T) and return the amplitude spectrum of the halftone signal, as well as the frequency domain set
		
		dt = T /100 →The sampling to be 1/100 of the signal period
		
		BW = 1/ dt →The bandwidth defined by the sampling in time
		
		N = length (x) →The length of the sinusoidal signal
		
		df = BW / N →Sampling in the frequency domain
		
		Start = - BW /2 →The first frequency of the signal's width spectrum
		
		Step = df →The step of the amplitude spectrum of the signal
		
		End = BW /2-d f →The last frequency of the signal's width spectrum
		
		f = Start : Step : End →The table of frequencies
		
		ph = fft (x) →The amplitude spectrum of the signal, where it is performed by the fast Fourier transform
		
		ph = fftshift (ph)/ N →The spectrum is shifted to be sym-metric about the vertical axis

	0.0000 - 0.0000i 0.0000 - 0.0000i f = Columns 1 through 11 -250.0000 -249.7500 -249.5000 -249.2500 -249.0000 -248.7500 -248.5000 -248.2500 -248.0000 -247.7500 -247.5000 Columns 12 through 22 -247.2500 -247.0000 -246.7500 -246.5000 -246.2500 -246.0000 -245.7500 -245.5000 -245.2500 -245.0000 -244.7500 … Columns 1.981 through 1.991 245.0000 245.2500 245.5000 245.7500 246.0000 246.2500 246.5000 246.7500 247.0000 247.2500 247.5000 Columns 1,992 through 2,000 247.7500 248.0000 248.2500 248.5000 248.7500 249.0000 249.2500 249.5000 249.7500	

`>> plot ( f , abs ( ph ))`
`>> xlabel ( 'Frequency Range ( Hz )' );`
`>> ylabel ( 'Amplitude ( Volt )' );`
`>> title( 'Width spectrum x(t) = 5*(2*pi*5*t+π)' );`
	Figure 2 .1	The plotting of the spectrum of the amplitude of the signal in the frequency domain defined. Add title and labels to the horizontal x- axis and vertical y- axis .

Graphic representations

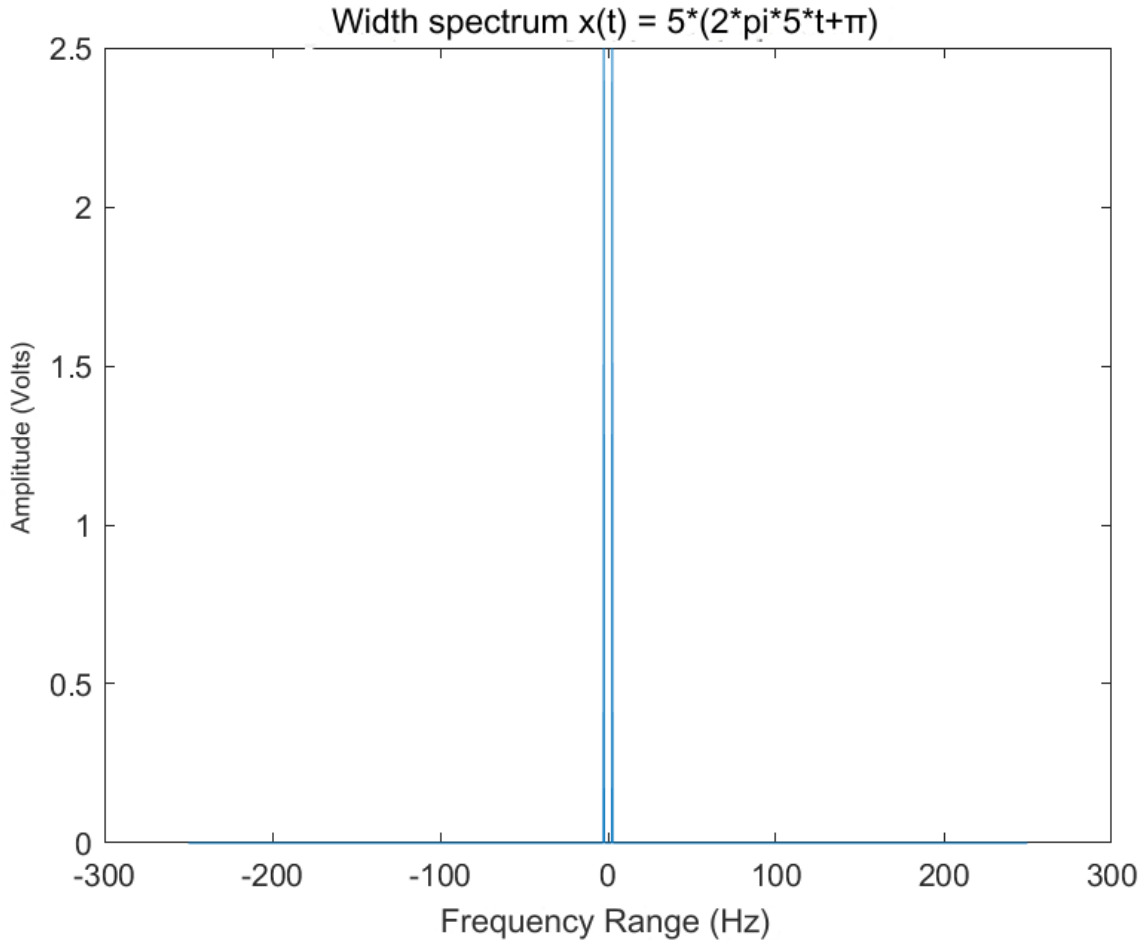

Figure 2.1. The amplitude spectrum of the sinusoidal signal

EXERCISE 3

Spectrum of a sinusoidal signal. Using the functions of questions 1) and 2) to create a sine of frequency 10 Hz with a sampling frequency of 1KHz, the amplitude and phase of the sine to be 5 Volt and 0^{0} respectively and the time duration 0.5 sec. Make the graph of the sine, find its spectrum and make the graph of the amplitude spectrum. Compare the amplitude spectrum plotted on the graph with the theoretical expression for the amplitude spectrum of the sine.

Matlab code

```
f0 = 10
fs = 1000
A = 5
ph = 0^0
T1 = 0.5
[x, t] = mine_sin ( f0 , fs, A, ph , T1)
[y, t] = mine_sin ( f0, fs, A/2, ph , T1)
```

```
T = 1/f0
[ph1,f] = phase_sin ( x ,T)
[ph2,f] = phase_sin ( y ,T)

subplot( 2 , 1 , 1 );
plot( t, x, 'b' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'x(t) = 5*sin(2*pi*10*t+0^0)' );
hold on
plot( t, y, 'r' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'x(t) = 2.5*sin(2*pi*10*t+0^0)' );
legend ( 'Sine signal' , 'Half width sine signal' );
hold off
subplot( 2 , 1 , 2 );
plot( f, abs(ph1), 'b' )
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum x(t) = 5*sin(2*pi*10*t+0^0)' );
hold on
plot( f, abs(ph2), 'r' )
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum x(t) = 2.5*sin(2*pi*10*t+0^0)' );
legend( 'Width spectrum x(t) = 5*sin(2*pi*10*t+0^0)' , 'Theoretical expression of the
spectrum' );
hold off
```

Explanation of the commands used

Commands	Results	Comments
`>> f0 = 10`	f0 = 10	The frequency of the signal.
`>> fs = 1000`	fs = 1000	The sampling frequency.
`>> A = 5`	A = 5	The width of the signal.
`>> ph = 0^0`	ph = 1	The phase of the signal
`>> T1 = 0.5`	T1 = 0.5000	The time duration of the signal

`>> [x, t] = mine_sin ( f0 , fs, A, ph , T1)`
 	
`Ts = 1/fs`
`Start = 0`
	Ts =
	
	1.0000e-03
		Call the function mine _ sin (f 0, fs , A , ph , T 1) and return the sine signal, as the time axis on which it was set.

Step = Ts End = T1-Ts t = Start:Step :End ; x = A*sin(2*pi*f0* t+ph);	Start = 0 Step = 1.0000e-03 End = 0.4990 x = Columns 1 through 11 4.2074 4.3687 4.5128 4.6390 4.7470 4.8362 4.9064 4.9572 4.9884 4.9999 4.9917 Columns 12 through 22 4.9638 4.9163 4.8494 4.7634 4.6586 4.5354 4.3943 4.2358 4.0606 3.8694 3.6630 … Columns 485 through 495 -0.0265 0.2875 0.6003 0.9108 1.2177 1.5198 1.8159 2.1048 2.3855 2.6567 2.9174 Columns 496 through 500 3.1666 3.4033 3.6266 3.8356 4.0294 t = Columns 1 through 11 0 0.0010 0.0020 0.0030 0.0040 0.0050 0.0060 0.0070 0.0080 0.0090 0.0100 Columns 12 through 22 0.0110 0.0120 0.0130 0.0140 0.0150 0.0160 0.0170 0.0180 0.0190 0.0200 0.0210 … Columns 485 through 495	Ts = 1/ fs →The sampling period Start = 0 →The starting time of the signal Step = Ts →The pitch of the signal End = T1-T s →The time when the signal completes a period x = A * sin (2* pi * f 0* t + ph) →The sinusoidal signal t = Start : Step : End →The time axis defined by the signal

	0.4840 0.4850 0.4860 0.4870 0.4880 0.4890 0.4900 0.4910 0.4920 0.4930 0.4940 Columns 496 through 500 0.4950 0.4960 0.4970 0.4980 0.4990	
>> [y, t] = mine_sin (f0 , fs, A/2, ph , T1) Ts = 1/fs Start = 0 Step = Ts End = T1-Ts t = Start:Step :End ; x = A*sin(2*pi*f0* t+ph); 	Ts = 1.0000e-03 Start = 0 Step = 1.0000e-03 End = 0.4990 y = Columns 1 through 5 2.1037 2.1843 2.2564 2.3195 2.3735 Columns 6 through 10 2.4181 2.4532 2.4786 2.4942 2.5000 … Columns 491 through 495 0.9080 1.0524 1.1927 1.3283 1.4587 Columns 496 through 500 1.5833 1.7017 1.8133 1.9178 2.0147 t = Columns 1 through 5 0 0.0010 0.0020 0.0030 0.0040 Columns 6 through 10	Call the function mine _ sin (f 0, fs , A /2, ph , T 1) and return the sine signal, as the time axis on which it was set. Ts = 1/ fs →The sampling period Start = 0 →The starting time of the signal Step = Ts →The pitch of the signal End = T1-T s →The time when the signal completes a period x = A * sin (2* pi * f 0* t + ph) →The sinusoidal signal t = Start : Step : End →The time axis defined by the signal

	0.0050 0.0060 0.0070 0.0080 0.0090 … Columns 491 through 495 0.4900 0.4910 0.4920 0.4930 0.4940 Columns 496 through 500 0.4950 0.4960 0.4970 0.4980 0.4990	
`>> T = 1/f0`	T = 0.1000	The signal period

`>> [ ph , f ] = phase_sin ( x , T )`
 	
`dt = T/100`
`BW = 1/dt`
`N = length(x)`
`df = BW/N`
`Start = -BW/2`
`Step = df`
`End = BW/2-df`
`f = Start:Step :End ;`
`ph = fft (x);`
`ph = fftshift ( ph )/N;`
	dt =
	
	1.0000e-03
	
	BW =
	
	1000
	
	N =
	
	500
	
	df =
	
	2
	
	Start =
	
	-500
	
	Step =
	
	2
	
	End =
	
	498
	
	ph =
	
	Columns 1 through 6
	
	0.0000 + 0.0000i 0.0000 + 0.0000i
	0.0000 + 0.0000i 0.0000 + 0.0000i
	0.0000 + 0.0000i - 0.0000 + 0.0000i
		Call the function phase _ sin (x , T) and return the amplitude spectrum of the halftone signal, as well as the frequency domain set
		
		dt = T /100 →The sampling to be 1/100 of the signal period
		
		BW = 1/ dt →The bandwidth defined by sampling in time
		
		N = length (x) →The length of the sinusoidal signal
		
		df = BW / N →Sampling in the frequency domain
		
		Start = - BW /2 →The first frequency of the signal's width spectrum
		
		Step = df →The step of the amplitude spectrum of the signal
		
		End = BW /2-d f →The last frequency of the signal's width spectrum
		
		f = Start : Step : End →The table of frequencies
		
		ph = fft (x) →The amplitude spectrum of the signal, where it is performed by the fast Fourier transform
		
		ph = fftshift (ph)/ N →The spectrum is shifted to be sym-metric about the vertical axis

	Columns 7 through 12 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i -0.0000 + 0.0000i 0.0000 + 0.0000i 0.0000 - 0.0000i … Columns 493 through 498 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i -0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i Columns 499 through 500 0.0000 - 0.0000i 0.0000 - 0.0000i f = Columns 1 through 19 - 500 - 498 -496 -494 -492 -490 -488 -486 -484 -482 -480 -478 -476 -474 -472 -470 -468 -466 -464 Columns 20 through 38 - 462 - 460 -458 -456 -454 -452 -450 -448 -446 -444 -442 -440 -438 -436 -434 -432 -430 -428 -426 ... Columns 476 through 494 450 452 454 456 458 460 462 464 466 468 470 472 474 476 478 480 482 484 486 Columns 495 through 500 488 490 492 494 496 498	

`>> [ph2,f] = phase_sin ( y ,T)`
 	
`dt = T/100`
`BW = 1/dt`
`N = length(x)`
`df = BW/N`
`Start = -BW/2`
`Step = df`
`End = BW/2-df`
`f = Start:Step :End ;`
`ph = fft (x);`
`ph = fftshift ( ph )/N;`
	dt =
	
	1.0000e-03
	
	BW =
	
	1000
	
	N =
	
	500
		Call the function phase _ sin (y , T) and return the amplitude spectrum of the halftone signal, as well as the frequency domain set
		
		dt = T /100 →The sampling to be 1/100 of the signal period
		
		BW = 1/ dt →The bandwidth defined by sampling in time
		
		N = length (x) →The length of the sinusoidal signal

	df = 2 Start = -500 Step = 2 End = 498 ph2 = Columns 1 through 2 0.0000 + 0.0000i 0.0000 + 0.0000i Columns 3 through 4 0.0000 + 0.0000i 0.0000 + 0.0000i … Columns 497 through 498 0.0000 - 0.0000i 0.0000 - 0.0000i Columns 499 through 500 0.0000 - 0.0000i 0.0000 - 0.0000i f = Columns 1 through 9 - 500 - 498 -496 -494 -492 -490 -488 -486 -484 Columns 10 through 18 - 482 - 480 -478 -476 -474 -472 -470 -468 -466 ... Columns 487 through 495 472 474 476 478 480 482 484 486 488	df = BW / N →Sampling in the frequency domain Start = - BW /2 →The first frequency of the signal's width spectrum Step = df →The step of the amplitude spectrum of the signal End = BW /2-d f →The last frequency of the signal's width spectrum f = Start : Step : End →The table of frequencies ph = fft (x) →The amplitude spectrum of the signal, where it is performed by the fast Fourier transform ph = fftshift (ph)/ N →The spectrum is shifted to be sym-metric about the vertical axis

	Columns 496 through 500 490 492 494 496 498	
>> subplot(2, 1, 1); >> plot(t, x, 'b') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('x(t) = 5*sin(2*pi*10*t+0^0)'); >> hold on ? >> plot(t, y, 'r') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('x(t) = 2.5*sin(2*pi*10*t+0^0)'); >> legend ('Sine signal', 'Half width sine sig-nal'); >> hold off ? >> subplot(2, 1, 2); >> plot(f, abs(ph1), 'b') >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum x(t) = 5*sin(2*pi*10*t+0^0)'); >> hold on ? >> plot(f, abs(ph2), 'r') >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum x(t) = 2.5*sin(2*pi*10*t+0^0)'); >> legend('Width spec-trum x(t) = 5*sin(2*pi*10*t+0^0)' , 'Theoretical expression of the spectrum'); >> hold off ?	Figure 3.1	The plotting of the two signals in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis . The signals are represented in the same shape and with different colors indicated in a new label. The plotting of the plots of the amplitude spectrum and the theoretical expression of the amplitude spectrum in the domain of defined frequencies. Add title and labels to the horizontal x- axis and vertical y- axis . The representation of the spectra is done in the same figure and with different colors indicated in a new label.

Graphic representations

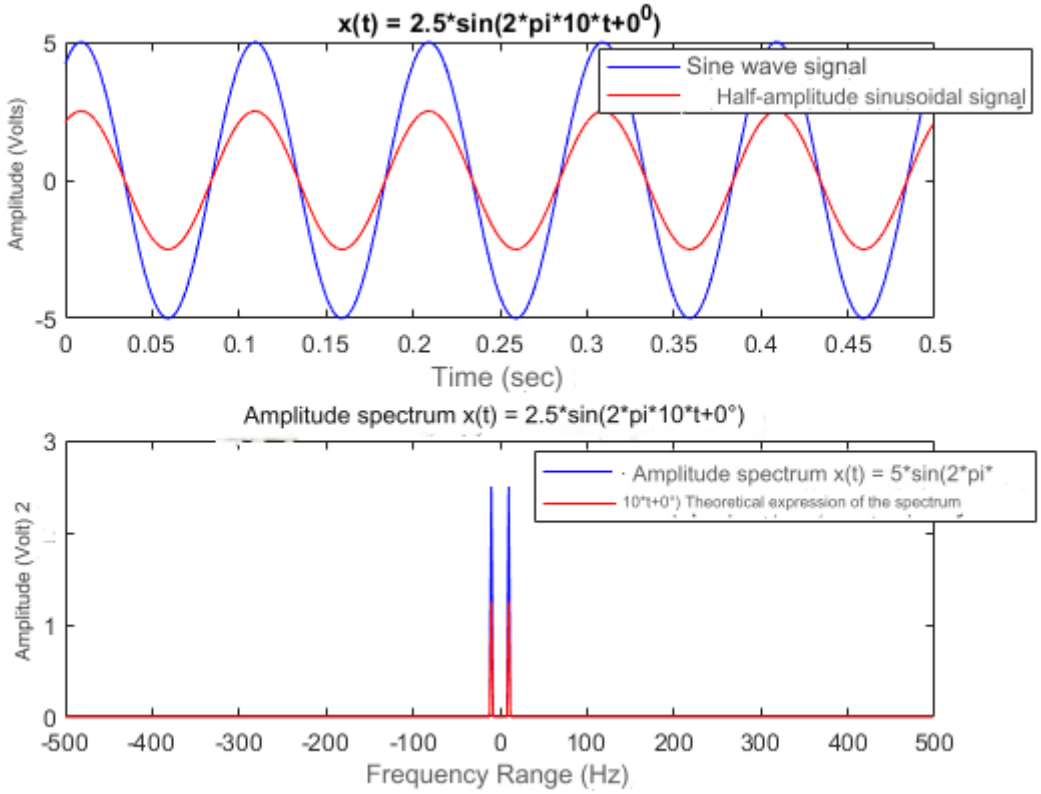

Figure 3.1. The sinusoidal signal, the amplitude spectrum of the signal and the theoretical expression of the spectrum

EXERCISE 4

Speech signal spectrum. Download and save the 3WORDS.WAV speech signal found at https://booksite.elsevier.com/9780080993881/ (data directory). Read the signal with the audioread function and store it in a vector. Hear the signal using the sound function. To plot the waveform of the signal as a function of time, and to represent the amplitude spectrum of the signal.

Code at Matlab

```
[x, Fs] = audioread ( '3WORDS.wav' )
sound( x, Fs );

N = length(x)
Start = 0
Step = 1
End = N-1
t = ( Start:Step :End )/Fs
```

```
subplot( 2 , 1 , 1 );
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Waveform 3WORDS.wav' );

dt = 1/(Fs*100)
BW = 1/dt
df = BW/N
Start = -BW/2
Step = df
End = BW/2-df
f = Start:Step :End
ph = fft (x)
ph = fftshift ( ph )/N
subplot( 2 , 1 , 2 );
plot( f, abs( ph ))
xlabel ( ' Field Frequencies (Hz)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Spectrum latitudinal of signal 3WORDS.wav' );
```

Explanation of the commands used

Commands	Results	Comments
`>> [x, Fs] = audioread ( '3WORDS.wav' )`	x = … -0.0656 -0.0672 -0.0674 -0.0686 -0.0658 -0.0681 Fs = 44100	Call the function audioread (filename) and save the signal read from the file '3W ORDS . wav » to the vector [x , Fs], which is defined in the frequency domain.
`>> sound( x, Fs )`		Hearing the signal
`>> N = length(x)`	N = 420864	The signal size
`>> Start = 0;`	Start = 0	The time origin at which the signal starts in the time field
`>> Step = 1;`	Step = 1	The pitch of the signal
`>> End = N-1;`	End = 420863	The time when the signal completes a period
`>> t = ( Start:Step :End )/Fs`	t = … Columns 420.856 through 420.860 9.5432 9.5432 9.5432 9.5433 9.5433 Columns 420.861 through 420.864	The range of times in which the signal waveform will be defined

	9.5433 9.5433 9.5434 9.5434	
>> subplot(2, 1, 1); >> plot(t, x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Waveform 3WORDS.wav');	Figure 4.1	Plotting the waveform of the signal in the specified time domain. Add title and labels to the horizontal x- axis and vertical y- axis .
>> dt = 1 / (Fs* 100)	dt = 2.2676e-07	Sampling to be 1/100 of the signal period
>> BW = 1/dt	BW = 4410000	The bandwidth defined by sampling in time
>> df = BW/N	df = 10.4784	Sampling in the frequency domain
>> Start = -BW/2	Start = -2205000	The first frequency of the signal's width spectrum
>> Step = df	Step = 10.4784	The step of the amplitude spectrum of the signal
>> End = BW/2-df	End = 2.2050e+06	The last frequency of the signal's width spectrum
>> f = Start:Step :End	f = … Columns 420.856 through 420.860 2.2049 2.2049 2.2049 2.2049 2.2049 Columns 420.861 through 420.864 2.2050 2.2050 2.2050 2.2050	The range of frequencies defined by the signal's amplitude spectrum
>> ph = fft(x);	ph = … 0.1318 + 0.0067i 0.0017 - 0.0216i 0.0212 - 0.0639i 0.0104 - 0.0249i -0.0281 - 0.0024i 0.0006 + 0.0043i 0.0066 + 0.0058i	The amplitude spectrum of the signal from the fast Fourier transform
>> ph = fftshift(ph)/N	ph = … -0.0000 + 0.0000i 0.0000 - 0.0000i -0.0000 - 0.0000i -0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 + 0.0000i 0.0000 + 0.0000i	The amplitude spectrum of the signal from the fast Fourier transform , symmetric about the vertical axis
>> subplot(2, 1, 2); >> plot(f, abs(ph))		Plotting the amplitude spectrum of the signal waveform in the

>> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum of the signal 3WORDS.wav'); 	Figure 4.1	specified frequency domain. Add title and labels to the horizontal x- axis and vertical y- axis .

Graphic representations

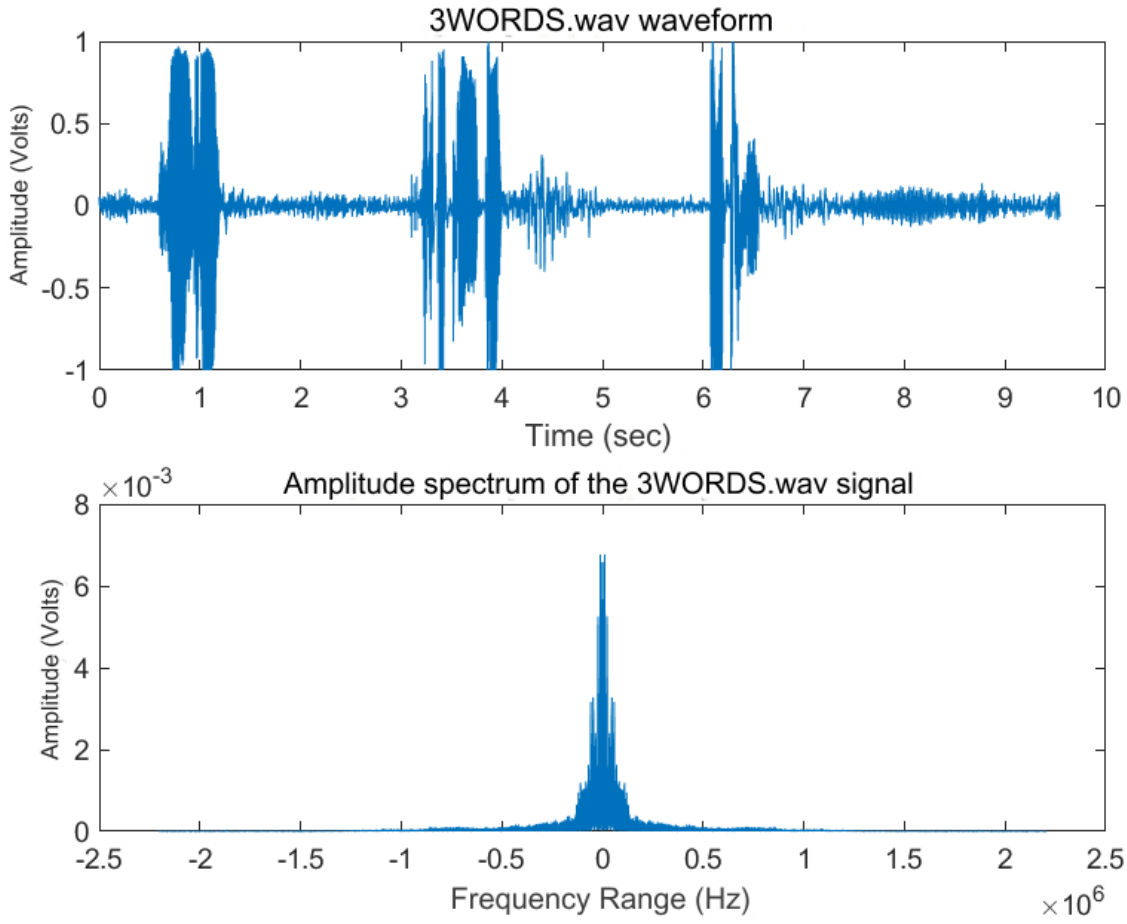

Figure 4.1. The waveform of the signal "3 WORDS . wav » and the amplitude spectrum

EXERCISE 5

Noisy signal spectrum. To create an information signal consisting of the superposition of cosine signals from 500Hz to 3000Hz, with a step of 100Hz.

Create the time axis by defining the elementary time change, dt, based on the fastest component and the observation duration based on the slowest component. Create the graphic representation of the signal.

You can also hear the signal with sound(x). Add 10 dB white additive Gaussian noise to the signal using awgn(). Also create a 2nd noisy signal with 5dB noise added. To make the corresponding graphical representations in new graphical windows. (Also listen for the two noisy signals). To represent the width spectrum of all signals.

Code at Matlab

```
Start = 500
Step = 100
End = 3000
f = Start:Step :End
ts = 1/End
tf = 1/Start
dt = tf /100
tw = 5* ts
t = dt:dt :tw ;

x = zeros(length(f), length(t));

for i = 1: length (f)
    x( i ,:) = cos(2*pi*f( i )*t);
end
signal_x = sum(x)

noise1 = 10
y = awgn ( x , noise1 );
signal_y = sum(y)

noise2 = 5
z = awgn ( x , noise2 );
signal_z = sum(z)

sound( signal_x );
pause( 1 );
sound( signal_y );
pause( 1 );
sound( signal_z );

figure
subplot (3, 1, 1)
plot( t, signal_x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Silent mark ' );
subplot (3, 1, 2)
plot( t, signal_y )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title ( 'Noisy signal with 10 db noise ' );
subplot (3, 1, 3)
plot( t, signal_z )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title ( 'Noisy signal with 5 db noise ' );
```

```
BW = 1/dt

N_x = length( signal_x )
df_x = BW/ N_x
f_x = -BW/2:df_x:BW/2-df_x
ph_x = fft ( signal_x )
ph_x = fftshift ( ph_x )/ N_x

N_y = length( signal_y )
df_y = BW/ N_y
f_y = -BW/2:df_y:BW/2-df_y
ph_y = fft ( signal_y )
ph_y = fftshift ( ph_y )/ N_y

N_z = length( signal_z )
df_z = BW/ N_z
f_z = -BW/2:df_z:BW/2-df_z
ph_z = fft ( signal_z )
ph_z = fftshift ( ph_z )/ N_z

figure
subplot( 3, 1, 1 )
plot( f_x , abs( ph_x ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum of quiet signal' );
subplot( 3, 1, 2 )
plot( f_y , abs( ph_y ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( '10db Noise Signal Width Spectrum' );
subplot( 3, 1, 3 )
plot( f_z , abs( ph_z ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( '5db Noise Signal Width Spectrum' );
```

Explanation of the commands used

Commands	Results	Comments
`>> Start = 500`	Start = 500	The first frequency of the information signal
`>> Step = 100`	Step = 100	The pitch of the information signal
`>> End = 3000`	End = 3000	The last frequency of the information signal

`>> f = Start:Step :End`	f = Columns 1 through 7 500 600 700 800 900 1000 1100 Columns 8 through 14 1200 1300 1400 1500 1600 1700 1800 Columns 15 through 21 1900 2000 2100 2200 2300 2400 2500 Columns 22 through 26 2600 2700 2800 2900 3000	The frequency domain of the information signal
`>> ts = 1/End`	ts = 3.3333e-04	The period of the slow component
`>> tf = 1/Start`	tf = 0.0020	The period of the fast component
`>> dt = tf /100`	dt = 2.0000e-05	The elementary time variation dt , where must be 1/100 base of the fastest component
`>> tw = 5* ts`	tw = 0.0017	The observation duration based on the slowest component
`>> t = dt:dt :tw ;`	t = Columns 1 through 8 0.0000 0.0000 0.0001 0.0001 0.0001 0.0001 0.0001 0.0002 Columns 9 through 16 0.0002 0.0002 0.0002 0.0002 0.0003 0.0003 0.0003 0.0003 ... Columns 73 through 80 0.0015 0.0015 0.0015 0.0015 0.0015 0.0016 0.0016 0.0016	The time axis defined by the information signal

	Columns 81 through 83 0.0016 0.0016 0.0017	
`>> x = zeros(length(f), length(t));`	x = Columns 1 through 14 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ...	Initialize the information signal with zeros

`>> for i = 1:length (f)`
`x( i ,:) = cos(2*pi*f( i )*t);`
`end`
	x =
	
	Columns 1 through 8
	
	0.9980 0.9921 0.9823 0.9686 0.9511 0.9298 0.9048 0.8763
	0 0 0 0 0 0 0 0 ...
	
	0.7624 0.9202 0.9950 0.9511 0.8090 0.5878 0.2365 -0.0879 -0.4029 -0.7026 -0.8994 -0.9936 -0.9745 -0.8375 -0.5979 -0.3209 0.0377 0.3914 0.6374 0.8763 0.9921
		Extraction of values in the information signal which is a superposition of cosine signals from 500 Hz to 3000 Hz with a step of 100 Hz

`>> signal_x = sum(x);`	signal_x = Columns 1 through 8 25.2613 23.1098 19.7326 15.4201 10.5360 5.4805 0.6502 -3.6003 Columns 9 through 16 -6.9914 -9.3426 -10.5850 -10.7618 -10.0159 -8.5686 -6.6899 -4.6657 ... Columns 73 through 80 1.2706 0.8654 0.4813 0.1751 -0.0092 -0.0466 0.0658 0.3090 Columns 81 through 83 0.6455 1.0246 1.3905	The silent information signal as a superposition of cosine signals

`>> noise1 = 10`	noise1 = 10	Noise 10 db
`>> y = awgn( x, noise1 );`	y = Columns 1 through 8 0.8101 0.8960 0.5944 1.0941 1.0868 0.9087 0.9553 0.8260 0.7258 0.9946 0.7112 1.2528 0.5375 1.2995 0.8158 0.4454 0.5162 1.2856 1.0751 1.2276 0.8481 0.6079 1.0008 0.2542 1.3589 1.3951 1.0733 1.0728 0.8036 0.2462 0.7650 0.2620 ... 0.6410 0.8338 1.1178 1.4662 0.8776 0.8841 0.1628 0.3763 -1.0197 -0.7486 -0.6561 -1.1883 -0.6614 -0.5213 -0.2318 -0.4739 0.1740 0.5796 1.2332 0.3974 1.0675	Add 10 db white additive Gaussian noise to the original signal x , by calling the awgn function
`>> signal_y = sum(y)`	signal_y = Columns 1 through 8 22.9002 22.6860 20.0765 16.1907 9.0868 4.5521 -1.2668 -3.5010 Columns 9 through 16 -5.9003 -8.9531 -10.0514 -12.5054 -7.9160 -10.0520 -6.1747 -3.5301 ... Columns 73 through 80 1.6906 -0.9508 0.6476 0.0166 1.1849 -0.0844 1.3084 0.6932 Columns 81 through 83 3.0925 2.1635 -1.8766	The noisy information signal by 10 db , as a superposition of cosine signals
`>> noise2 = 5`	noise2 = 5	Noise 5 db
`>> z = awgn( x, noise2 );`	z =	Add 5 db white additive Gaussian noise to the original

	Columns 1 through 8 0.7520 1.2249 0.0264 0.5428 0.2987 0.7918 0.5805 0.4221 1.3673 1.1399 1.4717 0.4031 1.3662 1.0128 0.4552 1.5135 1.4038 0.6589 0.8377 0.6981 -0.1465 1.0755 0.8121 1.0884 1.2937 0.3195 1.6573 1.9071 1.1470 -0.1203 0.8005 0.6694 0.2121 1.5155 0.6676 1.5626 1.6477 1.6615 0.6940 0.4016 0.8045 0.6421 0.4780 1.3705 0.5803 0.9875 0.3470 0.8359 0.4429 1.6927 0.2603 -0.0769 0.2456 0.3373 0.5621 -0.1473 ... -0.9936 -1.8569 -0.1212 0.4319 0.3906 0.4593 0.0548 0.5806 0.4440 0.8020 0.8485 0.9838 -0.1755 0.6588 -1.4354 -1.0890 -1.1063 -1.0711 -1.2186 -1.0090 0.4418 -1.2100 -0.0064 0.7656 0.8188 1.2589 1.6005	signal x , by calling the awgn function
`>> signal_z = sum(z)`	signal_z = Columns 1 through 8 22.2142 23.5261 15.4212 18.9177 12.0164 2.8770 -1.1085 -0.8664 Columns 9 through 16 -10.5473 -7.4706 -11.9253 -6.7793 -6.5683 -9.7836 -7.8709 -1.4459 ... Columns 73 through 80 0.6216 4.4685 2.6095 -3.3365 4.5727 -0.7240 -0.5830 0.4182 Columns 81 through 83 -3.4926 1.1024 1.1287	The noisy information signal by 5 db , as a superposition of cosine signals

`>> sound( signal_x );`
`>> pause( 1 );`
`>> sound( signal_y );`
`>> pause( 1 );`
`>> sound(signal_z);`
		Hearing the three information signals in succession one after the other with a 1 second pause

>> figure >> subplot (3, 1, 1) >> plot(t, signal_x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Silent mark '); >> subplot(3, 1, 2) >> plot(t, signal_y) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title ('Noisy signal with 10 db noise '); >> subplot (3, 1, 3) >> plot(t, signal_z) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title ('Noisy signal with 5 db noise ');	Figure 5.1	The plotting of the three information signals in the time domain defined. Add a title and labels to the horizontal x- axis and vertical y- axis for each graph. The graphs are displayed in the same window (figure).
>> BW = 1/dt	BW = 5.0000e+04	Bandwidth defined by sampling in time
>> N_x = length(signal_x)	N_x = 83	The magnitude of the noise signal as a superposition of cosine signals
>> df_x = BW/N_x	df_x = 602.4096	The sampling of the noise signal in the frequency domain
>> f_x = -BW/2:df_x:BW/2-df_x	f_x = 1.0e+04 * Columns 1 through 8 -2.5000 -2.4398 -2.3795 -2.3193 -2.2590 -2.1988 -2.1386 -2.0783 Columns 9 through 16 -2.0181 -1.9578 -1.8976 -1.8373 -1.7771 -1.7169 -1.6566 -1.5964 ... Columns 73 through 80 1.8373 1.8976 1.9578 2.0181 2.0783 2.1386 2.1988 2.2590	The table of frequencies that defines the silent signal

	Columns 81 through 83 2.3193 2.3795 2.4398	
>> ph_x = fft (signal_x) 	ph_x = 1.0e+02 * Columns 1 through 4 -0.1786 + 0.0000i 0.9644 + 0.9143i 1.0712 + 0.2185i 1.1258 - 0.0433i ... Columns 77 through 80 0.2162 + 0.5629i 0.2938 + 0.7615i 1.2417 + 1.3527i 1.2138 + 0.3279i Columns 81 through 83 1.1258 + 0.0433i 1.0712 - 0.2185i 0.9644 - 0.9143i	The amplitude spectrum of the noise signal calculated by the fast Fourier transform
>> ph_x = fftshift (ph_x)/ N_x 	ph_x = Columns 1 through 8 -0.0305 -0.0336 -0.0337 -0.0307 -0.0250 -0.0173 -0.0086 0.0000 Columns 9 through 16 0.0074 0.0128 0.0157 0.0158 0.0135 0.0092 0.0039 -0.0017 ... Columns 73 through 80 -0.0383 -0.0256 -0.0139 -0.0046 0.0013 0.0035 0.0018 -0.0029 Columns 81 through 83 -0.0097 -0.0175 -0.0248	The amplitude spectrum of the noise signal calculated from the fast Fourier transform , symmetric about the vertical axis
>> N_y = length(signal_y) 	N_y = 83	The magnitude of the noise by 10 db , as a superposition of cosine signals
>> df_y = BW/N_y 	df_y = 602.4096	The sampling of the noise by 10 db , signal in the frequency domain
>> f_y = -BW/2:df_y:BW/2-df_y 	f_y = 1.0e+04 * Columns 1 through 8	The table of frequencies defined the noise by 10 db signal

	-2.5000 -2.4398 -2.3795 -2.3193 -2.2590 -2.1988 -2.1386 -2.0783 ... Columns 73 through 80 1.8373 1.8976 1.9578 2.0181 2.0783 2.1386 2.1988 2.2590 Columns 81 through 83 2.3193 2.3795 2.4398	
`>> ph_y = fft ( signal_y )`	ph_y = 1.0e+02 * Columns 1 through 4 -0.3251 + 0.0000i 1.0101 + 0.9816i 1.0419 + 0.1359i 1.0707 + 0.0374i ... Columns 77 through 80 0.1017 + 0.5052i 0.3760 + 0.5763i 1.1582 + 1.4028i 1.2088 + 0.1883i Columns 81 through 83 1.0707 - 0.0374i 1.0419 - 0.1359i 1.0101 - 0.9816i	db noise amplitude spectrum of a signal calculated by the fast Fourier transform
`>> ph_y = fftshift ( ph_y )/ N_y`	ph_y = Columns 1 through 8 -0.0005 -0.0319 -0.0642 -0.0198 -0.0462 -0.0284 -0.0218 -0.0097 Columns 9 through 16 0.0213 0.0204 -0.0152 0.0290 0.0524 0.0148 0.0246 -0.0065 ... Columns 73 through 80 -0.0027 -0.0451 -0.0269 -0.0264 -0.0185 0.0155 -0.0078 0.0265 Columns 81 through 83 -0.0412 -0.0150 -0.0199	10 db noise amplitude spectrum of a signal calculated from the fast Fourier transform , symmetric about the vertical axis

`>> N_z = length( signal_z )`	N_z = 83	The magnitude of the noise by 5 db , as a superposition of cosine signals
`>> df_z = BW/N_z`	df_z = 602.4096	The sampling of the noise by 5 db signal in the frequency domain
`>> f_z = -BW/2:df_z:BW/2-df_z`	f_z = 1.0e+04 * Columns 1 through 8 -2.5000 -2.4398 -2.3795 -2.3193 -2.2590 -2.1988 -2.1386 -2.0783 ... Columns 73 through 80 1.8373 1.8976 1.9578 2.0181 2.0783 2.1386 2.1988 2.2590 Columns 81 through 83 2.3193 2.3795 2.4398	The table of frequencies defined the noise by 5 db signal
`>> ph_z = fft ( signal_z )`	ph_z = 1.0e+02 * Columns 1 through 4 -0.3714 + 0.0000i 0.9789 + 1.0479i 1.1951 + 0.1574i 0.9442 - 0.0003i ... Columns 77 through 80 0.4893 + 0.7939i 0.1724 + 0.6958i 1.2580 + 1.4488i 0.7754 + 0.2862i Columns 81 through 83 0.9442 + 0.0003i 1.1951 - 0.1574i 0.9789 - 1.0479i	The amplitude spectrum of the 5 db noise , signal calculated by the fast Fourier transform
`>> ph_z = fftshift ( ph_z )/ N_z`	ph_z = Columns 1 through 8 -0.1020 -0.0897 -0.0157 -0.0462 -0.0184 0.0050 0.0379 -0.0132 Columns 9 through 16	db noise amplitude spectrum of a signal calculated from the fast Fourier transform , symmetric about the vertical axis

	0.0173 0.0486 0.0104 -0.0116 0.0726 0.0127 0.0123 -0.0131 ... Columns 73 through 80 -0.0354 -0.0208 0.0113 -0.0442 0.0208 0.0461 -0.0309 -0.0156 Columns 81 through 83 0.0425 -0.0175 -0.0092	
>> figure >> subplot(3, 1, 1) >> plot(f_x , abs(ph_x)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Quiet Signal Width Spectrum'); >> subplot(3, 1, 2) >> plot(f_y , abs(ph_y)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('10db Noise Signal Width Spectrum'); >> subplot(3, 1, 3) >> plot(f_z , abs(ph_z)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('5db Noise Sig-nal Width Spectrum');	Figure 5.2	The plotting of the amplitude spectra of the three information signals in the time domain defined. Add a title and labels to the horizontal x- axis and vertical y- axis for each graph. The graphs are displayed in the same window (figure).

Graphic representations

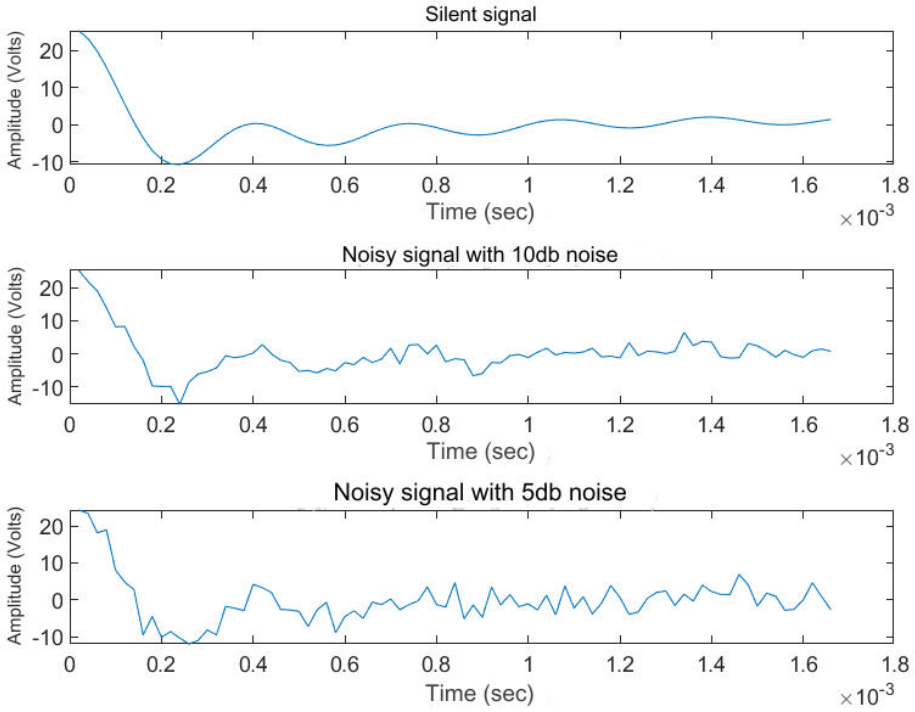

Figure 5.1. The graphs of the three modulated signals

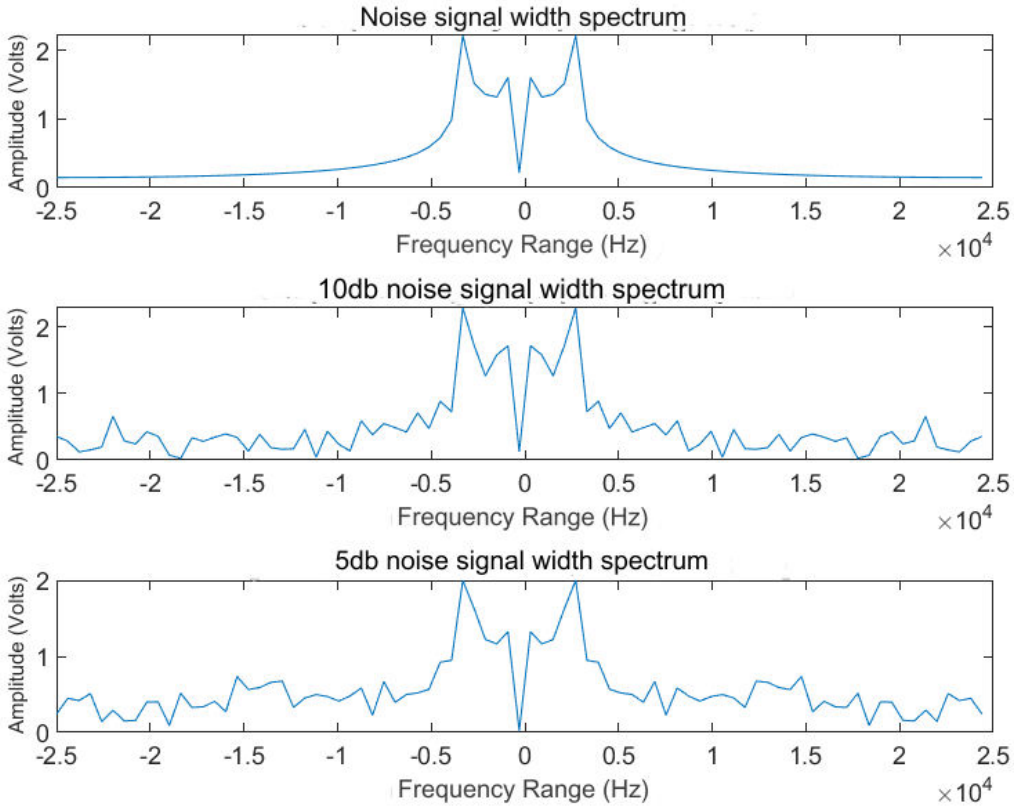

Figure 5.2. The plots of the amplitude spectra of the three modulated signals

EXERCISE 6

Spectrum of melody. Given the following Matlab program. Listen to the signal (xx(t)), to which musical notes do the frequencies of which it is composed correspond and to which well-known melody does the signal correspond? Make it the spectrum of signal melody .

```
fs=6000;
ts = 1/fs;
t=0:ts:1;
fr =[1567 1760 1975 1046];
x=zeros(length( fr ), length(t));
for i =1:length( fr )
x( i ,:)=cos(2*pi* fr ( i )*t);
end xx=[x(1,:) x(2,:) x(3,:) x(4,:)]; plot([0:ts:4+3* ts ],xx)
sound( xx,fs )
```

Code at Matlab

```
fs = 6000
ts = 1/fs
t = 0:ts:1
fr = [1567 1760 1975 1046]
x = zeros(length( fr ), length(t))
for i = 1:length ( fr )
 x( i ,:) = cos(2*pi* fr ( i )*t)
end
xx = [ x( 1,:) x(2,:) x(3,:) x(4,:)]
subplot( 2 , 1 , 1 );
plot([0:ts:4+3* ts ],xx )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Sign melody ' );

sound( xx,fs )

N = length(xx)
dt = ts /100
BW = 1/dt
df = BW/N
Start = -BW/2
Step = df
End = BW/2-df
f = Start:Step :End
ph = fft (xx)
ph = fftshift ( ph )/N
subplot( 2 , 1 , 2 );
plot( f, abs( ph ))
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Spectrum melody ' );
```

Explanation of the commands used

Commands	Results	Comments
`>> fs = 6000`	fs = 6000	The sampling frequency
`>> ts = 1/fs`	ts = 1.6667e-04	The sampling period
`>> t = 0:ts:1`	t = Columns 1 through 5 0 0.0002 0.0003 0.0005 0.0007 Columns 6 through 10 0.0008 0.0010 0.0012 0.0013 0.0015 … Columns 5,996 through 6,000 0.9992 0.9993 0.9995 0.9997 0.9998 Column 6.001 1,0000	The period of time that the melody signal is set
`>> fr = [1567 1760 1975 1046]`	fr = 1567 1760 1975 1046	A vector with musical notes
`>> x = zeros(length( fr ), length(t))`	x = … Columns 5,993 through 6,000 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 Column 6.001 0 0 0 0	Initialize the array that will contain the digitized notes, with zeros

`>> for i = 1:length ( fr )`
`x( i ,:) = cos(2*pi* fr ( i )*t)`
`end`
	x = …
	
	Columns 5,996 through 6,000
	
	-0.3437 0.9609 0.2089 -0.9902 -0.0701
		Assigning the notes in cosine format in order to combine them to reproduce the melody signal

	-0.9781 0.4633 0.7290 -0.8554 -0.2689 -0.6088 -0.4067 0.9969 -0.5446 -0.4772 0.6921 -0.3249 -0.9896 -0.5810 0.4577 Column 6.001 1,0000 1,0000 1,0000 1,0000	
`>> xx = [x(1,:) x(2,:) x(3,:) x(4,:)]`	xx = … Columns 23,996 through 24,000 -0.7889 0.1853 0.9585 0.6921 -0.3249 Columns 24.001 through 24.004 -0.9896 -0.5810 0.4577 1.0000	Creating the melody signal

`>> subplot( 2, 1, 1 );`
`>> plot([0:ts:4+3* ts ],xx )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( ' Sign melody ' );`
	Figure 6 .1	Plotting the waveform of the melody signal in the specified time domain. Add title and labels to the horizontal x- axis and vertical y- axis .

`>> sound( xx,fs )`		Hearing the melody signal
`>> N = length(xx)`	N = 24004	The size of the melody signal
`>> dt = ts /100`	dt = 1.6667e-06	Sampling to be 1/100 of the signal period
`>> BW = 1/dt`	BW = 600000	The bandwidth defined by sampling in time
`>> df = BW/N`	df = 24.9958	Sampling in the frequency domain
`>> Start = -BW/2`	Start = -300000	The first frequency of the signal's width spectrum
`>> Step = df`	Step = 24.9958	The step of the signal amplitude spectrum
`>> End = BW/2-df`	End = 2.9998e+05	The last frequency of the signal's width spectrum

`>> f = Start:Step:End`	f = … Columns 23,996 through 24,000 2.9978 2.9980 2.9983 2.9985 2.9988 Columns 24.001 through 24.004 2.9990 2.9993 2.9995 2.9998	The range of frequencies defined by the amplitude spectrum of the signal
`>> ph = fft (xx)`	ph = … Columns 24.001 through 24.002 0.0040 - 0.0000i 0.0000 + 0.0000i Columns 24.003 through 24.004 0.0000 + 0.0000i - 0.0000 + 0.0000i	The amplitude spectrum of the signal from the fast Fourier transform
`>> ph = fftshift ( ph )/N`	ph = … Columns 24.001 through 24.002 0.0000 - 0.0000i - 0.0000 + 0.0000i Columns 24.003 through 24.004 -0.0000 - 0.0000i 0.0000 + 0.0000i	The amplitude spectrum of the signal from the fast Fourier transform , symmetric about the vertical axis

`>> subplot( 2, 1, 2 );`
`>> plot( f, abs( ph ))`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( ' Spectrum mel-ody ' );`
	Figure 6 .1	Plotting the amplitude spectrum of the signal waveform in the specified frequency domain. Add title and labels to the horizontal x- axis and vertical y- axis .

Answers to the sub-questions

The notes and melody correspond to Beethoven's classic tune entitled "The Unrivaled 5th Symphony ".

Graphic representations

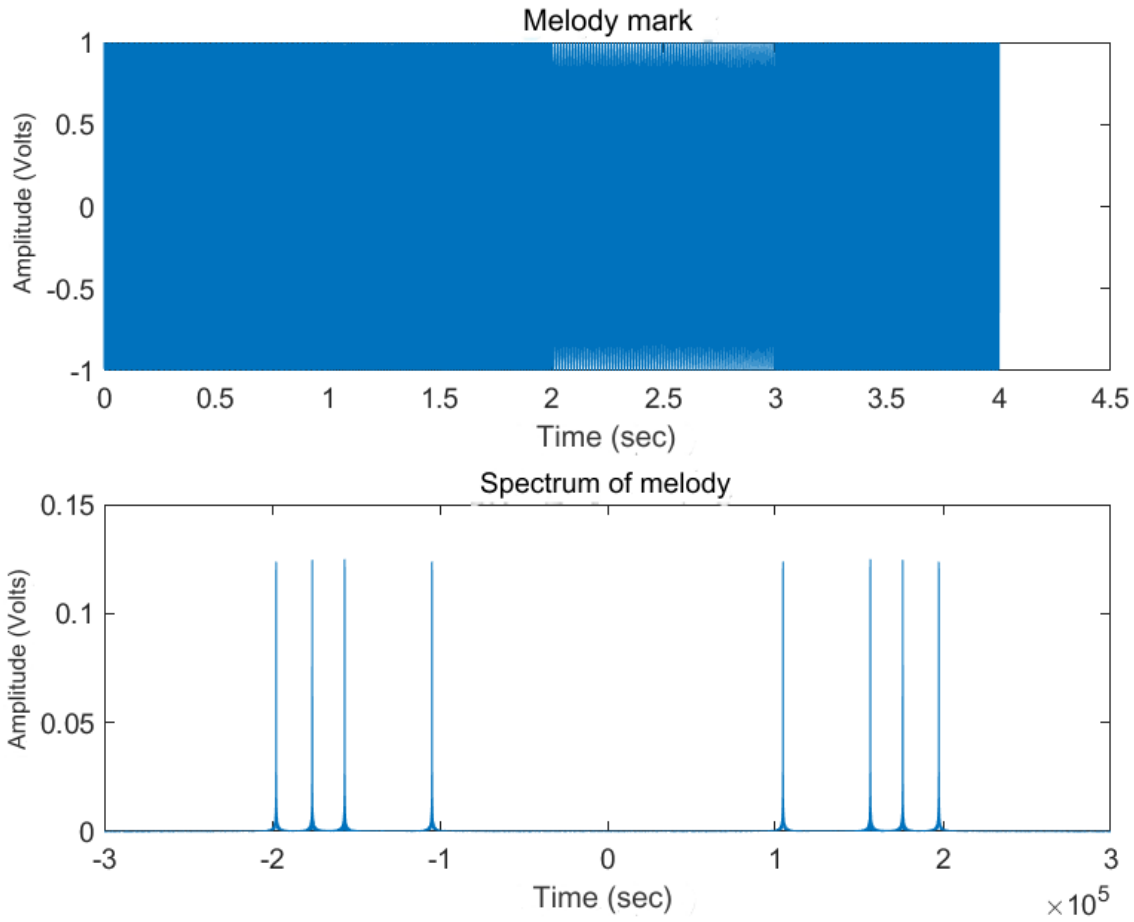

Figure 6.1. The waveform of the melody signal and the amplitude spectrum

EXERCISE 7

DSB configuration. To create an information signal consisting of the superposition of cosine signals from 500Hz with a step of 100Hz to 3000Hz.

100KHz frequency carrier. Perform DSB configuration. Then use the function that implements an ideal (quadratic) simplified bandpass filter:

```
function y = bpfilt ( signal, f1, f2,fs)
```

listed below and its arguments are, signal : the signal we want to filter, f1, f2 : the lower and upper frequency limit of the bandpass filter and fs : the sampling frequency.

Using the function generate a) a USB – upper sideband and b) an LSB – lower sideband modulated signal.

Plot the amplitude spectra of all signals.

Code at Matlab

```
Start = 500
Step = 100
End = 3000
f = Start:Step :End ;
fc = 1e5
Tc = 1/fc
dt = Tc/100
Ts = 1/3e3
tw = 5*Ts
t = dt:dt :tw ;

x = zeros(length(f), length(t));

for i = 1: length (f)
    x( i ,:) = cos(2*pi*f( i )*t);
end
signal = sum(x);
carrier = cos(2*pi*fc*t)
DSB = carrier.* signal
filter = bpfilt ( DSB , 500 , 3000 , 1/dt )

figure
subplot( 3, 1, 1 )
plot( t , signal )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Sign Information ' );
subplot( 3, 1, 2 )
plot( t , DSB )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB Signal Information ' );
subplot( 3, 1, 3 )
plot( t , filter )
xlabel ( ' Time (sec)' );
ylabel( 'Amplitude (Volt)' );
title( 'Filtered Information Signal' );

BW = 1/dt
N = length(t)
df = BW/N
f = -BW/2:df:BW/2-df;
ph_x = fft (signal);
ph_x = fftshift ( ph_x )/N;
ph_dsb = fft ( DSB )
ph_dsb = fftshift ( ph_dsb )/N
ph_filt = fft (filter)
ph_filt = fftshift ( ph_filt )/N

figure
subplot( 3, 1, 1 )
plot( f, abs( ph_x ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Information Signal Width Spectrum' );
subplot( 3, 1, 2 )
plot( f, abs( ph_dsb ))
```

```
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum of modulated information signal' );
subplot( 3, 1, 3 )
plot( f, abs( ph_filt ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum of filtered signal' );
```

Explanation of the commands used

Commands	Results	Comments
`>> Start = 500`	Start = 500	The first frequency of the information signal
`>> Step = 100`	Step = 100	The pitch of the information signal
`>> End = 3000`	End = 3000	The last frequency of the information signal
`>> f = Start:Step:End`	f = Columns 1 through 6 500 600 700 800 900 1000 Columns 7 through 12 1100 1200 1300 1400 1500 1600 Columns 13 through 18 1700 1800 1900 2000 2100 2200 Columns 19 through 24 2300 2400 2500 2600 2700 2800 Columns 25 through 26 2900 3000	The frequency domain of the information signal
`>> fc = 1e5`	fc = 100000	The carrier frequency
`>> Tc = 1/fc`	Tc =	The bearing period

	1.0000e-05	
>> dt = Tc/100	dt = 1.0000e-07	The elementary time shift dt , where must be 1/100 base of the fastest component (carrier frequency)
>> Ts = 1/3e3	Ts = 3.3333e-04	The period of the slowest component
>> tw = 5*Ts	tw = 0.0017	The observation duration based on the slowest component
>> t = dt:dt:tw	t = Columns 1 through 8 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 Columns 9 through 16 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 … Columns 16.657 through 16.664 0.0017 0.0017 0.0017 0.0017 0.0017 0.0017 0.0017 0.0017 Columns 16.665 through 16.666 0.0017 0.0017	The time axis defined by the information signal
>> x = zeros(length(f), length(t))	x = Columns 1 through 13 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ...	Initialize the information signal with zeros
>> for i = 1:length (f) x(i ,:) = cos(2*pi*f(i)*t) end	x = Columns 1 through 8 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 …	Extraction of values in the information signal which is a superposition of cosine signals from 500 Hz to 3000 Hz with a step of 100 Hz

	-0.5020 -0.5008 0.4979 0.4992 1.0000 1.0000 0.5023 0.5009 -0.4976 -0.4991 -1.0000 -1.0000 -0.5025 -0.5010 0 0 0 0	
`>> signal = sum(x)`	signal = Columns 1 through 8 26.0000 25.9999 25.9998 25.9997 25.9995 25.9993 25.9991 25.9988 Columns 9 through 16 25.9985 25.9981 25.9977 25.9973 25.9969 25.9964 25.9958 25.9952 … Columns 16.657 through 16.664 1.4846 1.4862 1.4878 1.4894 1.4910 1.4926 1.4942 1.4958 Columns 16.665 through 16.666 1.4974 1.4989	The silent information signal as a superposition of cosine signals
`>> carrier = cos(2*pi*fc*t)`	carrier = Columns 1 through 8 0.9980 0.9921 0.9823 0.9686 0.9511 0.9298 0.9048 0.8763 Columns 9 through 16 0.8443 0.8090 0.7705 0.7290 0.6845 0.6374 0.5878 0.5358 ... Columns 16.657 through 16.664 -0.9048 -0.8763 -0.8443 -0.8090 - 0.7705 -0.7290 -0.6845 -0.6374 Columns 16.665 through 16.666 -0.5878 -0.5358	The cosine signal with the carrier frequency
`>> DSB = carrier.*signal`	DSB = Columns 1 through 8	The modulated DSB information signal

	25.9487 25.7949 25.5393 25.1829 24.7270 24.1736 23.5247 22.7829 Columns 9 through 16 21.9513 21.0329 20.0316 18.9512 17.7961 16.5707 15.2800 13.9289 ... Columns 16.657 through 16.664 -1.3434 -1.3024 -1.2562 -1.2050 -1.1489 -1.0881 -1.0229 -0.9534 Columns 16.665 through 16.666 -0.8801 -0.8032	
`>> filter = bpfilt(DSB, 500, 3000, 1/dt)`	filter = -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 -0.0162 ...	The filtered USB information signal by calling the bpfilt function

`figure`
`subplot( 3, 1, 1 )`
`plot( t , signal )`
`xlabel ( ' Time (sec)' );`
`ylabel ( ' Amplitude (Volt)' );`
`title( ' Sign Information ' );`
`subplot( 3, 1, 2 )`
`plot( t , DSB )`
`xlabel ( ' Time (sec)' );`
`ylabel ( ' Amplitude (Volt)' );`
`title( 'DSB Signal Information ' );`
`subplot( 3, 1, 3 )`
`plot( t , filter )`
`xlabel ( ' Time (sec)' );`
`ylabel( 'Amplitude (Volt)' );`
`title( 'Filtered Information Signal' );`
	Figure 7.1	The plotting of the waveforms of the three signals in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis . The graphics are displayed together in the same window (figure)

`>> BW = 1/dt`	BW = 10000000	The bandwidth defined by sampling in time
`>> N = length(t)`	N = 16666	The size of the array of times the signals are defined
`>> df = BW/N`	df = 600.0240	Sampling in the frequency domain
`>> f = -BW/2:df:BW/2-df`	f = 1.0e+06 * Columns 1 through 8 -5.0000 -4.9994 -4.9988 -4.9982 -4.9976 -4.9970 -4.9964 -4.9958 ... Columns 16.657 through 16.664 4.9940 4.9946 4.9952 4.9958 4.9964 4.9970 4.9976 4.9982 Columns 16.665 through 16.666 4.9988 4.9994	The table of frequencies
`>> ph_x = fft(signal)`	ph_x = 1.0e+04 * Columns 1 through 4 -0.1034 + 0.0000i 2.3003 + 1.6864i 2.4305 + 0.1075i 2.4234 - 0.5958i ... Columns 16.661 through 16.664 0.1007 + 1.6452i 1.6578 + 3.4202i 2.3756 + 1.3567i 2.4234 + 0.5958i Columns 16.665 through 16.666 2.4305 - 0.1075i 2.3003 - 1.6864i	The amplitude spectrum of the information signal from the fast Fourier transform
`>> ph_x = fftshift ( ph_x )/N`	ph_x = Columns 1 through 4 0.0007 + 0.0000i 0.0007 + 0.0000i 0.0007 + 0.0000i 0.0007 + 0.0000i Columns 5 through 8	The amplitude spectrum of the information signal from the fast Fourier transform , symmetric about the vertical axis

	0.0007 + 0.0000i 0.0007 + 0.0000i 0.0007 + 0.0000i 0.0007 + 0.0000i ... Columns 16.661 through 16.664 -0.0034 - 0.0015i - 0.0034 - 0.0013i - 0.0034 - 0.0010i -0.0034 - 0.0008i Columns 16.665 through 16.666 -0.0034 - 0.0005i -0.0034 - 0.0003i	
`>> ph_dsb = fft(DSB)`	ph_dsb = Columns 1 through 4 0.0008 + 0.0000i 0.0008 + 0.0000i 0.0008 + 0.0000i 0.0008 + 0.0000i Columns 5 through 8 0.0008 + 0.0000i 0.0008 + 0.0000i 0.0008 + 0.0000i 0.0008 + 0.0000i ...	The amplitude spectrum of the modulated information signal from the fast Fourier transform
`>> ph_dsb = fftshift ( ph_dsb )/N`	ph_dsb = ... Columns 16.661 through 16.664 0.0008 - 0.0000i 0.0008 - 0.0000i 0.0008 - 0.0000i 0.0008 - 0.0000i Columns 16.665 through 16.666 0.0008 - 0.0000i 0.0008 - 0.0000i	The amplitude spectrum of the modulated information signal from the fast Fourier transform , symmetric about the vertical axis
`>> ph_filt = fft(filter)`	ph_filt = -0.0000 + 0.0000i -33.7611 + 2.5465i -33.7662 + 5.0936i -33.7748 + 7.6418i -33.7868 +10.1917i ...	The amplitude spectrum of the filtered information signal from the fast Fourier transform
`>> ph_filt = fftshift ( ph_filt )/N`	ph_filt = ... -0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 + 0.0000i 0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 + 0.0000i	The amplitude spectrum of the filtered information signal from the fast Fourier transform , symmetric about the vertical axis

>> figure >> subplot(3, 1, 1) >> plot(f, abs(ph_x)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Information Signal Width Spectrum'); >> subplot(3, 1, 2) >> plot(f, abs(ph_dsb)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum of modulated information signal'); >> subplot(3, 1, 3) >> plot(f, abs(ph_filt)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum of filtered signal');	Figure 7.2	The plotting of the waveforms of the amplitude spectra of the three signals in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis . The graphics are displayed together in the same window (figure)

Graphic representations

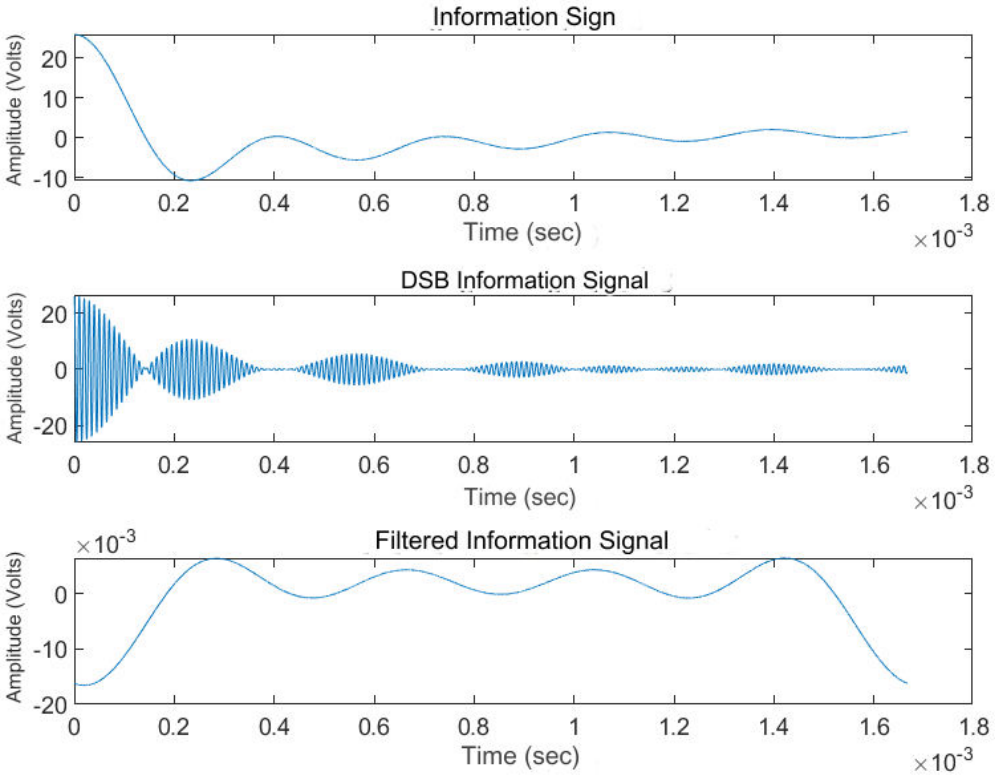

Figure 7.1. The graphical representations of the three signals

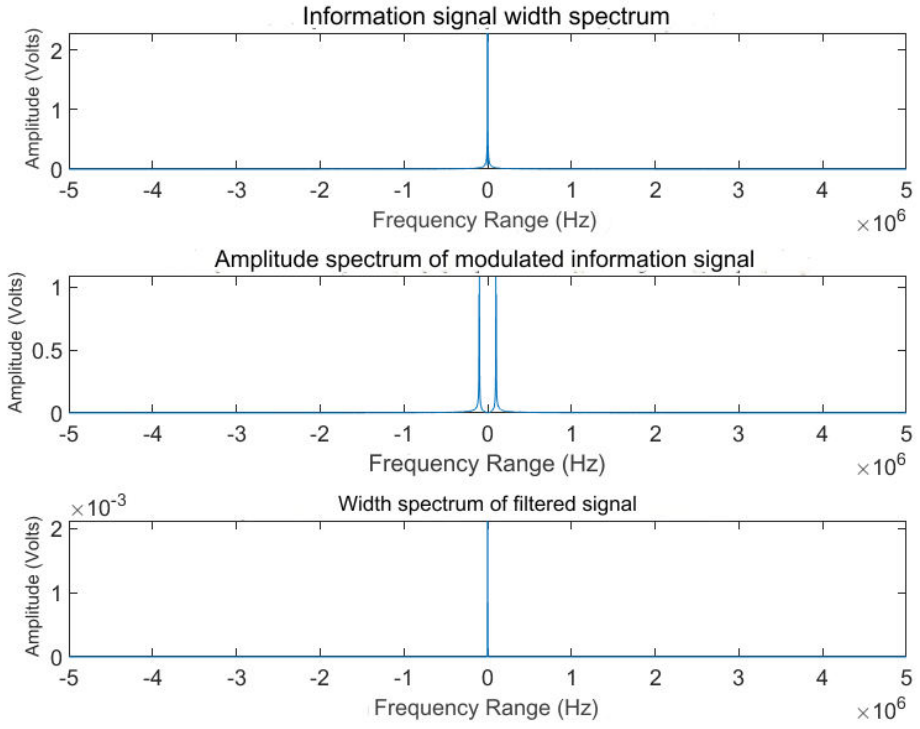

Figure 7.2. The plots of the amplitude spectra of three signals

EXERCISE 8

Aliasing effect. Representation of the aliasing effect due to undersampling. a) Create a 1KHz sinusoidal signal, lasting 2 sec. To sample the signal with 20KHz and then with 1.5KHz. Plot both signals on the same graph. b) Listen to the two signals with the soundsc(x,fs) function. What do you notice? c) Generate a sine signal that will produce the same sound as the sine sampled at 1.5KHz.

Code in Matlab

```
f = 1 e 3
t = 2

fs_x = 2e4
dt_x = 1/ fs_x
Start_x = 0
Step_x = dt_x
End_x = t
t_x = Start_ x:Step _x:End_x
x = sin(2*pi*f* t_x )

fs_y = 1.5e3
dt_y = 1/ fs_y
Start_y = 0
Step_y = dt_y
End_y = t
t_y = Start_ y:Step _y:End_y
y = sin(2*pi*f* t_y )

dt_sine = 1/2e3
start_sine = 0
Step_sine = dt_sine
End_sine = t
t_sine = Start_ sine:Step _sine:End_sine
sine = sin(2*pi*1.5e3* t_sine )

soundsc ( x, fs_x );
pause(t);
soundsc ( y, fs_y );
pause(t);
soundsc ( desired_signal , 1.5e3 );

subplot( 2 , 1 , 1 );
hold on
plot( t_x , x, 'b' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Aliasing Effect ' );
plot( t_y , y, 'r' )
legend( '20KHz signal ' , '1.5KHz signal ' );
hold off
subplot( 2 , 1 , 2 );
plot( t_sine , sine )
```

```
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'Sine signal' );
```

Explanation of the commands used

Commands	Results	Comments
`>> f = 1e3`	f = 1000	The frequency of the signal
`>> t = 2`	t = 2	The time duration of the signal
`>> fs_x = 2e4`	fs_x = 20000	The frequency of the sampled signal x (20K Hz)
`>> dt_x = 1/ fs_x`	dt_x = 5.0000e-05	The sampling of the signal x
`>> Start_x = 0`	Start_x = 0	The time origin at which the signal starts in the time field
`>> Step_x = dt_x`	Step_x = 5.0000e-05	The pitch of the signal
`>> End_x = t`	End_x = 2	The time when the signal completes a period
`>> t_x = Start_ x:Step _x:End_x`	t_x = … Columns 39,993 through 40,000 1.9996 1.9996 1.9997 1.9997 1.9998 1.9998 1.9999 1.9999 Column 40.001 2,0000	The range of times in which the signal waveform will be defined
`>> x = sin(2*pi*f* t_x )`	x = … Columns 39,993 through 40,000 -0.5878 -0.8090 -0.9511 -1.0000 -0.9511 -0.8090 -0.5878 -0.3090 Column 40.001 -0.0000	The sampled signal x (20K Hz)
`>> fs_y = 1.5e3`	fs_y =	The frequency of the sampled signal y (1.5K Hz)

	1500	
`>> dt_y = 1/ fs_y`	dt_y = 6.6667e-04	The sampling of the signal y
`>> Start_y = 0`	Start_y = 0	The time origin at which the signal starts in the time field
`>> Step_y = dt_y`	Step_y = 6.6667e-04	The pitch of the signal
`>> End_y = t`	End_y = 2	The time when the signal completes a period
`>> t_y = Start_ y:Step _y:End_y`	t_y = … Columns 2,993 through 3,000 1.9947 1.9953 1.9960 1.9967 1.9973 1.9980 1.9987 1.9993 Column 3.001 2,0000	The range of times in which the signal waveform will be defined
`>> y = sin(2*pi*f* t_y )`	y = … Columns 2,993 through 3,000 -0.8660 0.8660 -0.0000 -0.8660 0.8660 -0.0000 -0.8660 0.8660 Column 3.001 -0.0000	The sampled signal y (20K Hz)
`>> dt_sine = 1/2e3`	dt_sine = 5.0000e-04	The sampling of the sine signal
`>> Start_sine = 0`	start_sine = 0	The time origin at which the signal starts in the time field
`>> Step_sine = dt_sine`	Step_sine = 5.0000e-04	The pitch of the signal
`>> End_sine = t`	End_sine = 2	The time when the signal completes a period
`>> t_sine = Start_ sine:Step _sine:End_sine`	t_sine = … Columns 3,993 through 4,000 1.9960 1.9965 1.9970 1.9975 1.9980 1.9985 1.9990 1.9995 Column 4.001	The range of times in which the signal waveform will be defined

	2,0000	
>> sine = sin(2*pi*1.5e3* t_sine)	sine = … Columns 3,993 through 4,000 -0.0000 -1.0000 -0.0000 1.0000 0.0000 -1.0000 0.0000 1.0000 Column 4.001 -0.0000	sine signal that has the same sound as the sampled y signal (1.5K Hz)
>> soundsc (x, fs_x); >> pause(t); >> soundsc (y, fs_y); >> pause(t); >> soundsc(desired_signal, 1.5e3);		Hearing the 3 signals in succession with a pause of 2 seconds, subject to establishing the comparison
>> subplot(2, 1, 1); >> hold on ? >> plot(t_x , x, 'b') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Aliasing Effect '); >> plot(t_y , y, 'r') legend('20KHz signal ' , '1.5KHz signal '); >> hold off ? >> subplot(2, 1, 2); >> plot(t_sine , sine) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('Sine signal')	Figure 8.1	The plotting of the two sampled signals in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis . The signals are represented in the same shape and with different colors indicated in a new label. The plotting of the sinusoidal signal in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis .

Answers to the sub-questions

The signal sampled at 20 KHz sounds sharper than the signal at 1.5 KHz and this is because the frequency of the signal f = 1000 Hz is less than the sampling frequency by half for the first signal (fs /2 = 20000→ fs /2 = 10000 Hz > f = 1000 Hz). The result is that there are false frequencies due to aliasing .

Graphic representations

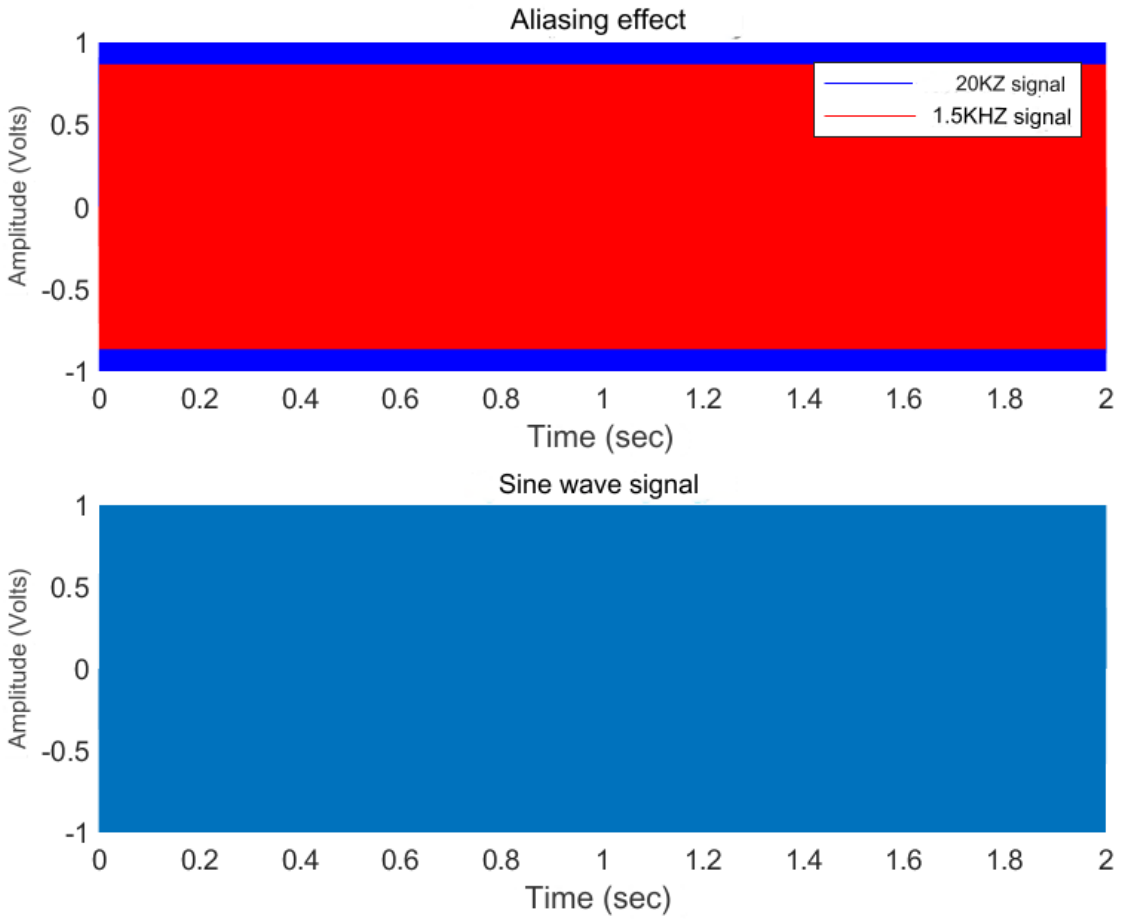

Figure 8.1. The sampled signals with a frequency of 20KHz and 1.5KHz , and the sinusoidal signal that produces the same sound as the 1.5KHz signal

EXERCISE 9

AM configuration. Generate an AM signal x1(t) with the following parameters: Ac =1 Volt, Am=0.5 Volt,fm=2 Hz, fc=100Hz, when the baseband signal is s(t)= Am sin(2πfmt). Consider t=[0:0.001:1].
Plot the modulated signal with plot(t,x1). Also graph the envelope of the signal.
Modulate the same signal s(t) with the ammod() function of Matlab's communication toolbox and name the signal x2(t).

What kind of configuration does ammod implement?
Pension ammod : y = ammod (s, fc, fs)
y: modulated signal
s: information signal
fc : carrier frequency
fs : sampling frequency - set fs=400

Code at Matlab

```
Ac = 1
Am = 0.5
fm = 2
fc = 100
fs = 400
t = 0:0.001:1
s = Am*sin(2*pi* fm *t)
carrier = Ac*cos(2*pi*fc*t)
x1 = carrier.* s
x2 = ammod ( s , fc , 400 )

subplot( 3, 1, 1 )
plot( t , s )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 's(t) = 0.5*sin(2*pi*2*t)' );
subplot( 3, 1, 2 )
plot( t, x1 )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB carrier signal .*s(t) = [1*cos(2*pi*100*t)].*[0.5*sin(2*pi*2*t)]' );
subplot( 3, 1, 3 )
plot( t, x2 )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB Signal Basic Zone ammod ()' );
```

Explanation of the commands used

Commands	Results	Comments
>> Ac = 1	Ac = 1	The width of the carrier signal
>> Am = 0.5	Am = 0.5000	The width of the baseband signal
>> fm = 2	fm = 2	The frequency of the baseband signal
>> fc = 100	fc = 100	The carrier frequency
>> fs = 400	fs = 400	The sampling frequency
>> t = 0:0.001:1	t = Columns 1 through 8 0 0.0010 0.0020 0.0030 0.0040 0.0050 0.0060 0.0070	The time domain defined by the baseband signal

	Columns 9 through 16 0.0080 0.0090 0.0100 0.0110 0.0120 0.0130 0.0140 0.0150 … Columns 993 through 1,000 0.9920 0.9930 0.9940 0.9950 0.9960 0.9970 0.9980 0.9990 Column 1.001 1,0000	
`>> s = Am*sin(2*pi* fm *t)`	s = Columns 1 through 8 0 0.0063 0.0126 0.0188 0.0251 0.0314 0.0377 0.0439 Columns 9 through 16 0.0502 0.0564 0.0627 0.0689 0.0751 0.0813 0.0875 0.0937 … Columns 993 through 1,000 -0.0502 -0.0439 -0.0377 -0.0314 -0.0251 -0.0188 -0.0126 -0.0063 Column 1.001 -0.0000	The baseband signal
`>> carrier = Ac*cos(2*pi*fc*t)`	carrier = Columns 1 through 8 1.0000 0.8090 0.3090 -0.3090 -0.8090 -1.0000 -0.8090 -0.3090 Columns 9 through 16 0.3090 0.8090 1.0000 0.8090 0.3090 -0.3090 -0.8090 -1.0000 … Columns 993 through 1,000 0.3090 -0.3090 -0.8090 -1.0000 -0.8090 -0.3090 0.3090 0.8090 Column 1.001 1,0000	The cosine signal with the carrier frequency

`>> x1 = carrier.*s`	x1 = Columns 1 through 8 0 0.0051 0.0039 -0.0058 -0.0203 -0.0314 -0.0305 -0.0136 Columns 9 through 16 0.0155 0.0457 0.0627 0.0557 0.0232 -0.0251 -0.0708 -0.0937 … Columns 993 through 1,000 -0.0155 0.0136 0.0305 0.0314 0.0203 0.0058 -0.0039 -0.0051 Column 1.001 -0.0000	The modulated DSB signal
`>> x2 = ammod(s, fc, 400)`	x2 = Columns 1 through 8 0 0.0000 -0.0126 -0.0000 0.0251 -0.0000 -0.0377 0.0000 Columns 9 through 16 0.0502 0.0000 -0.0627 0.0000 0.0751 -0.0000 -0.0875 0.0000 … Columns 993 through 1,000 -0.0502 0.0000 0.0377 0.0000 -0.0251 0.0000 0.0126 -0.0000 Column 1.001 -0.0000	SB modulated signal by calling the ammod function

`>> subplot( 3, 1, 1 )`
`>> plot( t, s )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 's(t) = 0.5*sin(2*pi*2*t)' );`
`>> subplot( 3, 1, 2 )`
`>> plot( t, x1 )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 'DSB signal`
	Figure 9.1	The plotting of the waveforms of the three signals in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis .

carrier.* s(t) = [1*cos(2*pi*100*t)].* [0.5*sin(2*pi*2*t)]'); >> subplot(3, 1, 3) >> plot(t, x2) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('DSB Signal Basic Zone ammod ()');		

Answers to the sub-questions

The ammod function implements Dual Sideband, amplitude modulation.

Graphic representations

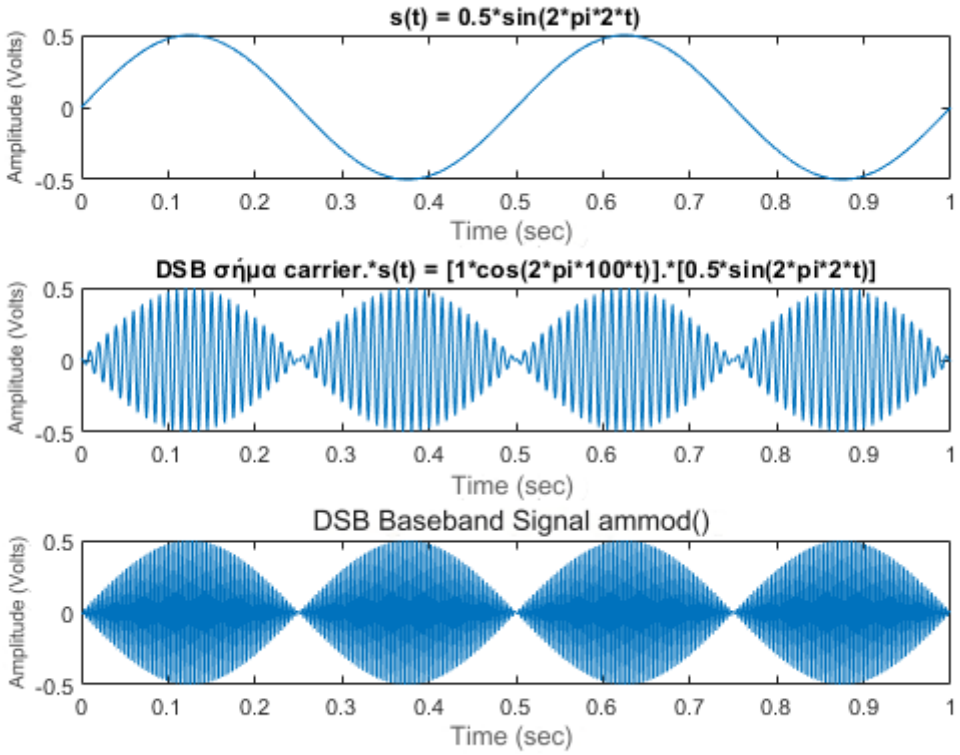

Figure 9.1. The graphical representations of the three signals

EXERCISE 10

Quantization. Create a sampled noise signal, normally distributed (use randn) with time axis t=0:0.001:.5. To be quantized a) in 8 and b) in 16 quantization levels. Plot the original, quantized, quantization level limits, quantization bands, and quantization error in the same graphics window

---

Code in Matlab

```
t = 0:0.001:.5
A = 1
signal = A* randn (size(t))
Nq = 8
step = A/(Nq/2)
partition = [- A+ step:step :A-step ]
codebook = [-A+(step/2 ): step :A -(step/2)]
[index, quants8, distor ] = quantiz ( signal, partition, codebook )
Nq = 16
step = A/(Nq/2)
partition = [- A+ step:step :A-step ]
codebook = [-A+(step/2 ): step :A -(step/2)]
[index, quants16, distor ] = quantiz ( signal, partition, codebook )

qerr8 = signal-quants8
qerr16 = signal-quants16

figure
subplot( 2, 1, 1 )
plot( t , signal )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Home mark ' );
hold on
stem( t, quants8, 'r*' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'The signal quantized to 8 levels' );
hold off
subplot( 2, 1, 2 )
plot( t , signal )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Home mark ' );
hold on
stem( t, quants16, 'r*' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'The signal quantized to 16 levels' );

figure
subplot( 2, 1, 1 )
plot( t, qerr8 )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'Quantization error at 8 levels' );
subplot( 2, 1, 2 )
plot( t, qerr16 )
xlabel ( ' Time (sec)' );
ylabel( 'Amplitude (Volt)' );
title( 'Quantization error at 16 levels' );
```

Explanation of the commands used

Commands	Results	Comments
`>> t = 0:0.001:.5`	t = Columns 1 through 8 0 0.0010 0.0020 0.0030 0.0040 0.0050 0.0060 0.0070 Columns 9 through 16 0.0080 0.0090 0.0100 0.0110 0.0120 0.0130 0.0140 0.0150 … Columns 489 through 496 0.4880 0.4890 0.4900 0.4910 0.4920 0.4930 0.4940 0.4950 Columns 497 through 501 0.4960 0.4970 0.4980 0.4990 0.5000	The time defined by the normally distributed sampled noise signal
`>> A = 1`	A = 1	The amplitude of the normally distributed noise signal
`>> signal = A* randn (size(t))`	signal = Columns 1 through 8 -0.3750 0.8205 -0.3749 0.8785 -0.9980 0.0705 -0.2905 -0.7783 Columns 9 through 16 0.0886 -1.4737 -1.6398 -1.4533 -0.8034 0.8426 0.0498 1.2480 … Columns 489 through 496 -1.9140 0.4958 -0.7042 -0.0773 -0.0999 0.3224 -1.3084 0.2371 Columns 497 through 501 -0.4174 -1.4898 -0.1419 -1.0460 2.8641	The sampled normally distributed noise signal by calling the randn function
`>> Nq = 8`	Nq = 8	Number of quantization levels
`>> step = A/(Nq/2)`	step =	Quantization step

	0.2500	
`>> partition = [- A+ step:step :A-step ]`	partition = -0.7500 -0.5000 -0.2500 0 0.2500 0.5000 0.7500	The vector that defines the quantization limits
`>> codebook = [-A+(step/2 ): step :A -(step/2)]`	codebook = -0.8750 -0.6250 -0.3750 -0.1250 0.1250 0.3750 0.6250 0.8750	The vector containing the quantization levels
`>> [index, quants8, distor ] = quantiz ( signal, partition, codebook )`	index = Columns 1 through 13 2 7 2 7 0 4 2 0 4 0 0 0 0 Columns 14 through 26 7 4 7 7 2 0 4 2 7 1 7 5 5 quants8 = Columns 1 through 8 -0.3750 0.8750 -0.3750 0.8750 -0.8750 0.1250 -0.3750 -0.8750 Columns 9 through 16 0.1250 -0.8750 -0.8750 -0.8750 -0.8750 0.8750 0.1250 0.8750 … Columns 489 through 496 -0.8750 0.3750 -0.6250 -0.1250 -0.1250 0.3750 -0.8750 0.1250 Columns 497 through 501 -0.3750 -0.8750 -0.1250 -0.8750 0.8750 distort = 0.2192	The 8-level quantized signal
`>> Nq = 16`	Nq = 16	Number of quantization levels
`>> step = A/(Nq/2)`	step = 0.1250	Quantization step
`>> partition = [- A+ step:step :A-step ]`	partition =	The vector that defines the quantization limits

	Columns 1 through 8 -0.8750 -0.7500 -0.6250 -0.5000 -0.3750 -0.2500 -0.1250 0 Columns 9 through 15 0.1250 0.2500 0.3750 0.5000 0.6250 0.7500 0.8750	
`>> codebook = [-A+(step/2): step :A -(step/2)]`	codebook = Columns 1 through 8 -0.9375 -0.8125 -0.6875 -0.5625 -0.4375 -0.3125 -0.1875 -0.0625 Columns 9 through 16 0.0625 0.1875 0.3125 0.4375 0.5625 0.6875 0.8125 0.9375	The vector containing the quantization levels
`>> [index, quants16, distor ] = quantiz ( signal, partition, codebook )`	index = Columns 1 through 13 5 14 5 15 0 8 5 1 8 0 0 0 1 Columns 14 through 26 14 8 15 15 4 0 8 5 15 2 15 10 11 … Columns 482 through 494 10 15 1 2 15 15 2 0 11 2 7 7 10 Columns 495 through 501 0 9 4 0 6 0 15 quants16 = Columns 1 through 8 -0.3125 0.8125 -0.3125 0.9375 -0.9375 0.0625 -0.3125 -0.8125 Columns 9 through 16 0.0625 -0.9375 -0.9375 -0.9375 -0.8125 0.8125 0.0625 0.9375 … Columns 489 through 496 -0.9375 0.4375 -0.6875 -0.0625 -0.0625 0.3125 -0.9375 0.1875	The signal quantized by 16 levels

	Columns 497 through 501 -0.4375 -0.9375 -0.1875 -0.9375 0.9375 distort = 0.1898	
>> qerr8 = signal-quants8	qerr8 = Columns 1 through 8 0.0000 -0.0545 0.0001 0.0035 -0.1230 -0.0545 0.0845 0.0967 Columns 9 through 16 -0.0364 -0.5987 -0.7648 -0.5783 0.0716 -0.0324 -0.0752 0.3730 … Columns 489 through 496 -1.0390 0.1208 -0.0792 0.0477 0.0251 -0.0526 -0.4334 0.1121 Columns 497 through 501 -0.0424 -0.6148 -0.0169 -0.1710 1.9891	The quantization error by 8 levels
>> qerr16 = signal-quants16	qerr16 = Columns 1 through 8 -0.0625 0.0080 -0.0624 -0.0590 -0.0605 0.0080 0.0220 0.0342 Columns 9 through 16 0.0261 -0.5362 -0.7023 -0.5158 0.0091 0.0301 -0.0127 0.3105 … Columns 489 through 496 -0.9765 0.0583 -0.0167 -0.0148 -0.0374 0.0099 -0.3709 0.0496 Columns 497 through 501 0.0201 -0.5523 0.0456 -0.1085 1.9266	The quantization error by 16 levels
>> figure >> subplot(2, 1, 1) >> stem(t, signal) >> xlabel (' Time (sec)'); 		Plotting the original signal quantized by 8 levels in the time domain defined.

>> ylabel (' Amplitude (Volt)'); >> title(' Home mark '); >> hold on ? >> stem(t, quants8, 'r*') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('The signal quantized in 8 levels'); >> legend('Original sig-nal' , 'Quantized to 8 levels') >> hold off ? >> subplot(2, 1, 2) >> stem(t, signal) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Home mark '); >> hold on ? >> stem(t, quants16, 'r*') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('The signal quantized at 16 levels'); >> legend('Original sig-nal' , 'Quantized to 16 levels')	Figure 10.1	The signals are represented in the same shape and with different colors indicated in a new label. The plotting of the original signal quantized by 16 levels in the time domain defined (in the same graphic subwindow figure). Add title and labels to the horizontal x- axis and vertical y- axis . The signals are represented in the same shape and with different colors indicated in a new label.
>> figure >> subplot(2, 1, 1) >> plot(t, qerr8) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('Quantization error at 8 levels'); >> subplot(2, 1, 2) >> plot(t, qerr16) >> xlabel (' Time (sec)'); >> ylabel('Amplitude (Volt)'); >> title('Quantization error at 16 levels');	Figure 10.2	The plots of the 8-level quantization error and 16-level quantization error in the time domain are defined. Add title and labels to the horizontal x-axis and vertical y- axis .

Graphic representations

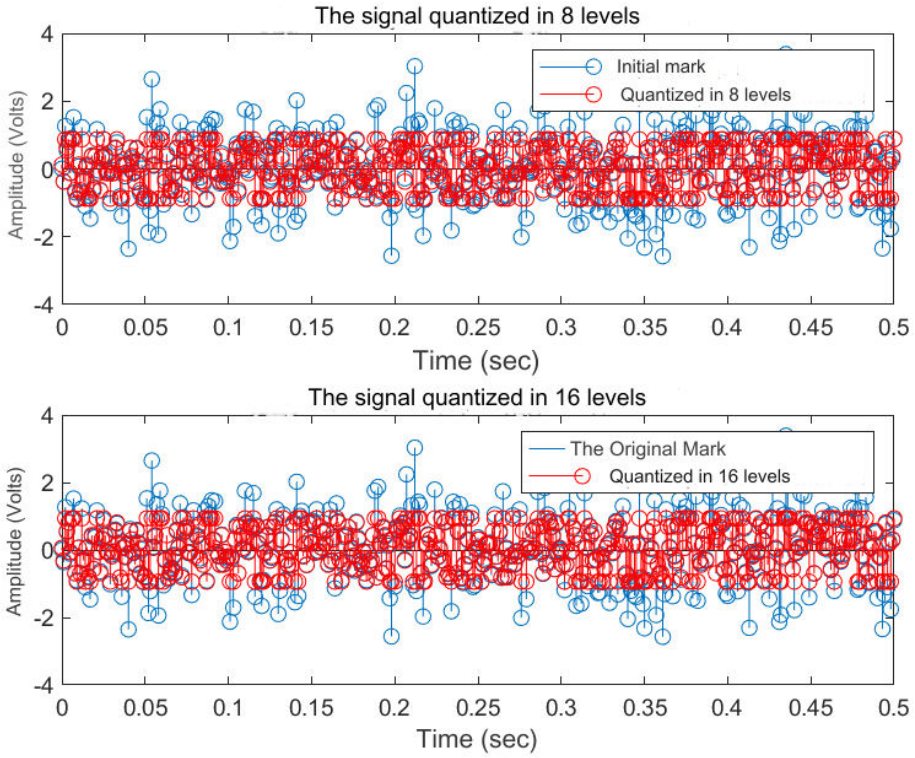

Figure 10.1. The original signal and the quantized by 8 levels and 16

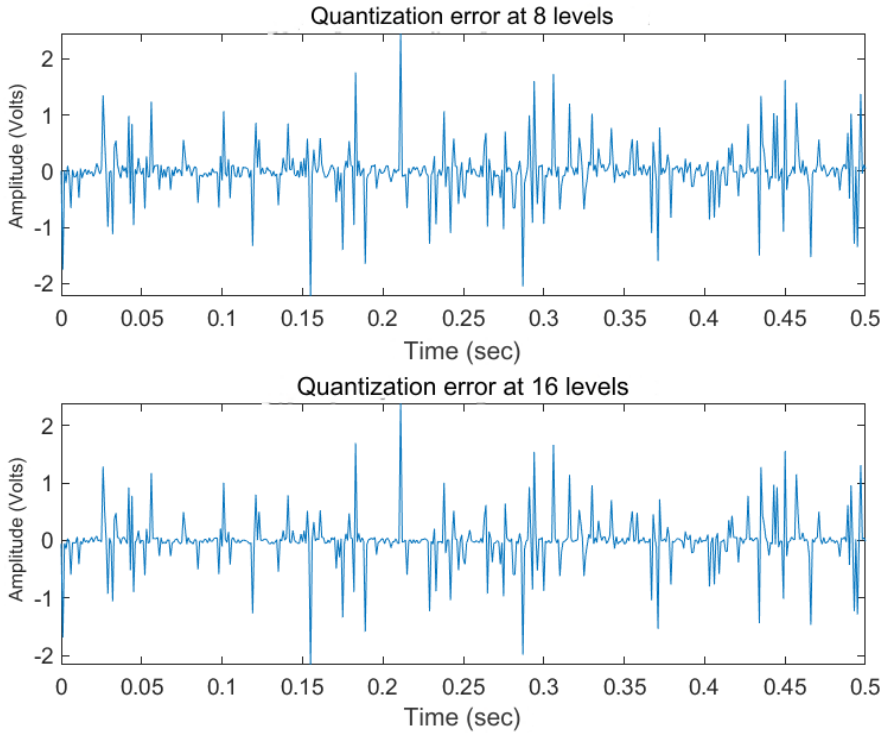

Figure 10.2. The quantization errors by 8 levels and 16

EXERCISE 11

Pulse code modulation. Create a program that will PCM modulate a sine signal of amplitude 4V and frequency 10 Hz. The quantizer will use a 4 bit code word.

Plot the original signal, the quantized signal and the quantization error

---

Code in Matlab

```
A = 4
f = 10
bits = 4
fs = 1e3
Ts = 1/fs
t = 0:Ts :1
pcm = A*cos(2*pi*f*t)
D = max( pcm ) - min( pcm ) / (2^bits)
partition = [-min( pcm )+ D:D:max ( pcm )+D]
codebook = [min( pcm )+(D/ 2): D :max ( pcm )+(D/2)]
[index, quantz , distort ] = quantiz ( pcm , partition, codebook)
qerr = pcm - quantz

subplot( 2, 1, 1 )
stem( t, pcm )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title ( 'Original by PCM signal' );
hold on
stem( t, quantz , 'r' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'The PCM quantized signal' );
legend( 'Original signal' , 'Quantized signal' );
hold off
subplot( 2, 1, 2 )
plot( t, qerr )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title ( 'Quantization error' );
```

Explanation of the commands used

Commands	Results	Comments
>> A = 4	A = 4	The amplitude of the PCM sine signal

`>> f = 10`	f = 10	The frequency of the PCM sine signal
`>> bits = 4`	bits = 4	The codeword bits used by the quantizer
`>> fs = 1e3`	fs = 1000	The sampling frequency of the PCM sine signal
`>> Ts = 1/fs`	Ts = 1.0000e-03	The sampling period of the PCM sine signal
`>> t = 0:Ts :1`	t = Columns 1 through 8 0 0.0010 0.0020 0.0030 0.0040 0.0050 0.0060 0.0070 Columns 9 through 16 0.0080 0.0090 0.0100 0.0110 0.0120 0.0130 0.0140 0.0150 … Columns 993 through 1,000 0.9920 0.9930 0.9940 0.9950 0.9960 0.9970 0.9980 0.9990 Column 1.001 1,0000	The time axis defined by the PCM sine signal
`>> pcm = A*cos(2*pi*f*t)`	pcm = Columns 1 through 8 4.0000 3.9921 3.9685 3.9291 3.8743 3.8042 3.7191 3.6193 Columns 9 through 16 3.5052 3.3773 3.2361 3.0821 2.9159 2.7382 2.5497 2.3511 … Columns 993 through 1,000 3.5052 3.6193 3.7191 3.8042 3.8743 3.9291 3.9685 3.9921 Column 1.001	The PCM sine signal

	4,0000	
`>> D = max( pcm ) - min( pcm ) / (2^bits)`	D = 4.2500	The quantization step
`>> partition = [-min( pcm )+ D:D:max ( pcm )+D]`	partition = 8.2500	The vector that defines the quantization limits
`>> codebook = [min( pcm )+(D/ 2): D :max ( pcm )+(D/2)]`	codebook = -1.8750 2.3750	The vector containing the quantization levels
`>> [index, quantz , distort ] = quantiz ( pcm , partition, codebook)`	index = Columns 1 through 13 0 0 0 0 0 0 0 0 0 0 0 0 0 Columns 14 through 26 0 0 0 0 0 0 0 0 0 0 0 0 0 … Columns 976 through 988 0 0 0 0 0 0 0 0 0 0 0 0 0 Columns 989 through 1.001 0 0 0 0 0 0 0 0 0 0 0 0 0 quantz = Columns 1 through 8 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 Columns 9 through 16 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 … Columns 993 through 1,000 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 -1.8750 Column 1.001 -1.8750 distort = 11.5386	PCM quantized signal

`>> qerr = pcm - quantz`	qerr = Columns 1 through 8 5.8750 5.8671 5.8435 5.8041 5.7493 5.6792 5.5941 5.4943 Columns 9 through 16 5.3802 5.2523 5.1111 4.9571 4.7909 4.6132 4.4247 4.2261 … Columns 993 through 1,000 5.3802 5.4943 5.5941 5.6792 5.7493 5.8041 5.8435 5.8671 Column 1.001 5.8750	The quantization error

`>> subplot( 2, 1, 1 )`
`>> stem( t, pcm )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( ' Home by PCM signal ' );`
`>> hold on ?`
`>> stem( t, quantz , 'r' )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 'The PCM quantized signal' );`
`>> legend( 'Original signal' , 'Quantized signal' );`
`>> hold off ?`
`>> subplot( 2, 1, 2 )`
`>> plot( t, qerr )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( ' Error quantization ' );`
	Figure 11.1	The plotting of the original signal with the quantized in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis .
		The signals are represented in the same shape and with different colors indicated in a new label.
		
		The plotting of the quantization error in the time domain defined. Add title and labels to the horizontal x- axis and vertical y- axis .

Graphic representations

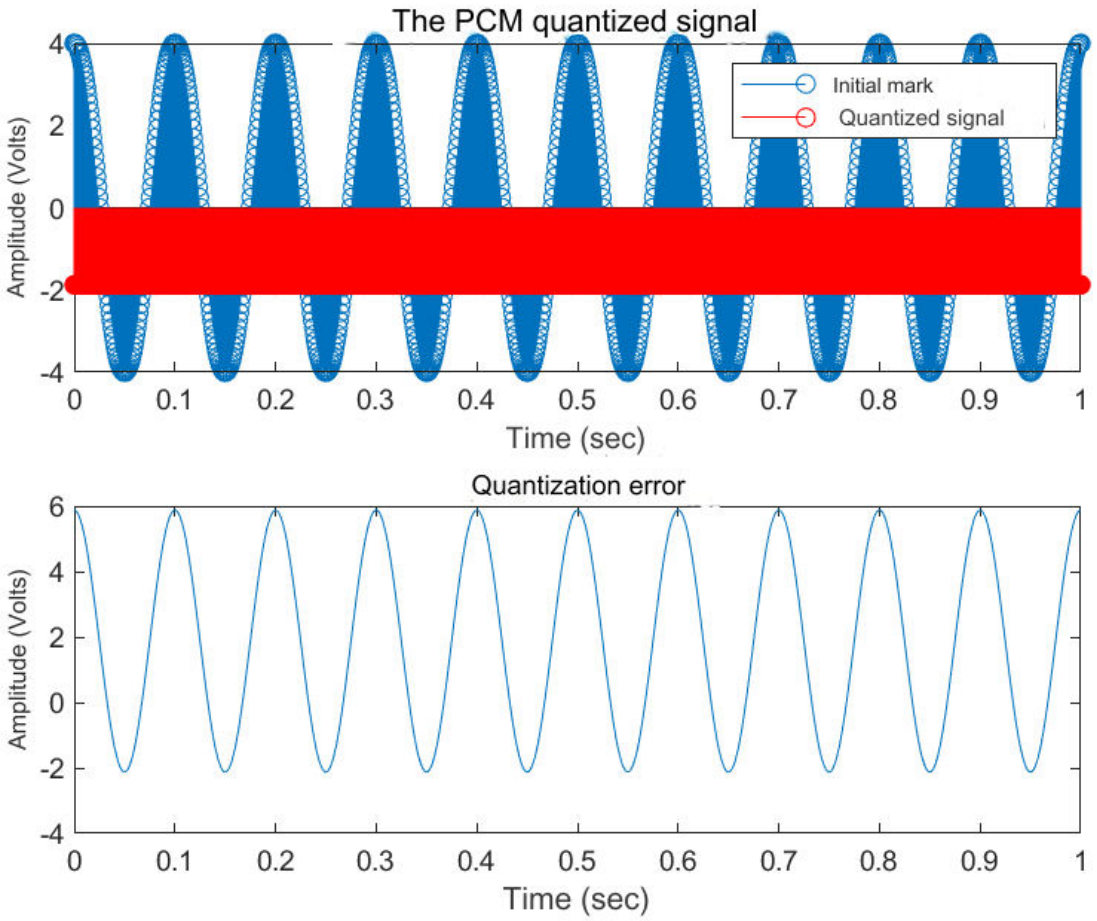

Figure 11.1. The PCM signal, quantized and quantized error

EXERCISE 12

AWGN. A white noise signal consists of a set of independent and identically distributed (iid) random variables. In the discrete case the white noise signal consists of a sequence of samples that are independent and generated from the same probability distribution (eg uniform or Gaussian).

White Gaussian noise can be generated in Matlab with the randn function. For example, with the following commands White Gaussian noise (length 30 samples) with zero mean value and standard deviation σ=1 is produced.

```
mu=0; sigma=1;
noise= sigma * randn ( 1.30)+mu
```

The Power Spectral Density function (PSD) shows the amount of power contained per unit of spectrum.
The power spectral density of a random process can be calculated from the Fourier transform of its autocorrelation function.
It is assumed that its value is constant over the entire frequency range and equal to the variation of the power of the noise signal.

A simple psd estimator is the periodogram. It consists of taking the DTFT of the signal samples and then squaring the resulting measure. Matlab's periodogram function can be used to calculate and plot the periodogram.

Generate white Gaussian noise signal of length 100000 samples using randn and plot it. Assume that the Gaussian pdf has mean 0 and standard deviation σ= 2 (variance σ^2=4)

In a new chart create the histogram of the signal (consider 100 bins) and find the characteristic shape of the Gaussian pdf.

Generate second white Gaussian noise signal of length 100000 samples using randn and plot it. Assume that the Gaussian pdf has mean 2 and standard deviation σ= 1 (variance σ^2=1)

Create the histogram of the signal and plot it on the same chart as the previous histogram.

Code in Matlab

```
mu 1 = 0
sigma 1 = 2
noise1 = sigma1* randn ( 1 , 1e5 )+mu1
mu2 = 2
sigma2 = 1
noise2 = sigma2* randn ( 1 , 1e5 )+mu2

figure
subplot( 2, 1, 1 )
plot(noise1)
xlabel ( ' Field Frequencies (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'White noise with deviation σ=2 and mean μ=0' );
subplot ( 2, 1, 2 )
plot ( noise 2)
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'White noise with deviation σ=1 and mean μ=2' );

figure
periodogram(noise1)
xlabel ( ' Field Frequencies (Hz)' );
ylabel ( ' Amplitude (Volt)' );
hold on
periodogram( noise2, 'r' )
xlabel ( ' Field Frequencies (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'White noise histograms' );
legend( 'μ=0, σ=2' , 'μ=2, σ=1' )
hold off
```

Explanation of the commands used

Commands	Results	Comments

`>> mu1 = 0`	mu1 = 0	The mean value of the 1st white Gaussian noise signal
`>> sigma1 = 2`	sigma1 = 2	The standard deviation of the 1st white Gaussian noise signal
`>> noise1 = sigma1* randn ( 1 , 1e5 )+mu1`	noise1 = Columns 1 through 8 1.3655 1.7103 2.1762 0.9338 -0.5144 0.2377 1.4888 -1.7957 Columns 9 through 16 4.2076 -2.8829 0.5616 -0.6782 -1.3151 1.5597 -1.0030 0.1553 … Columns 99.985 through 99.992 1.6403 -2.0347 0.0485 4.9516 1.0348 -2.6646 -0.7485 1.6910 Columns 99,993 through 100,000 -2.6861 0.0780 0.2311 -1.9004 2.2033 0.7322 4.4186 1.5603	White Gaussian noise with length 100000 samples, mean 0 and standard deviation σ=1, by calling function randn
`>> mu2 = 2`	mu2 = 2	The mean value of the 2nd white Gaussian noise signal
`>> sigma2 = 1`	sigma2 = 1	The standard deviation of the 2nd white Gaussian noise signal
`>> noise2 = sigma2* randn ( 1 , 1e5 )+mu2`	noise2 = Columns 1 through 8 1.4259 2.3675 2.0810 2.6742 2.8459 2.5834 1.7723 2.1029 Columns 9 through 16 2.1361 3.4816 1.4447 1.2012 1.3418 4.0322 -0.1374 2.6815 ... Columns 99.985 through 99.992 2.3759 2.0879 3.3325 4.5223 2.1067 1.4199 0.2975 0.7912 Columns 99,993 through 100,000 1.6721 0.9367 3.8008 1.5593 1.6946 1.5766 1.0450 1.5674	White Gaussian noise with length 100000 samples, mean 0 and standard deviation σ=1, by calling function randn

>> figure >> subplot(2, 1, 1) >> plot(noise1) >> xlabel (' Field Frequencies (Hz)'); >> ylabel ('Amplitude (Volt)'); >> title('White noise with deviation σ=2 and mean μ=0'); >> subplot (2, 1, 2) >> plot (noise 2) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('White noise with deviation σ=1 and mean μ=2');	Figure 12.1	The plotting of white Gaussian noise signals. Add title and labels to the horizontal x- axis and vertical y- axis .
>> figure >> periodogram(noise1) >> xlabel (' Field Frequencies (Hz)'); >> ylabel (' Amplitude (Volt)'); >> hold on ? >> periodogram(noise2, 'r') >> xlabel (' Field Frequencies (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('White noise histograms'); >> legend('μ=0, σ=2' , 'μ=2, σ=1') >> hold off ?	Figure 12.2	Plotting histograms of white Gaussian noise signals. Add title and labels to the horizontal x- axis and vertical y- axis . The representation is done in the same graph with different colors, they are detailed in a new label

Graphic representations

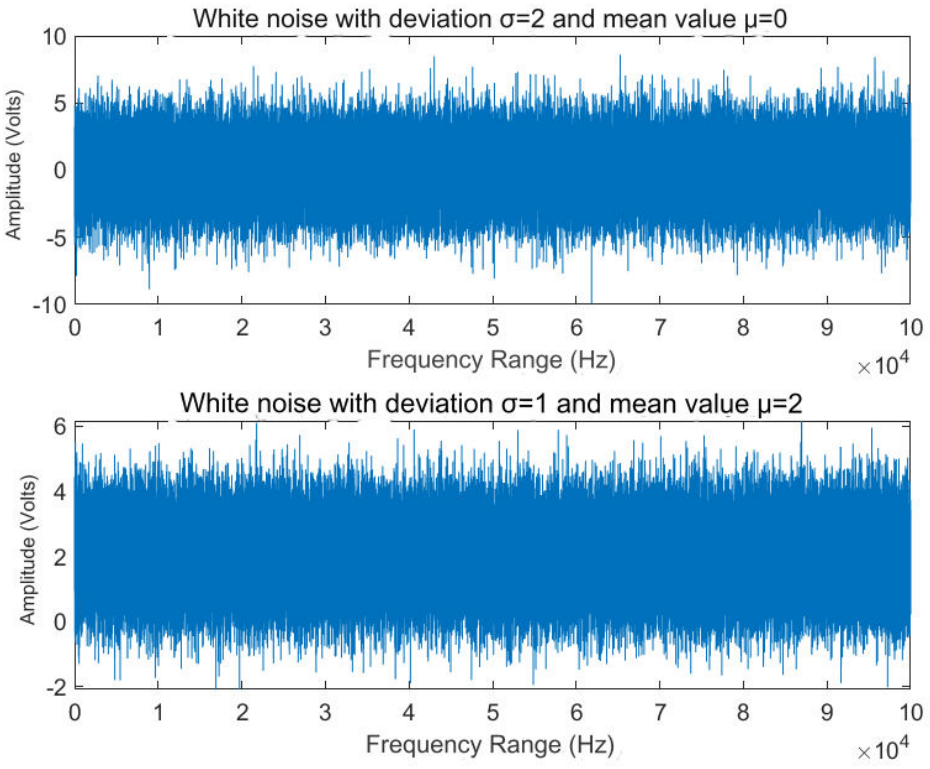

Figure 12.1. The graphs of white Gaussian noises

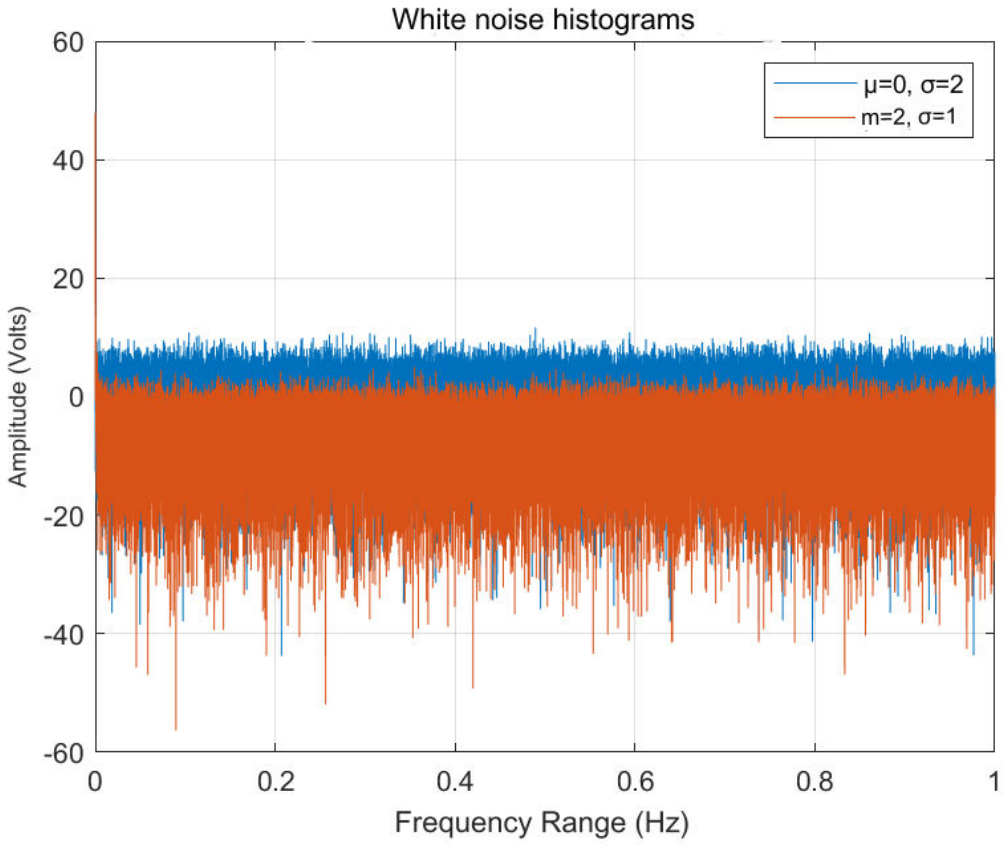

Figure 12.2. The histograms of white Gaussian noises

EXERCISE 13

AWGN Power Spectral Density . (psd , power spectral density). Generate a white Gaussian noise random process consisting of 1000 random sequences of 512 samples each following a Gaussian distribution (eg in a matrix: v 1000x512). Each sequence will have zero mean value and standard deviation σ=2. For each sequence use the following code to find it

```
psd V( i ,:) = abs( fft (v( i ,:))).^2/512; % periodogram
```

Find the mean of the psd and graph the result. Confirm that the value of psd is constant and equal to σ^2.

Code in Matlab

```
mu = 0
sigma = 2
v = sigma* randn ( 1000 , 512 )+mu
avgPsd = 0
sumPsd = 0
for i = 1:length (noise)
    V( i ,:) = abs( fft (v( i ,:))).^2/512;
    sumPsd = sumPsd + V( i )
end
avgPsd = sumPsd /1000
avgPsd1 = mean(V)

plot(avgPsd1)
xlabel( 'Frequency Range (Hz)' );
ylabel ( 'Amplitude ( Volt )' );
title( 'Average of psd of 1000 sequences' );
```

Explanation of the commands used

Commands	Results	Comments
`>> mu = 0`	mu = 0	The mean value of the white Gaussian noise signal
`>> sigma = 2`	sigma = 2	The standard deviation of the white Gaussian noise signal
`>> v = sigma* randn (1000, 512)+mu`	v = Columns 1 through 8 3.8249 -1.1246 0.3775 2.5157 2.4725 -3.9642 -1.9633 -1.9399 5.6140 -4.2115 -0.9531 -0.3660 0.7940 -2.6966 4.4315 -0.2953	Gaussian white noise of 1000 random sequences with length 512 samples, mean 0 and standard deviation σ=2, by calling the function randn

	3.1560 -0.9850 -0.7067 1.9983 0.3434 1.9822 -4.0861 -3.3196 … 3.5462 -2.7221 -3.5510 -0.3134 0.3562 -0.6895 0.7042 1.5849 2.7243 -0.4727 -3.3637 -0.3231 -0.9516 1.3280 0.1036 0.0107 -0.0959 -1.9396 1.8280 -0.1028 2.3807 -1.6210 0.6504 -0.1415 -0.4067 3.5955 1.0180 0.4932 2.1076 4.4519 -0.7780 -0.0918 -3.6291 3.5903 3.2589 3.3181 -1.4148 -0.5505 -0.3852 -3.3140	
`>> avgPsd = 0`	avgPsd = 0	Initialize the mean of the psd of the 1 sequence to 0
`>> sumPsd = 0`	sumPsd = 0	Initialize the psd sum of the 1 sequence with 0

`>> for i = 1:length (noise)`
`  V( i ,:) = abs( fft (v( i ,:))).^2/512;`
`  sumPsd = sumPsd + V( i )`
`end`
	sumPsd =
	
	1.4180
	
	sumPsd =
	
	6.0312
	
	…
	
	sumPsd =
	
	4.2808e+03
	
	sumPsd =
	
	4.2808e+03
		Calculation of the psd of each of the 1000 sequences

`>> avgPsd = sumPsd / 1000`	avgPsd = 4.2808	Calculation of the mean of the psd of the 1 sequence
`>> avgPsd1 = mean(V)`	avgPsd1 = Columns 1 through 8 4.2808 4.0702 3.9227 4.0876 4.0597 3.9584 3.7613 3.8519 Columns 9 through 16 3.9508 4.0201 3.9204 4.2222 4.1745 3.7997 4.1273 3.7192	Calculation of the means of the psd of the 1000 sequences by calling the mean function

`>> plot ( avgPsd 1)`
`>> xlabel( 'Frequency Range (Hz)' );`
`>> ylabel( 'Amplitude (Volt)' );`
`>> title( 'Average of psd of 1000 sequences' );`
	Figure 13.1	Plotting the graph of the means in the frequency domain defined.
		Add title and labels to the horizontal x- axis and vertical y- axis .

Answers to the sub-questions

Observing the "Results" column of the line "`>> avgPsd 1 = mean ( V )`" , the value of psd is constant and equal to $\sigma^2 = 2^2 = 4$.

Graphic representations

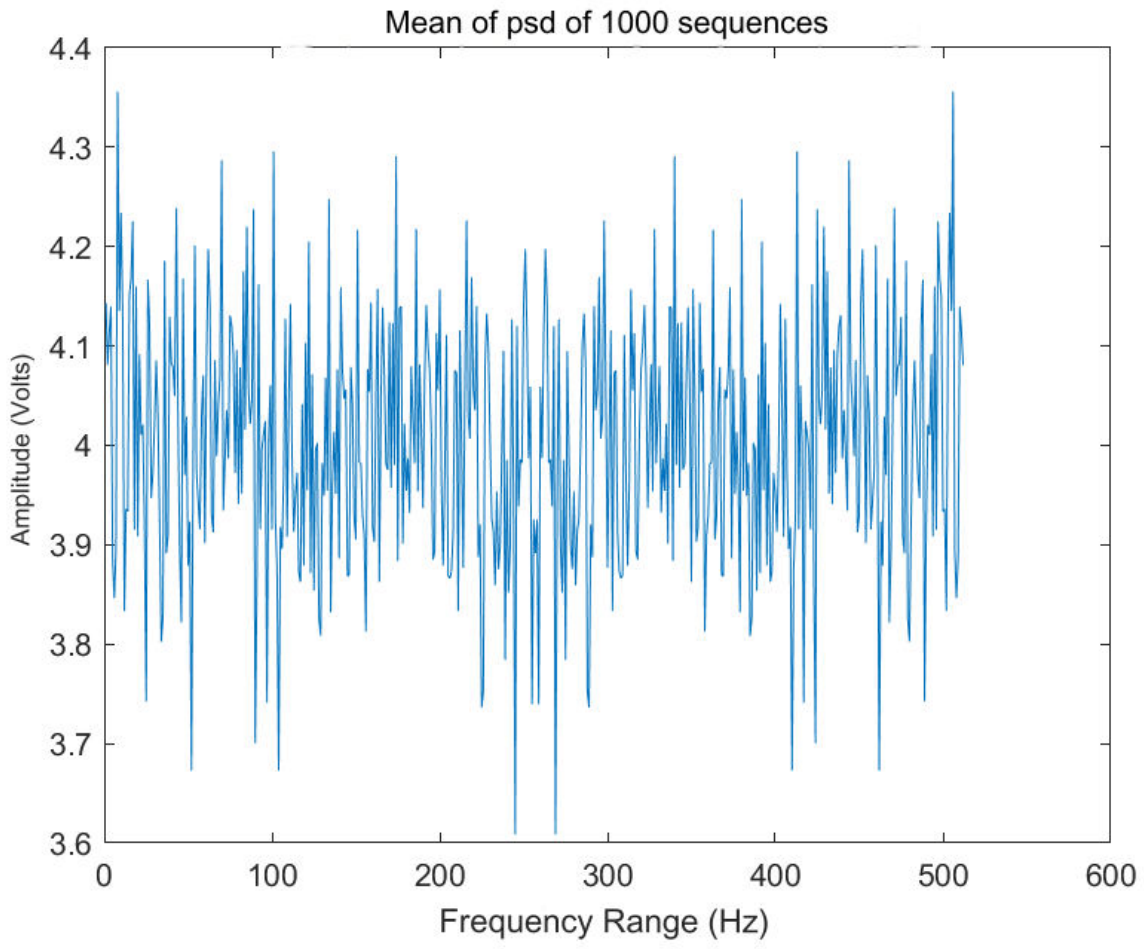

Figure 13.1. The mean psd plot of 1000 sequences

EXERCISE 14

sinc function To generate the signals given by the following relation

$$m1(t)= sinc\ (2tf1)$$
$$m2(t)= sinc\ (4tf1)$$

Where f 1=100 Hz .
Make the graphic representation of the signals from -0.05 to 0.05 sec with a sampling step dt=0.0001.
Represent the x-axis on the two graphs. At which points do the zeros of the signals occur and why?
Display the amplitude spectrum of the signals, but find the number of fft (and f-axis) points from the command:

```
Nf=2^ceil(log2(N));
```

Where N is the number of points on the t axis.
Why is this command used?

What is the theoretical spectrum of the signals m1(t) and m2(t)? Compare the range of the two spectra and explain.

Code in Matlab

```
f 1 = 100
dt = 0.0001
t1 = -0. 05:dt :0.005
m1 = sinc (2*t1*f1)
m2 = sinc (4*t1*f1)
N = length(t1)
Nf = 2^ceil(log2(N))
f = linspace ( -f1, f1, Nf )
ph_m1 = fft ( m1, Nf )
ph_m1 = fftshift (ph_m1)/N
ph_m2 = fft ( m2, Nf )
ph_m2 = fftshift (ph_m2)/N

figure
subplot( 2, 1, 1 )
plot( t1 , m1 )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'm1(t) = sinc (2*t*100)' );
subplot( 2, 1, 2 )
plot( t1 , m2 )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'm2(t) = sinc (4*t*100)' );

figure
subplot( 2 , 1 , 1 );
plot( f, abs(ph_m1))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum m1( t ) = sinc(2*t*100)' );
subplot( 2 , 1 , 2 );
```

```
plot( f, abs(ph_m2))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum m2( t ) = sinc(4*t*100)' );
```

Explanation of the commands used

Commands	Results	Comments
`>> f1 = 100`	f1 = 100	The frequency of signals
`>> dt = 0.0001`	dt = 1.0000e-04	The signal sampling step
`>> t1 = -0. 05:dt :0.005`	t1 = Columns 1 through 8 -0.0500 -0.0499 -0.0498 -0.0497 -0.0496 -0.0495 -0.0494 -0.0493 Columns 9 through 16 -0.0492 -0.0491 -0.0490 -0.0489 -0.0488 -0.0487 -0.0486 -0.0485 … Columns 537 through 544 0.0036 0.0037 0.0038 0.0039 0.0040 0.0041 0.0042 0.0043 Columns 545 through 551 0.0044 0.0045 0.0046 0.0047 0.0048 0.0049 0.0050	The field of times in which the signals are defined
`>> m1 = sinc(2*t1*f1)`	m1 = Columns 1 through 8 -0.0000 -0.0020 -0.0040 -0.0060 -0.0080 -0.0099 -0.0119 -0.0137 Columns 9 through 16 -0.0156 -0.0174 -0.0191 -0.0207 -0.0223 -0.0238 -0.0252 -0.0265 … Columns 537 through 544	The signal m 1 by calling the function sinc

	0.3406 0.3136 0.2867 0.2601 0.2339 0.2080 0.1826 0.1576 Columns 545 through 551 0.1332 0.1093 0.0860 0.0635 0.0416 0.0204 0.0000	
`>> m2 = sinc(4*t1*f1)`	m2 = Columns 1 through 8 -0.0000 -0.0020 -0.0040 -0.0059 -0.0077 -0.0094 -0.0110 -0.0124 Columns 9 through 16 -0.0137 -0.0147 -0.0154 -0.0160 -0.0163 -0.0163 -0.0161 -0.0156 … Columns 537 through 544 -0.2171 -0.2146 -0.2090 -0.2004 -0.1892 -0.1756 -0.1600 -0.1426 Columns 545 through 551 -0.1238 -0.1039 -0.0833 -0.0623 -0.0412 -0.0204 -0.0000	The signal m 2 by calling the function sinc
`>> N = length(t1)`	N = 551	The number of points on the axis of times
`>> Nf = 2^ceil(log2(N))`	Nf = 1024	The number of points of the fft fast Fourier transform by calling the ceil function
`>> f = linspace ( -f1, f1, Nf )`	f = Columns 1 through 8 -100. 0000 - 99.8045 -99.6090 -99.4135 -99.2180 -99.0225 -98.8270 -98.6315 Columns 9 through 16 -98.4360 -98.2405 -98.0450 -97.8495 -97.6540 -97.4585 -97.2630 -97.0674 … Columns 1.009 through 1.016 97.0674 97.2630 97.4585 97.6540 97.8495 98.0450 98.2405 98.4360 Columns 1.017 through 1.024	The range of frequencies defined by the amplitude spectrum of signals

	98.6315 98.8270 99.0225 99.2180 99.4135 99.6090 99.8045 100.0000	
`>> ph_m1 = fft( m1, Nf )`	ph_m1 = Columns 1 through 4 53.9674 + 0.0000i -54.7114 - 2.4628i 52.8596 + 4.9278i -52.5721 - 7.4527i Columns 5 through 8 49.5004 + 9.9850i -48.2526 -12.6731 i 43.6977 +15.3818i -41.5475 - 18.5011i … Columns 1.017 through 1.020 34.5518 -21.7163i -41.5475 +18.5011 i 43.6977 -15.3818i -48.2526 +12.6731i Columns 1.021 through 1.024 49.5004 - 9.9850i -52.5721 + 7.4527i 52.8596 - 4.9278i -54.7114 + 2.4628i	The amplitude spectrum of the signal m 1 by calling the fft function of the fast Fourier transform
`>> ph_m1 = fftshift (ph_m1)/N`	ph_m1 = Columns 1 through 4 -0.0000 + 0.0000i 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i Columns 5 through 8 -0.0000 + 0.0000i 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i … Columns 1.017 through 1.020 0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i Columns 1.021 through 1.024 -0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i	The amplitude spectrum of the signal m 1 by calling the function fft of the fast Fourier transform , symmetric about the vertical axis
`>> ph_m2 = fft( m2, Nf )`	ph_m2 = Columns 1 through 4 23.6605 + 0.0000i -23.8801 - 2.1357i 23.5240 + 4.2465i -23.6082 - 6.3113i	The amplitude spectrum of the m 2 signal by calling the fft function of the fast Fourier transform

	Columns 5 through 8 23.0998 + 8.3031i -23.0365 -10.2066 i 22.3460 +11.9941i -22.1122 - 13.6579i … Columns 1.017 through 1.020 21.1980 -15.1699i -22.1122 +13.6579i 22.3460 -11.9941i - 23.0365 +10.2066i Columns 1.021 through 1.024 23.0998 - 8.3031i -23.6082 + 6.3113i 23.5240 - 4.2465i -23.8801 + 2.1357i	
`>> ph_m2 = fftshift (ph_m2)/N`	ph_m2 = Columns 1 through 4 0.0000 + 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i -0.0000 + 0.0000i Columns 5 through 8 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 + 0.0000i … Columns 1.017 through 1.020 -0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i Columns 1.021 through 1.024 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i -0.0000 - 0.0000i	The amplitude spectrum of the signal m 2 by calling the function fft of the fast Fourier transform , symmetric about the vertical axis

`>> figure`
`>> subplot( 2, 1, 1 )`
`>> plot( t1, m1 )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 'm1(t) = sinc (2*t*100)' );`
`>> subplot( 2, 1, 2 )`
`>> plot( t1, m2 )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 'm2(t) = sinc (4*t*100)' );`
	Figure 14.1	The drawing of the graphical representations of the signals m 1, m 2 in the time domain defined. Add title and labels to the x and y axes.

>> figure >> subplot(2, 1, 1); >> plot(f, abs(ph_m1)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum m1(t) = sinc(2*t*100)'); >> subplot(2, 1, 2); >> plot(f, abs(ph_m2)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum m2(t) = sinc(4*t*100)');	Figure 14.2	The drawing of the graphical representations of the amplitude spectra of the signals m 1, m 2 in the field of the defined frequencies. Add title and labels to the x and y axes .

Answers to the sub-questions

The function sinc (x) implements the sine operation sin (x)/ x , where x = n π (n integer). Zeros of the signals occur when x tends to π or 2π, where sin (π) = 0 = sin (2π). Therefore, the signals are zeroed for times t = n π / 2 f 1 for signal m 1 and t = n π / 4 f 1 for signal m 2.

The command " `Nf = 2^ceil(log2(N))` " is used because the ceil function rounds the log 2(N) result to the nearest integer. For example if log 2(N) = 3.6, then ceil (log 2(N)) = 4. Obviously, the number of points at which the amplitude spectrum of a signal is calculated with the fast Fourier transform (fft) cannot be decimal .

The theoretical amplitude spectrum of the two signals differs in bandwidth because they have different parameters. The m 2 signal has twice the input of the m 2 signal resulting in a larger bandwidth and a wider spectrum.

Graphic representations

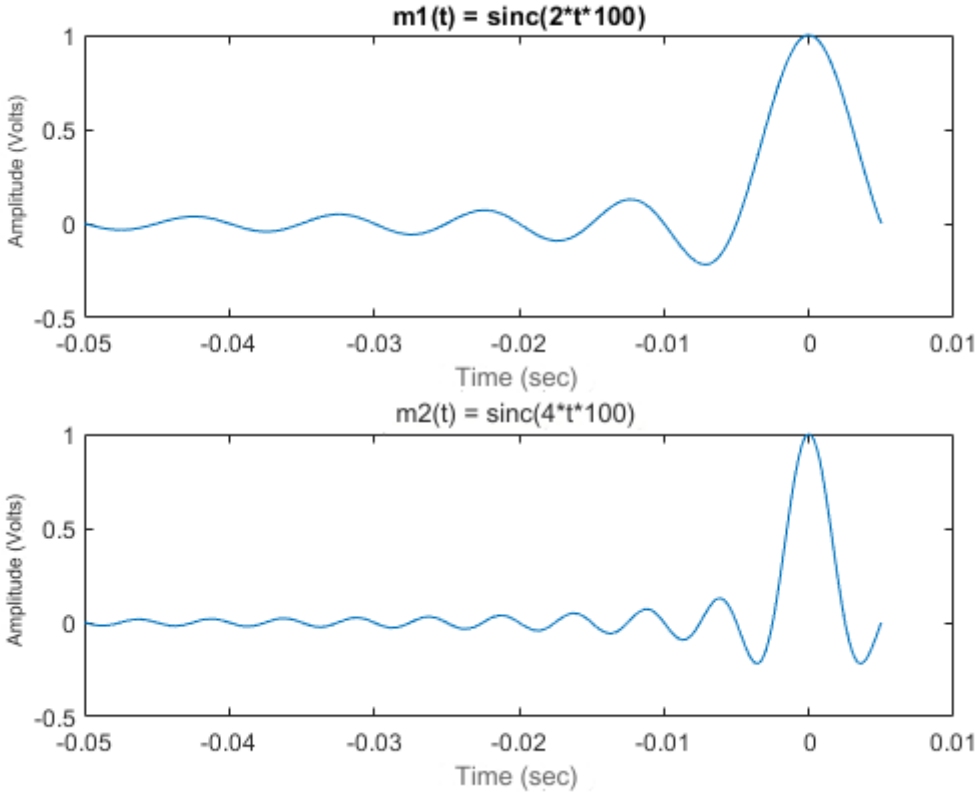

Figure 14.1. The graphs of the signals m 1 and m 2

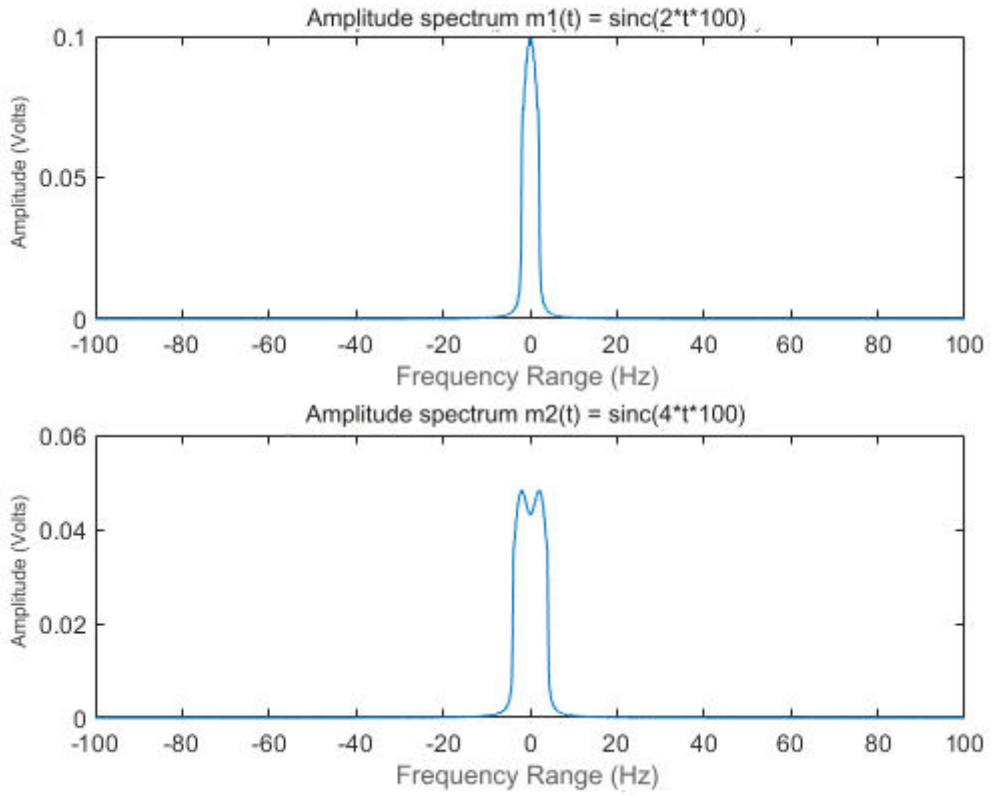

Figure 14.2. The plots of the amplitude spectra of the m 1 and m 2 signals

EXERCISE 15

DSB configuration. Spectrum. To generate an information signal given by the following relation

$$m(t)= \text{sinc}\ (2tf1)$$

Where f1=100Hz.
Display the spectrum of the signal, but find the number of fft (and f-axis) points from the command:

```
Nf=2^ceil(log2(N));
```

Where N is the number of points on the t axis.

To modulate the signal by DSB with a frequency carrier of 600Hz.
Make a graphical display of the modulated signal and its spectrum.

Code in Matlab

```
f 1 = 100
dt = 0.0001
t = -0. 05:dt :0.005
m = sinc (2*t*f1)
N = length(t)
Nf = 2^ceil(log2(N))
f = linspace ( -f1, f1, Nf )
ph_m = fft ( m, Nf )
ph_m = fftshift ( ph_m )/N

fc = 600
DSB = m.*cos(2*pi*fc*t)
ph_dsb = fft ( DSB, Nf )
ph_dsb = fftshift ( ph_dsb )/N

figure
subplot( 2, 1, 1 )
plot( t , m )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'm(t) = sinc (2*t*100)' );
subplot( 2, 1, 2 )
plot( f, abs( ph_m ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum m(t) = sinc(2*t*100)' );

figure
subplot( 2 , 1 , 1 );
plot( t , DSB )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB signal m(t).*cos(2*pi*600*t) = sinc (2*t*100).*cos(2*pi*600*t)' );
subplot( 2 , 1 , 2 );
```

```
plot( f, abs( ph_dsb ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum m(t).*cos(2*pi*600*t) = sinc(2*t*100).*cos(2*pi*600*t)' );
```

Explanation of the commands used

Commands	Results	Comments
>> f1 = 100	f1 = 100	The frequency of the information signal
>> dt = 0.0001	dt = 1.0000e-04	The sampling step of the signal
>> t = -0. 05:dt :0.005	t = Columns 1 through 8 -0.0500 -0.0499 -0.0498 -0.0497 -0.0496 -0.0495 -0.0494 -0.0493 Columns 9 through 16 -0.0492 -0.0491 -0.0490 -0.0489 -0.0488 -0.0487 -0.0486 -0.0485 … Columns 537 through 544 0.0036 0.0037 0.0038 0.0039 0.0040 0.0041 0.0042 0.0043 Columns 545 through 551 0.0044 0.0045 0.0046 0.0047 0.0048 0.0049 0.0050	The field of times defined by the information signal
>> m = sinc(2*t*f1)	m = Columns 1 through 8 -0.0000 -0.0020 -0.0040 -0.0060 -0.0080 -0.0099 -0.0119 -0.0137 Columns 9 through 16 -0.0156 -0.0174 -0.0191 -0.0207 -0.0223 -0.0238 -0.0252 -0.0265 … Columns 537 through 544 0.3406 0.3136 0.2867 0.2601 0.2339 0.2080 0.1826 0.1576	The information signal by calling the function sinc

	Columns 545 through 551 0.1332 0.1093 0.0860 0.0635 0.0416 0.0204 0.0000	
>> N = length(t)	N = 551	The number of points on the axis of times
>> Nf = 2^ceil(log2(N))	Nf = 1024	The number of points of the fft fast Fourier transform by calling the ceil function
>> f = linspace (-f1, f1, Nf)	f = Columns 1 through 8 -100. 0000 - 99.8045 -99.6090 -99.4135 -99.2180 -99.0225 - 98.8270 -98.6315 Columns 9 through 16 -98.4360 -98.2405 -98.0450 - 97.8495 -97.6540 -97.4585 - 97.2630 -97.0674 … Columns 1.009 through 1.016 97.0674 97.2630 97.4585 97.6540 97.8495 98.0450 98.2405 98.4360 Columns 1.017 through 1.024 98.6315 98.8270 99.0225 99.2180 99.4135 99.6090 99.8045 100.0000	The frequency domain defined by the width spectrum of the information signal and in series the DSB modulated signal
>> ph_m = fft(m, Nf)	ph_m = Columns 1 through 4 53.9674 + 0.0000i -54.7114 - 2.4628i 52.8596 + 4.9278i - 52.5721 - 7.4527i Columns 5 through 8 49.5004 + 9.9850i -48.2526 - 12.6731 i 43.6977 +15.3818i - 41.5475 -18.5011i … Columns 1.017 through 1.020 34.5518 -21.7163i -41.5475 +18.5011 i 43.6977 -15.3818i - 48.2526 +12.6731i Columns 1.021 through 1.024	The amplitude spectrum of the information signal by calling the fft function of the fast Fourier transform

	49.5004 - 9.9850i -52.5721 + 7.4527i 52.8596 - 4.9278i - 54.7114 + 2.4628i	
>> ph_m = fftshift (ph_m)/N	ph_m = Columns 1 through 4 -0.0000 + 0.0000i 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i Columns 5 through 8 -0.0000 + 0.0000i 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i … Columns 1.017 through 1.020 0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i Columns 1.021 through 1.024 -0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i	The amplitude spectrum of the information signal by calling the function fft of the fast Fourier transform , symmetric about the vertical axis
>> fc = 600	fc = 600	The carrier frequency
>> DSB = m.*cos(2*pi*fc*t)	DSB = Columns 1 through 8 -0.0000 -0.0019 -0.0029 - 0.0026 -0.0005 0.0031 0.0076 0.0120 Columns 9 through 16 0.0155 0.0168 0.0154 0.0111 0.0042 -0.0045 -0.0135 -0.0215 … Columns 537 through 544 0.1825 0.0588 -0.0537 -0.1394 -0.1892 -0.2015 -0.1811 - 0.1381 Columns 545 through 551 -0.0849 -0.0338 0.0054 0.0270 0.0303 0.0190 0.0000	DSB modulated signal
>> ph_dsb = fft (DSB, Nf)	ph_dsb = Columns 1 through 4	DSB- modulated signal by calling the fft function of the fast Fourier transform

	-0.1293 + 0.0000i 0.1547 - 0.0340i - 0.1136 + 0.0663i 0.1242 - 0.0951i Columns 5 through 8 -0.0696 + 0.1190i 0.0693 - 0.1365i - 0.0056 + 0.1469i 0.0005 - 0.1491i … Columns 1.017 through 1.020 0.0656 - 0.1433i 0.0005 + 0.1491i - 0.0056 - 0.1469i 0.0693 + 0.1365i Columns 1.021 through 1.024 -0.0696 - 0.1190i 0.1242 + 0.0951i - 0.1136 - 0.0663i 0.1547 + 0.0340i	
>> ph_dsb = fftshift (ph_dsb)/N	ph_dsb = Columns 1 through 4 -0.0000 + 0.0000i 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i Columns 5 through 8 -0.0000 + 0.0000i 0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i … Columns 1.017 through 1.020 0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i Columns 1.021 through 1.024 -0.0000 - 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 + 0.0000i	DSB- modulated signal by calling the fft function of the fast Fourier transform , symmetric about the vertical axis
>> figure >> subplot(2, 1, 1) >> plot(t, m) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('m(t) = sinc (2*t*100)'); >> subplot(2, 1, 2) >> plot(f, abs(ph_m))	Figure 15.1	The drawing of the graphical representations of the information signal in the defined time domain and of the amplitude spectrum in the defined frequency domain. Add title and labels to the x and y axes .

>> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum m(t) = sinc(2*t*100)');		
>> figure >> subplot(2, 1, 1); >> plot(t , DSB) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('DSB signal m(t).*cos(2*pi*600*t) = sinc(2*t*100).* cos(2*pi*600*t)'); >> subplot(2, 1, 2); >> plot(f, abs(ph_dsb)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title ('Width spec-trum m (t).* cos (2* pi *600* t) = sinc (2* t *100).* c os (2*pi*600*t)');	Figure 15.2	The plotting of the DSB modulated signal in the defined time domain and the amplitude spectrum in the defined frequency domain. Add title and labels to the x and y axes .

Graphic representations

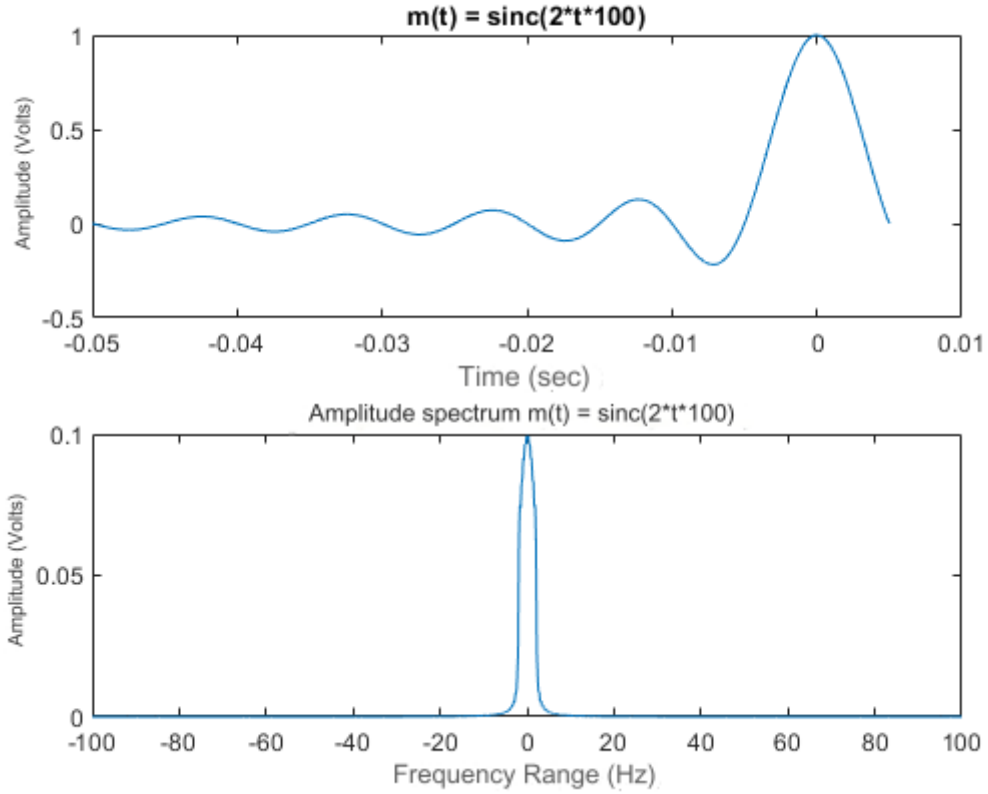

Figure 15.1. The graphs of the information signal and the amplitude spectrum

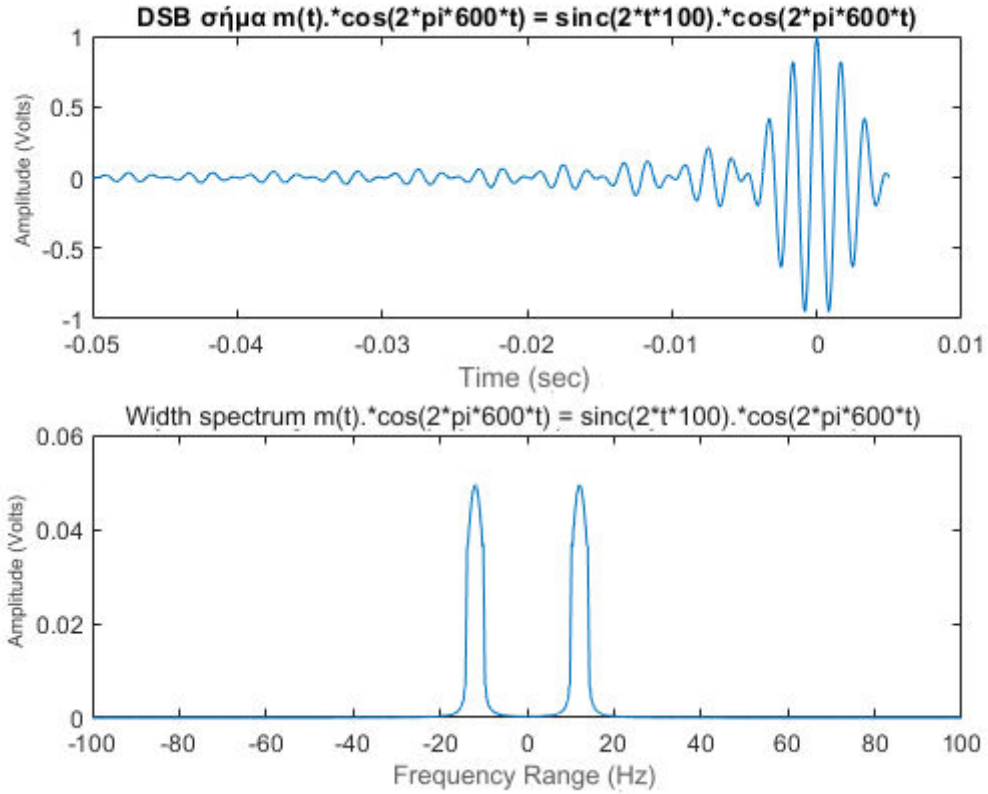

Figure 15.2. The plots of the DSB modulated signal and the amplitude spectrum

EXERCISE 16

DSB demodulation. a) Create a cosine signal of amplitude Am=0.5 Volt, frequency fa=1KHz. The time axis should be set from 0 to 5 periods of the signal. Create a graphics window and split it into 3 subwindows. In the 1st subwindow you plot x(t) against t.

b) To create a cosine signal of amplitude Ac=2.5 Volt, frequency fc ten times the frequency of the information signal The time axis remains the same.

Display the graph of y(t) against t in the 2nd subwindow.

c) To modulate the carrier signal, y(t), from the information signal, x(t), by DSB.

The modulated signal, z(t), be represented in terms of t, in the 3rd subwindow.

d) To perform conformal demodulation of the signal, using a local oscillator, in accordance with the carrier signal of the transmitter and then to pass the signal through a low-pass filter butterworth_filter(in, dt, order, fcut, fcenter), using the corresponding file (select the appropriate parameters).

In a new graphical window to represent simultaneously (by freezing the window) the original information signal and the demodulated one (after the filter). Choose different colors for the two graphs.

Code in Matlab

```
Am = 0.5
fa = 1e3
Ta = 1/fa
dt = Ta/100
Tw = 5*Ta
t = dt:dt :Tw
x = Am*cos(2*pi*fa*t)

Ac = 2.5
fc = 10 * fa
y = Ac*cos(2*pi*fc*t)

z = x.*y

demod = butterworth_filter ( z , dt , 10 , fc , fa )

figure
subplot( 3, 1, 1 )
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'x(t) = 0.5*cos(2*pi*1000*t)' );
subplot( 3, 1, 2 )
plot( t , y )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'y(t) = 2.5*cos(2*pi*10000*t)' );
subplot( 3, 1, 3 )
plot( t , z )
```

```
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB signal x(t).*y(t) = 0.5*cos(2*pi*1000*t) .* 2.5*cos(2*pi*10000*t)' );


figure
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
hold on
plot( t, demod , 'r' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Signal DSB Demodulation ' );
legend( 'x(t) = 0.5*cos(2*pi*1000*t)' , ' Demodulated signal ' )
hold off
```

Explanation of the commands used

Commands	Results	Comments
`>> Am = 0.5`	Am = 0.5000	The width of the information signal
`>> fa = 1e3`	fa = 1000	The frequency of the information signal
`>> Ta = 1/fa`	Ta = 1.0000e-03	The period of the information signal
`>> dt = Ta/100`	dt = 1.0000e-05	The sampling step
`>> Tw = 5*Ta`	Tw = 0.0050	The time duration of the signal
`>> t = dt:dt :Tw`	t = Columns 1 through 8 0.0000 0.0000 0.0000 0.0000 0.0001 0.0001 0.0001 0.0001 Columns 9 through 16 0.0001 0.0001 0.0001 0.0001 0.0001 0.0001 0.0002 0.0002 ... Columns 489 through 496 0.0049 0.0049 0.0049 0.0049 0.0049 0.0049 0.0050 0.0050 Columns 497 through 500	The time axis defined by the information signal

	0.0050 0.0050 0.0050 0.0050	
`>> x = Am*cos(2*pi*fa*t)`	x = Columns 1 through 8 0.4990 0.4961 0.4911 0.4843 0.4755 0.4649 0.4524 0.4382 Columns 9 through 16 0.4222 0.4045 0.3853 0.3645 0.3423 0.3187 0.2939 0.2679 ... Columns 489 through 496 0.3853 0.4045 0.4222 0.4382 0.4524 0.4649 0.4755 0.4843 Columns 497 through 500 0.4911 0.4961 0.4990 0.5000	The information signal
`>> Ac = 2.5`	Ac = 2.5000	The amplitude of the carrier cosine signal
`>> fc = 10 * fa`	fc = 10000	The carrier frequency
`>> y = Ac*cos(2*pi*fc*t)`	y = Columns 1 through 8 2.0225 0.7725 -0.7725 -2.0225 -2.5000 -2.0225 -0.7725 0.7725 Columns 9 through 16 2.0225 2.5000 2.0225 0.7725 -0.7725 -2.0225 -2.5000 -2.0225 ... Columns 489 through 496 2.0225 2.5000 2.0225 0.7725 -0.7725 -2.0225 -2.5000 -2.0225 Columns 497 through 500 -0.7725 0.7725 2.0225 2.5000	The carrier cosine signal
`>> z = x.*y`	z = Columns 1 through 8	The shaped by DSB carrying signal y (t) from information signal x (t)

	1.0093 0.3832 -0.3794 -0.9795 -1.1888 -0.9403 -0.3495 0.3385 Columns 9 through 16 0.8538 1.0113 0.7792 0.2816 -0.2644 -0.6446 -0.7347 -0.5419 ... Columns 489 through 496 0.7792 1.0113 0.8538 0.3385 -0.3495 -0.9403 -1.1888 -0.9795 Columns 497 through 500 -0.3794 0.3832 1.0093 1.2500	
`>> demod = butter-worth_filter ( z, dt, 10 , fc, fa)`	demod = Columns 1 through 4 0.5216 - 0.1419i 0.2226 + 0.1647i -0.1657 + 0.4042i -0.5046 + 0.4840i Columns 5 through 8 -0.6697 + 0.3769i - 0.5983 + 0.1304i -0.3144 - 0.1545i 0.0801 - 0.3658i ... Columns 493 through 496 -0.2814 + 0.3624i - 0.4788 + 0.4567i -0.5275 + 0.3742i -0.3938 + 0.1395i Columns 497 through 500 -0.1091 - 0.1587i 0.2350 - 0.4030i 0.5170 - 0.4946i 0.6294 - 0.3949i	The demodulated signal using a local oscillator from a low-pass filter by calling the function butterworth _ filter

`>> figure`
`>> subplot( 3, 1, 1 )`
`>> plot( t, x )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 'x(t) = 0.5*cos(2*pi*1000*t)' );`
`>> subplot( 3, 1, 2 )`
`>> plot( t, y )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( 'y(t) = 2.5*cos(2*pi*10000*t)' );`
`>> subplot( 3, 1, 3 )`
`>> plot( t, z )`
	Figure 16.1	The plotting of the information signal x (t), the carrier signal y (t) and the modulated signal y (t) from the information signal x (t) against DSB signal in the time domain are defined. Add title and labels to the x and y axes .

>> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('DSB signal x(t).*y(t) = 0.5*cos(2*pi*1000*t) .* 2.5*cos(2*pi*10000*t)');		
>> figure >> plot(t, x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> hold on ? >> plot(t, demod , 'r') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Signal DSB Demodulation '); >> legend('x(t) = 0.5*cos(2*pi*1000*t)' , ' Demodulated signal ') >> hold off ?	Figure 16.2	The plots of the information signal x (t) and the DSB-demodulated signal (after the filter) in the time domain are defined. Add title and labels to the x and y axes . Representation is on the same plot and signals are distinguished by relabelled colors

Graphic representations

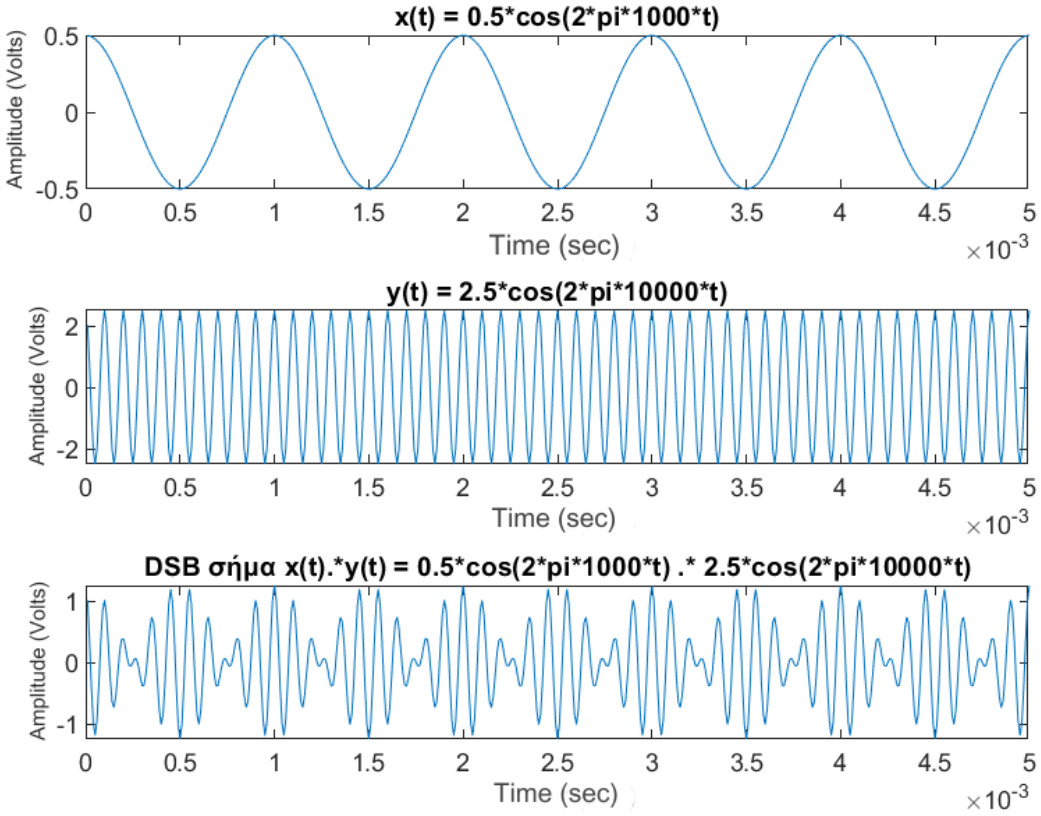

Figure 16.1. The information signal, the carrier signal and the DSB signal

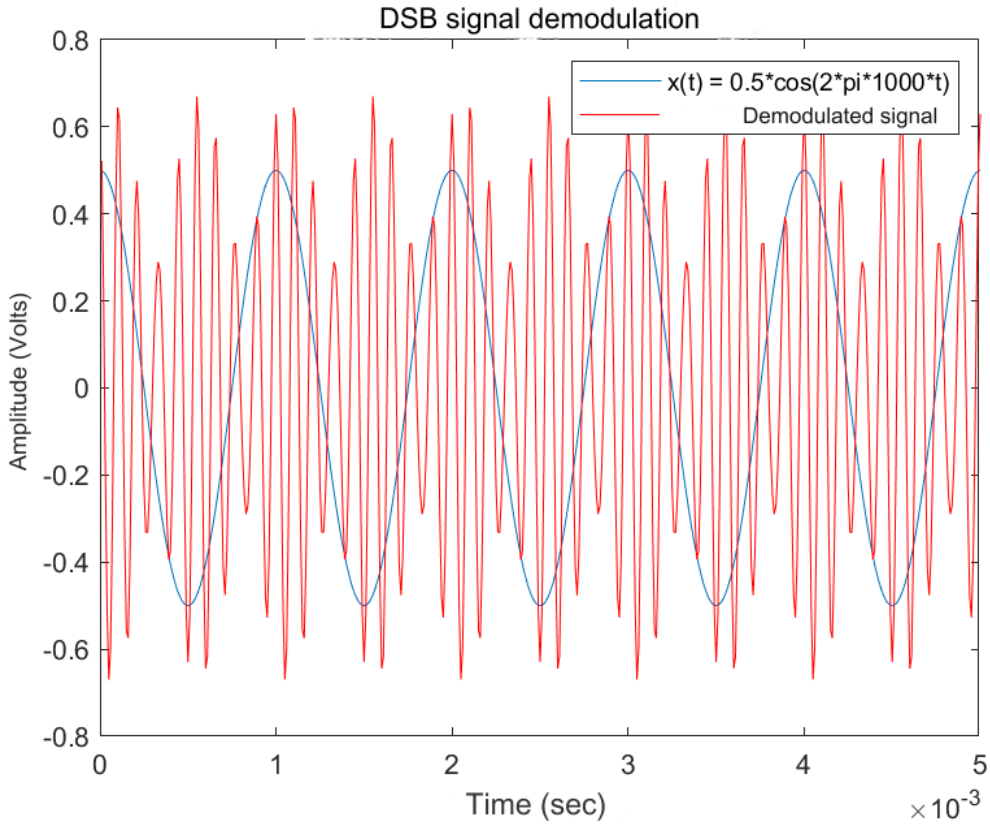

Figure 16.2. The information signal and the demodulated after filter

EXERCISE 17

DSB demodulation with a non-ideal local oscillator. Repeat the procedure of question (d) of exercise 16, for a non-ideal local oscillator with some small phase difference from the carrier signal (eg 0.1π). In a new graphic window, the original information signal and the demodulated one (after the filter) should be represented at the same time.

Repeat the process for a phase difference such that there is a total loss of the demodulated signal. In a new graphic window, the original information signal and the demodulated one (after the filter) should be represented at the same time.

Code at Matlab

```
Am = 0.5
fa = 1e3
Ta = 1/fa
dt = Ta/100
Tw = 5*Ta
ph = 0.1*pi
t = dt:dt :Tw
x = Am*cos(2*pi*fa*t)

Ac = 2.5
fc = 10 * fa

y = Ac*cos(2*pi*fc*t)
z = x.*y
demod = butterworth_filter ( z , dt , 10 , fc , fa )

y_non = Ac*cos(2*pi*fc* t+ph )
z_non = x.* y_non
demod_non = butterworth_filter ( z_non , dt , 10, fc, fa)

ph_loss = pi;
y_loss = Ac*cos(2*pi*fc* t+ph_loss )
z_loss = x.* y_loss
demod_loss = butterworth_filter ( z_loss , dt , 10, fc, fa)

figure
subplot( 3, 1, 1 )
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'x(t) = 0.5*cos(2*pi*1000*t)' );
subplot( 3, 1, 2 )
plot( t , y )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'y(t) = 2.5*cos(2*pi*10000*t)' );
subplot( 3, 1, 3 )
plot( t , z )
xlabel ( ' Time (sec)' );
```

```
ylabel ( ' Amplitude (Volt)' );
title( 'DSB signal x(t).*y(t) = 0.5*cos(2*pi*1000*t) .* 2.5*cos(2*pi*10000*t)' );

figure
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
hold on
plot( t, demod , 'r' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Signal DSB Demodulation ' );
legend( 'x(t) = 0.5*cos(2*pi*1000*t)' , ' Demodulated signal ' )
hold off

figure
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
hold on
plot( t, demod_non , 'r' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB signal demodulation with 0.1π phase difference' );
legend( 'x(t) = 0.5*cos(2*pi*1000*t)' , 'Demodulated signal' )
hold off

figure
plot( t , x )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
hold on
plot( t, demod_loss , 'r' )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'DSB signal demodulation with total energy loss' );
legend( 'x(t) = 0.5*cos(2*pi*1000*t)' , 'Demodulated signal' )
hold off
```

Explanation of the commands used

Commands	Results	Comments
`>> Am = 0.5`	Am = 0.5000	The width of the information signal
`>> fa = 1e3`	fa = 1000	The frequency of the information signal
`>> Ta = 1/fa`	Ta = 1.0000e-03	The period of the information signal
`>> dt = Ta/100`	dt = 1.0000e-05	The sampling step
`>> Tw = 5*Ta`	Tw = 0.0050	The time duration of the signal

`>> ph = 0.1*pi`	ph = 0.3142	The phase difference of the ideal non-local oscillator with the carrier signal of the local one
`>> t = dt:dt :Tw`	t = Columns 1 through 8 0.0000 0.0000 0.0000 0.0000 0.0001 0.0001 0.0001 0.0001 Columns 9 through 16 0.0001 0.0001 0.0001 0.0001 0.0001 0.0001 0.0002 0.0002 ... Columns 489 through 496 0.0049 0.0049 0.0049 0.0049 0.0049 0.0049 0.0050 0.0050 Columns 497 through 500 0.0050 0.0050 0.0050 0.0050	The time axis defined by the information signal
`>> x = Am*cos(2*pi*fa*t)`	x = Columns 1 through 8 0.4990 0.4961 0.4911 0.4843 0.4755 0.4649 0.4524 0.4382 Columns 9 through 16 0.4222 0.4045 0.3853 0.3645 0.3423 0.3187 0.2939 0.2679 ... Columns 489 through 496 0.3853 0.4045 0.4222 0.4382 0.4524 0.4649 0.4755 0.4843 Columns 497 through 500 0.4911 0.4961 0.4990 0.5000	The information signal
`>> Ac = 2.5`	Ac = 2.5000	The amplitude of the carrier cosine signal
`>> fc = 10 * fa`	fc = 10000	The carrier frequency
`>> y = Ac*cos(2*pi*fc*t)`	y = Columns 1 through 8	The carrier cosine signal

	2.0225 0.7725 -0.7725 -2.0225 -2.5000 -2.0225 -0.7725 0.7725 Columns 9 through 16 2.0225 2.5000 2.0225 0.7725 -0.7725 -2.0225 -2.5000 -2.0225 ... Columns 489 through 496 2.0225 2.5000 2.0225 0.7725 -0.7725 -2.0225 -2.5000 -2.0225 Columns 497 through 500 -0.7725 0.7725 2.0225 2.5000	
`>> z = x.*y`	z = Columns 1 through 8 1.0093 0.3832 -0.3794 -0.9795 -1.1888 -0.9403 -0.3495 0.3385 Columns 9 through 16 0.8538 1.0113 0.7792 0.2816 -0.2644 -0.6446 -0.7347 -0.5419 ... Columns 489 through 496 0.7792 1.0113 0.8538 0.3385 -0.3495 -0.9403 -1.1888 -0.9795 Columns 497 through 500 -0.3794 0.3832 1.0093 1.2500	The DSB -modulated carrier signal y (t) from the information signal x (t)
`>> demod = butter-worth_filter ( z, dt, 10 , fc, fa)`	demod = Columns 1 through 4 0.5216 - 0.1419i 0.2226 + 0.1647i -0.1657 + 0.4042i -0.5046 + 0.4840i Columns 5 through 8 -0.6697 + 0.3769i - 0.5983 + 0.1304i -0.3144 - 0.1545i 0.0801 - 0.3658i ... Columns 493 through 496 -0.2814 + 0.3624i - 0.4788 + 0.4567i -0.5275 + 0.3742i -0.3938 + 0.1395i	The demodulated signal using a local oscillator from a low-pass filter by calling the function butterworth _ filter

	Columns 497 through 500 -0.1091 - 0.1587i 0.2350 - 0.4030i 0.5170 - 0.4946i 0.6294 - 0.3949i	
`>> y_non = Ac*cos(2*pi*fc* t+ph )`	y_non = Columns 1 through 8 1.4695 -0.0000 -1.4695 -2.3776 -2.3776 -1.4695 0.0000 1.4695 Columns 9 through 16 2.3776 2.3776 1.4695 0.0000 -1.4695 -2.3776 -2.3776 -1.4695 ... Columns 489 through 496 2.3776 2.3776 1.4695 0.0000 -1.4695 -2.3776 -2.3776 -1.4695 Columns 497 through 500 0.0000 1.4695 2.3776 2.3776	The carrier signal of the local non-ideal oscillator that has a phase difference of 0.1π from the carrier signal of the local
`>> z_non = x.*y_non`	z_non = Columns 1 through 8 0.7333 -0.0000 -0.7217 -1.1515 -1.1306 -0.6831 0.0000 0.6439 Columns 9 through 16 1.0038 0.9618 0.5661 0.0000 -0.5030 -0.7578 -0.6988 -0.3937 ... Columns 489 through 496 0.9160 0.9618 0.6204 0.0000 -0.6648 -1.1053 -1.1306 -0.7116 Columns 497 through 500 0.0000 0.7289 1.1865 1.1888	The DSB modulation of the signal with a non-ideal local oscillator having a phase difference of 0.1π from the carrier signal of the local
`>> demod_non = butter-worth_filter ( z_non , dt , 10, fc, fa)`	demod_non = Columns 1 through 4 0.3801 + 0.0120i 0.0218 + 0.3006i -0.3558 + 0.4701i -0.6159 + 0.4561i Columns 5 through 8	The DSB demodulation of the signal with a non-ideal local oscillator having a phase difference of 0.1π from the carrier signal of the local

	-0.6610 + 0.2685i - 0.4714 - 0.0144i - 0.1137 - 0.2792i 0.2834 - 0.4243i ... Columns 493 through 496 -0.3925 + 0.4237i - 0.5148 + 0.4323i - 0.4682 + 0.2689i -0.2516 - 0.0085i Columns 497 through 500 0.0721 - 0.2927i 0.3934 - 0.4699i 0.5944 - 0.4670i 0.5936 - 0.2825i	
`>> ph_loss = pi;`	ph_loss = 3 . 142	The phase difference of the ideal non-local oscillator with the carrier signal of the local such that there is total loss of the modulated signal
`>> y_loss = Ac*cos(2*pi*fc* t+ph_loss )`	y_loss = Columns 1 through 8 -2.0225 -0.7725 0.7725 2.0225 2.5000 2.0225 0.7725 -0.7725 Columns 9 through 16 -2.0225 -2.5000 -2.0225 -0.7725 0.7725 2.0225 2.5000 2.0225 … Columns 489 through 496 -2.0225 -2.5000 -2.0225 -0.7725 0.7725 2.0225 2.5000 2.0225 Columns 497 through 500 0.7725 -0.7725 -2.0225 -2.5000	The carrier signal of the non-ideal local oscillator that has a phase difference π from the carrier signal of the local oscillator such that there is total loss of the modulated signal
`>> z_loss = x.* y_loss`	z_loss = Columns 1 through 8 -1.0093 -0.3832 0.3794 0.9795 1.1888 0.9403 0.3495 -0.3385 Columns 9 through 16 -0.8538 -1.0113 -0.7792 -0.2816 0.2644 0.6446 0.7347 0.5419 … Columns 489 through 496 -0.7792 -1.0113 -0.8538 -0.3385 0.3495 0.9403 1.1888 0.9795	DSB modulation of the signal with a non-ideal local oscillator having a phase difference π from the local oscillator carrier such that there is total loss of the modulated signal

	Columns 497 through 500 0.3794 -0.3832 -1.0093 -1.2500	
>> demod_loss = butter-worth_filter (z_loss , dt , 10, fc, fa)	demod_loss = Columns 1 through 4 -0.5216 + 0.1419i - 0.2226 - 0.1647i 0.1657 - 0.4042i 0.5046 - 0.4840i Columns 5 through 8 0.6697 - 0.3769i 0.5983 - 0.1304i 0.3144 + 0.1545i - 0.0801 + 0.3658i … Columns 493 through 496 0.2814 - 0.3624i 0.4788 - 0.4567i 0.5275 - 0.3742i 0.3938 - 0.1395i Columns 497 through 500 0.1091 + 0.1587i - 0.2350 + 0.4030i - 0.5170 + 0.4946i -0.6294 + 0.3949i	The DSB demodulation of the signal having a phase difference π from the local oscillator carrier such that there is a total loss of the modulated signal
>> figure >> subplot(3, 1, 1) >> plot(t, x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('x(t) = 0.5*cos(2*pi*1000*t)'); >> subplot(3, 1, 2) >> plot(t, y) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('y(t) = 2.5*cos(2*pi*10000*t)'); >> subplot(3, 1, 3) >> plot(t, z) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('DSB signal x(t).*y(t) = 0.5*cos(2*pi*1000*t) .* 2.5*cos(2*pi*10000*t)');	Figure 17.1	The plotting of the information signal x (t), the carrier signal y (t) and the modulated signal y (t) from the information signal x (t) against DSB signal in the time domain are defined. Add title and labels to the x and y axes .
>> figure >> plot(t, x) >> xlabel (' Time (sec)');		The plots of the information signal x (t) and the DSB-demodulated signal (after the filter) in the time domain are

>> ylabel (' Amplitude (Volt)'); >> hold on ? >> plot(t, demod , 'r') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Signal DSB Demodulation '); >> legend('x(t) = 0.5*cos(2*pi*1000*t)' , ' Demodulated signal ') >> hold off ?	Figure 17.2	defined. Add title and labels to the x and y axes . Representation is on the same plot and signals are distinguished by relabelled colors
>> figure >> plot(t, x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> hold on ? >> plot(t, demod_non , 'r') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('DSB signal de-modulation with 0.1π phase difference'); >> legend('x(t) = 0.5*cos(2*pi*1000*t)' , 'Demodulated signal') >> hold off ?	Figure 17.3	The plots of the information signal x (t) and the DSB - demodulated signal using a non-ideal local oscillator in the time domain defined and having a phase difference of 0.1π from the local oscillator carrier (after filtering) . Add title and labels to the x and y axes . Representation is on the same plot and signals are distinguished by relabelled colors
>> figure >> plot(t, x) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> hold on ? >> plot(t, demod_loss , 'r') >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('DSB signal de-modulation with total en-ergy loss'); >> legend('x(t) = 0.5*cos(2*pi*1000*t)' , 'Demodulated signal') >> hold off ?	Figure 17.4	Plotting the information signal x (t) and the DSB - demodulated signal in the time domain defined and phase difference π from the local oscillator carrier (after the filter) such that there is total loss of the modulated signal. Add title and labels to the x and y axes . Representation is on the same plot and signals are distinguished by relabelled colors

Graphic representations

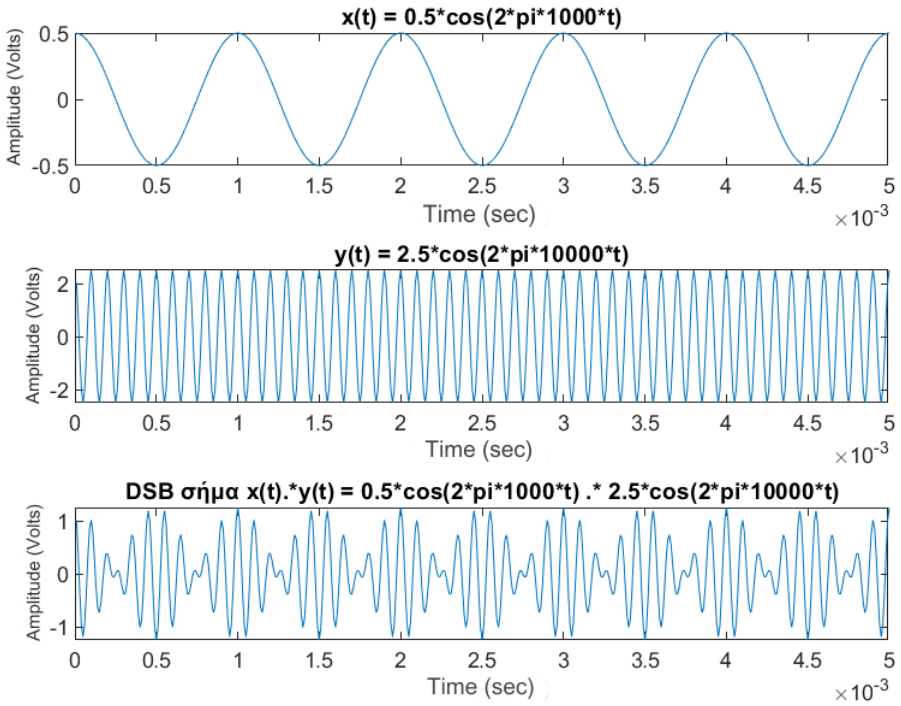

Figure 17.1. The information signal, the carrier signal and the DSB signal

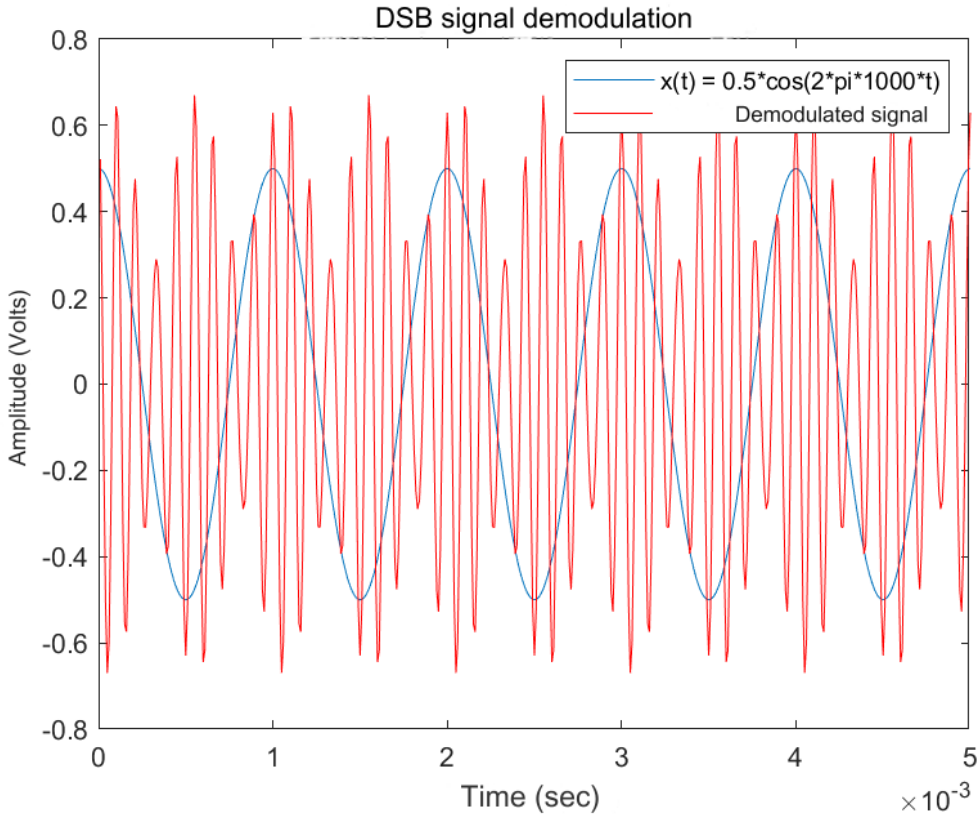

Figure 17.2. The information signal and the demodulated after filter

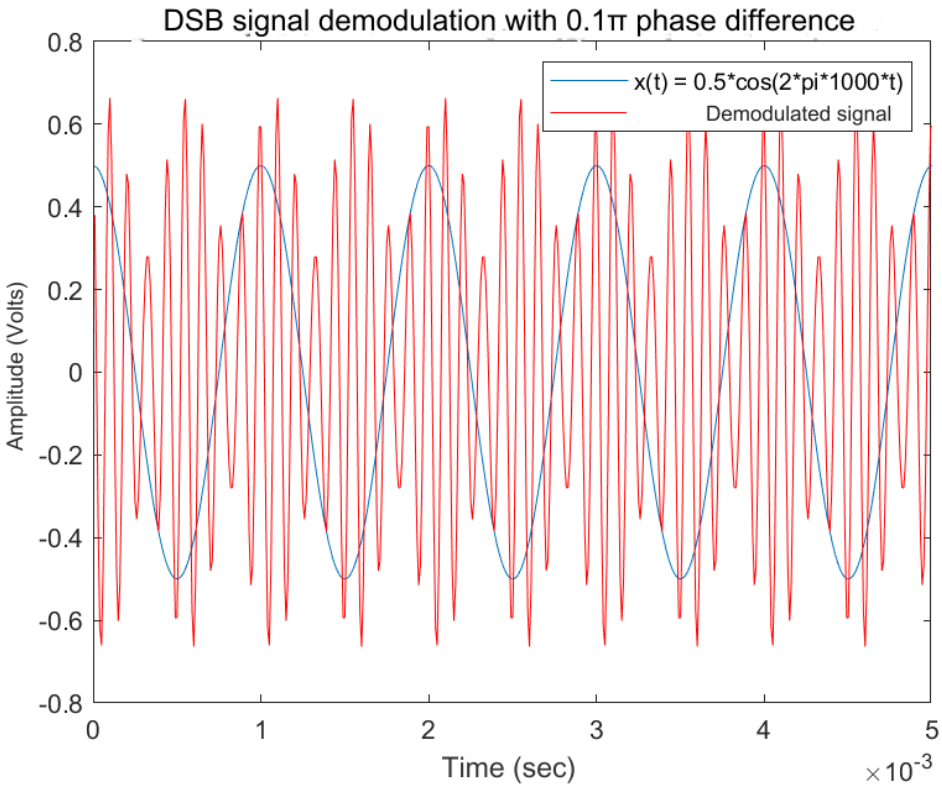

Figure 17.3. The information signal and the demodulated one using a non-ideal local oscillator

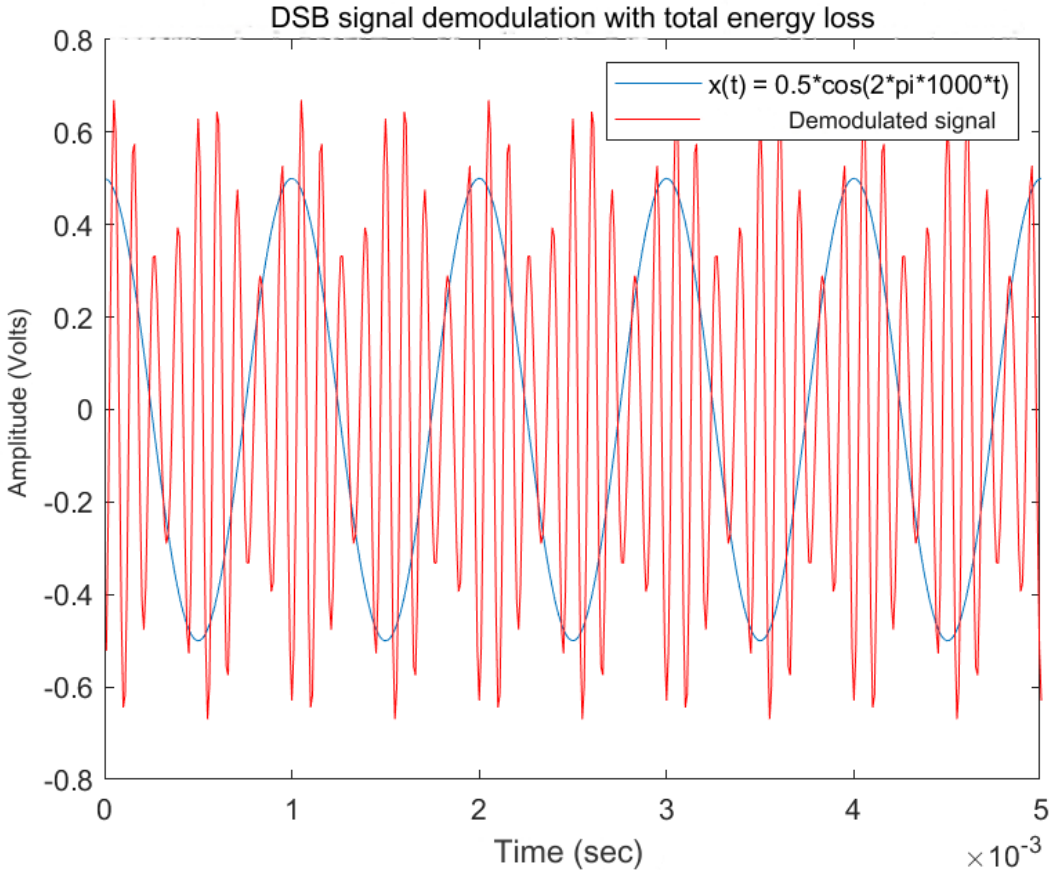

Figure 17.4. The information signal and the demodulated one with phase difference π, after filter

EXERCISE 18

Shannon Hartley theorem. Consider the Shannon Hartley theorem:

$$C = B\ log_2\left(1 + \frac{S}{N}\right)\ bits/sec$$

If instead of the average power N of the noise we consider the power spectral density N0/2 then the above relationship takes the form:

$$C = B\ log_2\left(1 + \frac{S}{N_0}\right)\ bits/sec$$

As the S/N ratio tends to infinity, then the capacity of the C channel also tends to infinity. Correspondingly, when the bandwidth of channel B tends to infinity, the capacity of channel C tends to a specific value:

$$\lim_{B\to\infty} C = 1.44\frac{S}{N_0}$$

Write a program to calculate and represent the capacity of channel C with frequency bandwidth B=3000Hz as a function of the S/N0 ratio (give values to the S/N0 ratio from –20 to 30dB). To collapse values, use the semilogx command instead of plot.

Code at Matlab

```
B = 3e3
S_N0 = -20:1:30
C = B*log2(1 + S_N0/B)

semilogx ( S_N0 , C )
xlabel ( 'S/N0 (dB)' );
ylabel( 'C Channel Capacity (bits/sec)' );
title( 'Shannon Hartley Theorem' );
```

Explanation of the commands used

Commands	Results	Comments
`>> B = 3e3`	B = 3000	The frequency bandwidth
`>> S_N0 = -20:1:30`	S_N0 = Columns 1 through 13 -20 -19 -18 -17 -16 -15 -14 -13 -12 -11 -10 -9 -8	The ratio S / N 0 in order to use the formula with the power spectral density N0/2

	Columns 14 through 26 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 Columns 27 through 39 6 7 8 9 10 11 12 13 14 15 16 17 18 Columns 40 through 51 19 20 21 22 23 24 25 26 27 28 29 30	
`>> C = B*log2(1 + S_N0/B)`	C = Columns 1 through 8 -28. 9505 - 27.4984 -26.0467 - 24.5956 -23.1449 -21.6947 -20.2450 - 18.7958 Columns 9 through 16 -17. 3471 - 15.8988 -14.4510 - 13.0038 -11.5570 -10.1107 -8.6648 - 7.2195 Columns 17 through 24 -5.7746 -4.3303 -2.8864 -1.4429 0 1.4425 2.8844 4.3259 Columns 25 through 32 5.7669 7.2075 8.6475 10.0871 11.5262 12.9648 14.4030 15.8406 Columns 33 through 40 17.2778 18.7145 20.1507 21.5865 23.0218 24.4566 25.8909 27.3248 Columns 41 through 48 28.7581 30.1911 31.6235 33.0554 34.4869 35.9179 37.3485 38.7785 Columns 49 through 51 40.2081 41.6372 43.0659	The capacity of the channel

`>> semilogx ( S _ N 0, C )`
`>> xlabel ( ' S / N 0 ( dB )' );`
`>> ylabel( 'C channel capacity (bits/sec)' );`
	Figure 18.1	The collapse of the values in a graph, in the field of db defined. Add labels to the x and y axes

`>> title( 'Shannon Hartley Theorem' );`		

Graphic representations

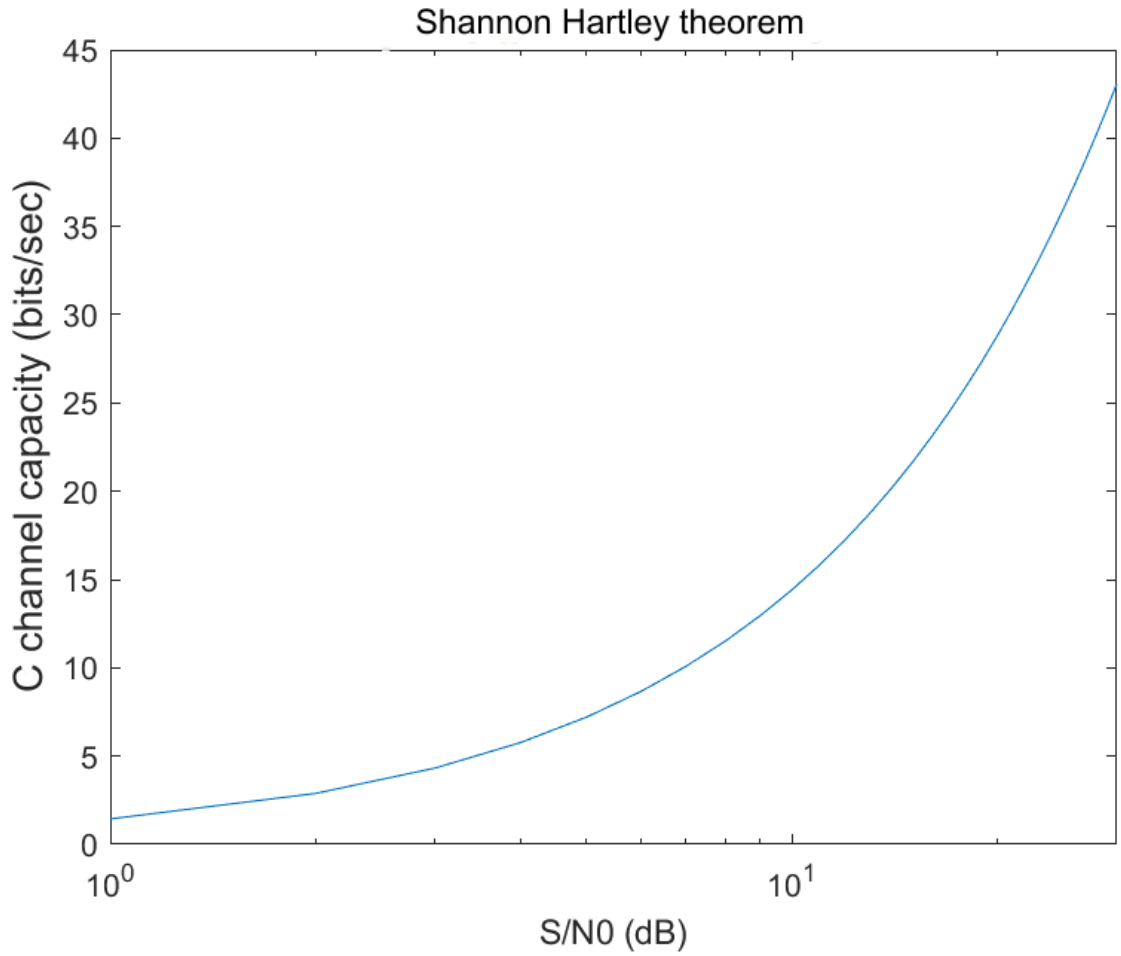

Figure 18.1. The capacity of the channel with 3KHz frequency bandwidth z

EXERCISE 19

Channel capacity. Write a program to calculate and represent the capacity of channel C with an S/N0 ratio equal to 25dB as a function of the frequency bandwidth of channel B (values from 1 to 100000). To collapse values, use the semilogx command instead of plot.
The value to which the capacitance tends is the one expected based on Theory;

Code at Matlab

```
B = 1:1:1e5
S_N0 = 25
C = B.*log2(1 + S_N 0./ B)

semilogx ( B, C)
xlabel ( ' Range zone of frequencies (Hz)' );
ylabel( 'C Channel Capacity (bits/sec)' );
title( 'Shannon Hartley Theorem' );
```

Explanation of the commands used

Commands	Results	Comments
>> B = 1:1:1e5	B = Columns 1 through 6 1 2 3 4 5 6 Columns 7 through 12 7 8 9 10 11 12 … Columns 99.991 through 99.996 99991 99992 99993 99994 99995 99996 Columns 99,997 through 100,000 99997 99998 99999 100000	The frequency bandwidth
>> S_N0 = 25	S_N0 = 25	The ratio S / N 0 in order to use the formula with the power spectral density N0/2
>> C = B.*log2(1 + S_N 0./ B)	C = Columns 1 through 8 4.7004 7.5098 9.6672 11.4319 12.9248 14.2154 15.3485 16.3552 Columns 9 through 16 17.2578 18.0735 18.8154 19.4939 20.1173 20.6927 21.2256 21.7208 … Columns 99.985 through 99.992 36.0629 36.0629 36.0629 36.0629 36.0629 36.0629 36.0629 36.0629 Columns 99,993 through 100,000	The capacity of the channel

	36.0629 36.0629 36.0629 36.0629 36.0629 36.0629 36.0629 36.0629	
>> semilogx(B, C) >> xlabel('Frequency bandwidth (Hz)'); >> ylabel('C channel capacity (bits/sec)'); >> title('Shannon Hartley Theorem');	Figure 1 9 .1	The collapse of the values in a graph, in the field of db defined. Add labels to the x and y axes

Answers to the sub-questions

The value to which the channel capacity tends is the expected one, since the frequency bandwidth B tends to infinity and therefore the value of the capacity is:

$$\lim_{B\to\infty} C = 1.44\frac{S}{N_0} \rightarrow \lim_{B\to\infty} C = 1.44 * 25 \rightarrow \lim_{B\to\infty} C = 36$$

Graphic representations

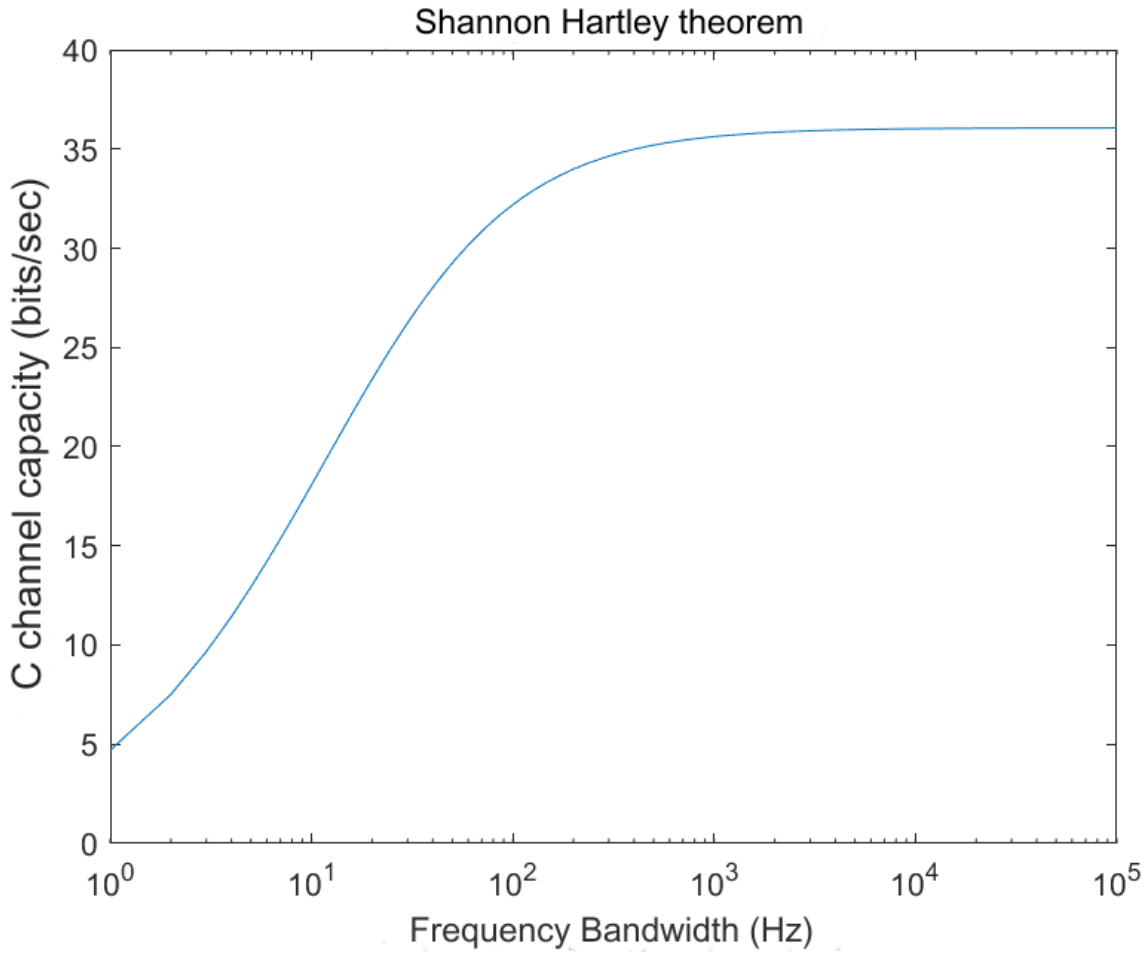

Figure 19.1. The capacity of the channel with an S / N ratio of 0.25 db

EXERCISE 20

Huffman Coding. To create the message sequence:

' my name is <insert here>'

Store the message in a variable named myname.

To generate the source alphabet and the vector of probability of occurrence of the symbols. To create the Huffman codebook. Huffman coding of the message. Calculate the entropy of the source, the efficiency of the code and its redundancy.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Graphic representations

EXERCISE 21

Binary PSK Modulation. Let the message sequence m=[1 1 0 0 0 1 1 0 1]. Suppose that each bit pulse has a duration of T=1ms and the sampling interval is dt=10^{-7} s. Let the carrier have a frequency of fc=5000Hz. Binary PSK modulate the message and then plot the spectrum of the information signal and the modulated.

Code in Matlab

```
m = [1 1 0 0 0 1 1 0 1]
T = 1 e -3
dt = 1e-7
fc = 5e3
Nm = length(m)
Tw = Nm*T
t = 0:dt :Tw-dt
Np = round(T/dt)
pm = []
for i = 1:Nm
pm = [pm m( i )* ones( 1, Np )]
end
```

```
carrier = cos(2*pi*fc*t)
psk = pm.*carrier

BW = 1/dt
Nt = length(t)
df = BW/ Nt
f = -BW/2:df:BW/2-df
ph_pm = fft (pm)
ph_pm = fftshift ( ph_pm )/ Nt
ph_psk = fft ( psk )
ph_psk = fftshift ( ph_psk )/ Nt

figure
subplot( 2, 1, 1 )
plot( t, pm )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( ' Sign information ' );
subplot( 2, 1, 2 )
plot( f, abs( ph_pm ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Information Signal Width Spectrum' );

figure
subplot( 2, 1, 1 )
plot( t, psk )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
title( 'PSK modulated signal' );
subplot( 2, 1, 2 )
plot( f, abs( ph_psk ))
xlabel( 'Frequency Range (Hz)' );
ylabel( 'Amplitude (Volt)' );
title( 'Width spectrum of PSK modulated signal' );
```

Explanation of the commands used

Commands	Results	Comments
`>> m = [1 1 0 0 0 1 1 0 1]`	m = 1 1 0 0 0 1 1 0 1	The message sequence
`>> T = 1e-3`	T = 1.0000e-03	The duration of each pulse – bit
`>> dt = 1e-7`	dt = 1.0000e-07	The sampling interval
`>> fc = 5e3`	fc = 5000	The carrier frequency
`>> Nm = length(m)`	Nm =	The size of the message

	9	
`>> Tw = Nm*T`	Tw = 0.0090	The length of the message
`>> t = 0:dt :Tw-dt`	t = Columns 1 through 8 0 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 Columns 9 through 16 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 … Columns 89.985 through 89.992 0.0090 0.0090 0.0090 0.0090 0.0090 0.0090 0.0090 0.0090 Columns 89,993 through 90,000 0.0090 0.0090 0.0090 0.0090 0.0090 0.0090 0.0090 0.0090	The time the message is set
`>> Np = round(T/dt)`	Np = 10000	The number of samples per pulse
`>> pm = []`	pm = []	The vector to contain the samples per pulse, initialized with the empty vector

`>> for i = 1:Nm`
`pm = [pm m( i )* ones( 1, Np )]`
`end`
	pm =
	
	Columns 1 through 13
	
	1 1 1 1 1 1 1 1 1 1 1 1 1
	
	Columns 14 through 26
	
	1 1 1 1 1 1 1 1 1 1 1 1 1 …
		Assigning the samples per pulse to the vector, that is, generating the information signal

`>> carrier = cos(2*pi*fc*t)`	carrier = Columns 1 through 8 1.0000 1.0000 1.0000 1.0000 0.9999 0.9999 0.9998 0.9998 Columns 9 through 16 0.9997 0.9996 0.9995 0.9994 0.9993 0.9992 0.9990 0.9989 …	The badge bearer

	Columns 89.985 through 89.992 0.9987 0.9989 0.9990 0.9992 0.9993 0.9994 0.9995 0.9996 Columns 89,993 through 90,000 0.9997 0.9998 0.9998 0.9999 0.9999 1.0000 1.0000 1.0000	
`>> psk = pm.*carrier`	psk = Columns 1 through 8 1.0000 1.0000 1.0000 1.0000 0.9999 0.9999 0.9998 0.9998 Columns 9 through 16 0.9997 0.9996 0.9995 0.9994 0.9993 0.9992 0.9990 0.9989 … Columns 89.985 through 89.992 0.9987 0.9989 0.9990 0.9992 0.9993 0.9994 0.9995 0.9996 Columns 89,993 through 90,000 0.9997 0.9998 0.9998 0.9999 0.9999 1.0000 1.0000 1.0000	The modulation of the information signal by DSB
`>> BW = 1/dt`	BW = 10000000	The bandwidth defined by the sampling in time
`>> Nt = length(t)`	Nt = 90000	The number of points in the time table
`>> df = BW/Nt`	df = 111.1111	Sampling in the frequency domain
`>> f = -BW/2:df:BW/2-df`	f = 1.0e+06 * Columns 1 through 8 -5.0000 -4.9999 -4.9998 -4.9997 -4.9996 -4.9994 -4.9993 -4.9992 Columns 9 through 16 -4.9991 -4.9990 -4.9989 -4.9988 -4.9987 -4.9986 -4.9984 -4.9983 … Columns 89.985 through 89.992	The table of frequencies

	4.9982 4.9983 4.9984 4.9986 4.9987 4.9988 4.9989 4.9990 Columns 89,993 through 90,000 4.9991 4.9992 4.9993 4.9994 4.9996 4.9997 4.9998 4.9999	
`>> ph_pm = fft(pm)`	ph_pm = 1.0e+04 * Columns 1 through 4 5.0000 + 0.0000i 1.4106 + 0.7462i 0.2451 - 2.0190i 0.8270 + 0.0001i Columns 5 through 8 -0.2303 + 0.8229i 0.1840 + 0.6584i -0.4135 - 0.0001i -0.0698 - 0.5769i … Columns 89.993 through 89.996 -0.1764 - 0.0932i - 0.0698 + 0.5769i -0.4135 + 0.0001i 0.1840 - 0.6584i Columns 89,997 through 90,000 -0.2303 - 0.8229i 0.8270 - 0.0001i 0.2451 + 2.0190i 1.4106 - 0.7462i	The amplitude spectrum of the information signal by calling the fft function from the fast Fourier transform
`>> ph_pm = fftshift ( ph_pm )/ Nt`	ph_pm = Columns 1 through 4 0.0000 + 0.0000i - 0.0000 + 0.0000i 0.0000 + 0.0000i -0.0000 + 0.0000i Columns 5 through 8 -0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i … Columns 89.993 through 89.996 -0.0000 + 0.0000i 0.0000 + 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i Columns 89,997 through 90,000 -0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 - 0.0000i -0.0000 - 0.0000i	The amplitude spectrum of the information signal by calling the function fft from the fast Fourier transform , symmetric about the vertical axis by calling the function fftshift
`>> ph_psk = fft( psk )`	ph_psk =	The amplitude spectrum of binary-modulated PSK by

	1.0e+04 * Columns 1 through 4 -0.0000 + 0.0000i - 0.0007 - 0.0003i - 0.0003 + 0.0040i -0.0037 + 0.0001i Columns 5 through 8 0.0017 - 0.0066i - 0.0024 - 0.0082i 0.0075 - 0.0001i 0.0019 + 0.0143i … Columns 89.993 through 89.996 0.0057 + 0.0031i 0.0019 - 0.0143i 0.0075 + 0.0001i - 0.0024 + 0.0082i Columns 89,997 through 90,000 0.0017 + 0.0066i - 0.0037 - 0.0001i - 0.0003 - 0.0040i -0.0007 + 0.0003i	calling the fft function from the fast Fourier transform
`>> ph_psk = fftshift ( ph_psk )/ Nt`	ph_psk = Columns 1 through 4 -0.0000 + 0.0000i - 0.0000 + 0.0000i 0.0000 + 0.0000i -0.0000 + 0.0000i Columns 5 through 8 -0.0000 - 0.0000i - 0.0000 + 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i … Columns 89.993 through 89.996 -0.0000 + 0.0000i 0.0000 + 0.0000i 0.0000 + 0.0000i - 0.0000 - 0.0000i Columns 89,997 through 90,000 -0.0000 + 0.0000i - 0.0000 - 0.0000i 0.0000 - 0.0000i -0.0000 - 0.0000i	The amplitude spectrum of binary-modulated PSK by calling the function fft from the fast Fourier transform , symmetric about the vertical axis by calling the function fftshift

`>> figure`
`>> subplot( 2, 1, 1 )`
`>> plot( t, pm )`
`>> xlabel ( ' Time (sec)' );`
`>> ylabel ( ' Amplitude (Volt)' );`
`>> title( ' Sign information ' );`
`>> subplot( 2, 1, 2 )`
`>> plot( f, abs( ph_pm ))`
`>> xlabel( 'Frequency Range (Hz)' );`
	Figure 21.1	The drawing of the graphical representations of the information signal in the defined time domain and of the amplitude spectrum in the defined frequency domain. Add title and labels to the x and y axes .

>> ylabel('Amplitude (Volt)'); >> title('Information Signal Width Spectrum');		
>> figure >> subplot(2, 1, 1) >> plot(t, psk) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title('PSK modulated signal'); >> subplot(2, 1, 2) >> plot(f, abs(ph_psk)) >> xlabel('Frequency Range (Hz)'); >> ylabel('Amplitude (Volt)'); >> title('Width spectrum of modulated signal by PSK');	Figure 21.2	The plotting of the PSK modulated signal in the defined time domain and the amplitude spectrum in the defined frequency domain. Add title and labels to the x and y axes .

Graphic representations

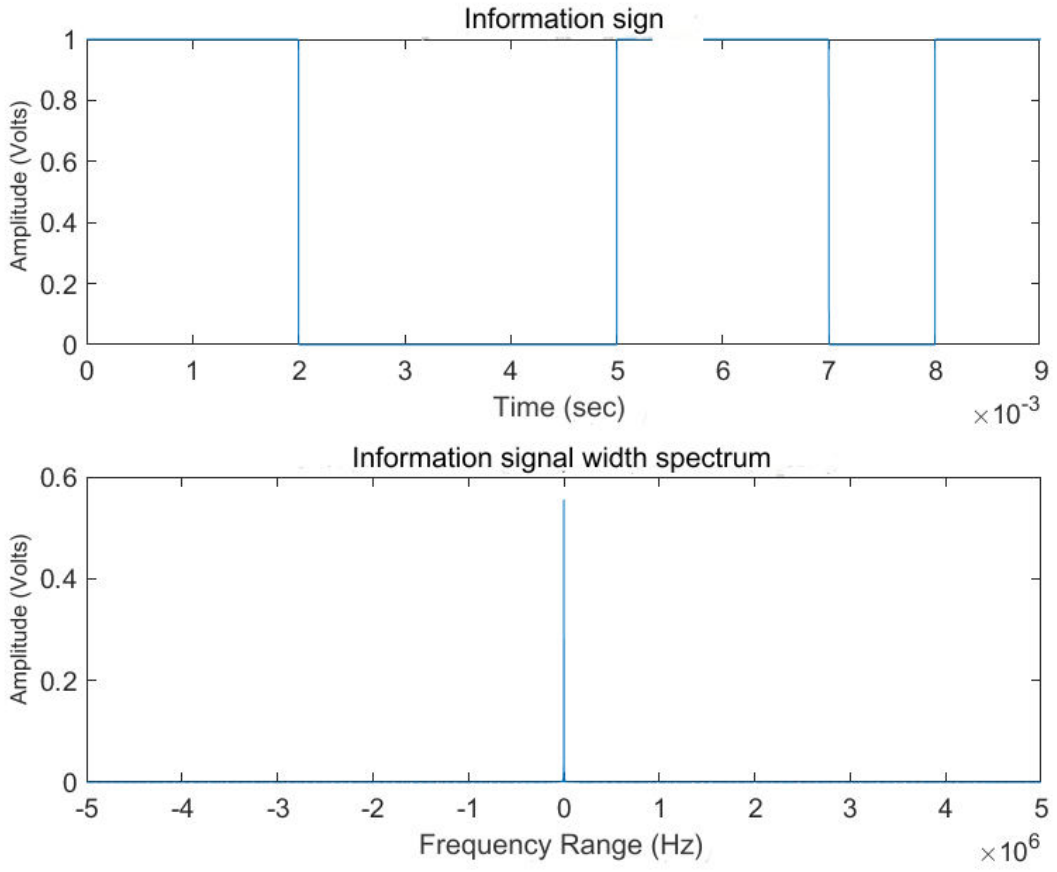

Figure 21.1. The graphs of the information signal and the amplitude spectrum

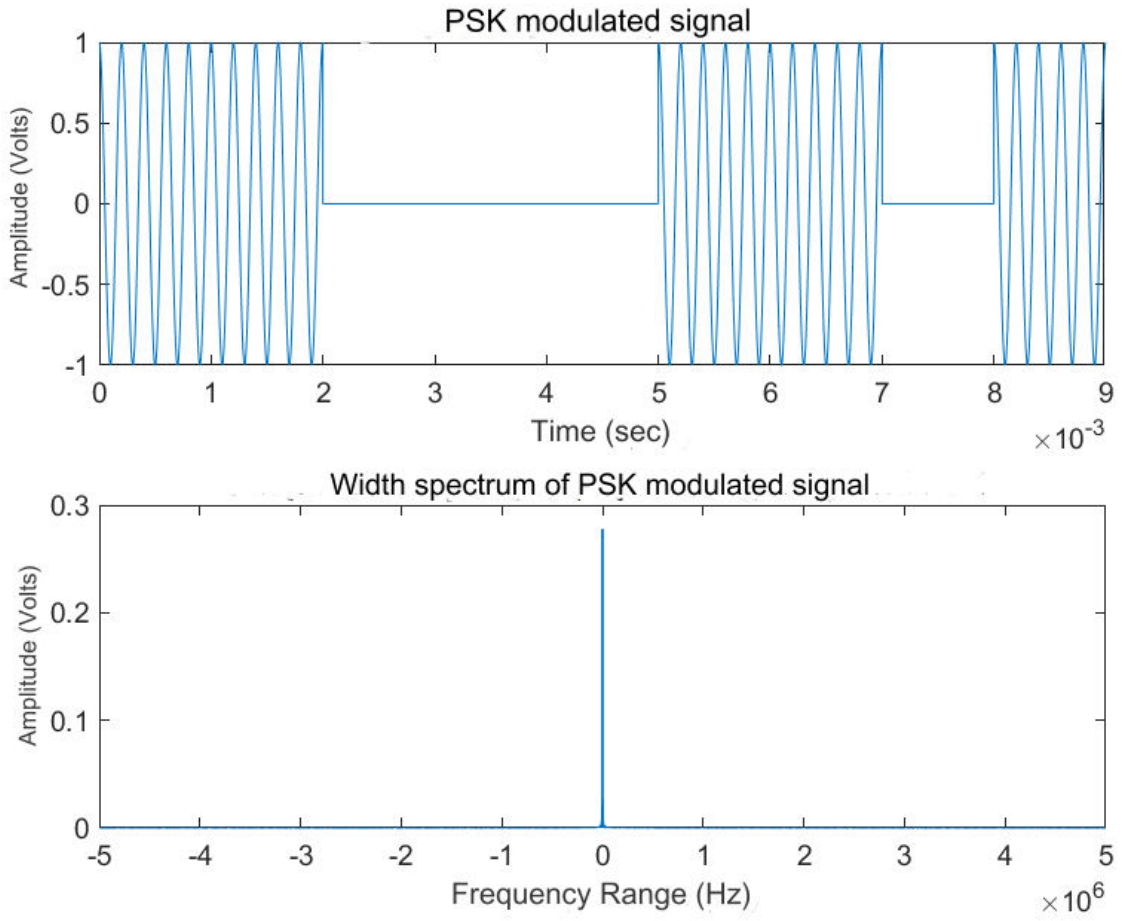

Figure 22.2. The plots of the PSK modulated signal and the amplitude spectrum

EXERCISE 22

QAM configuration. Run the following Matlab code to generate and plot 16QAM modulated data. Explain in detail what each command does.

Write the corresponding code for a) 16QAM and SNR=5dB b)4 QAM and 20dB.
Use the scatterplot() function alternatively to produce the signal constellations.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Graphic representations

EXERCISE 23

Text entropy. Write code to calculate the Entropy of an English text.

Hint: you can identify the different letters that appear in the text and then calculate the probability of their occurrence (frequency of occurrence of the character / set of characters in the text). Then you calculate the entropy based on the definition. (For simplicity you can consider the text to consist only of letters and spaces. You can also consider a short text and store it in a variable e.g. text= 'This is a short text '?) What are the chances of text characters appearing;

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Graphic representations

EXERCISE 24

Source entropy. Given a DMS source with probabilities of transmitted symbols:

0.2, 0.05, 0.03, 0.1, 0.3, 0.02, 0.22, 0.08.

Apply Huffman coding to the source and then generate a sequence of 200 symbols of the source. The sequence to be Huffman encoded.

The program should display for each symbol of the source (x1,x2,....) the probability of its appearance and the code word assigned to it, the printout will be as follows:

x1 probability : 0.3 codeword :--->

0 0

x2 probability : 0.25 codeword :--->

0 1

.....

Then calculate the entropy of the source and the average code word length. Also calculate the performance and redundancy of the code.

Code in Matlab

Main program

```
symbols = 1:1:8

p = [0.2 0.05 0.03 0.1 0.3 0.02 0.22 0.08]
dict = huffmandict ( symbols, p )
sig = randsrc ( 200 , 1 , [ symbols ; p ])
comp = huffmanenco ( sig, dict )


for i = 1:1:8
symbol = i
prob = p( i )
codeword = dict ( symbol, 2 )
    fprintf ( ' x%d probability : %.2f codeword :--->\n\t %d %d\n' , i , prob, code-
word{1, 1})
end

H = myentropy (p)
fprintf ( ' Entropy source %.2f\n' , H)
avgLenDict = mean(comp)
fprintf ( ' Avg length code of word %.2f\n' , avgLenDict )
performance = H / avgLenDict
fprintf ( ' Performance source %.2f\n' , performance)
pleonasmos = 1 - performance
fprintf ( ' Redundancy source %.2f\n' , pleonasmos )
```

Code function myentropy

```
function H= myentropy (prob)

Hi=prob.*log2( 1./ (prob+1e-9));
H=sum(Hi);
```

Explanation of the commands used

Commands	Results	Comments
`>> symbols = 1:1:8`	symbols = 1 2 3 4 5 6 7 8	The symbols
`>> p = [0.2 0.05 0.03 0.1 0.3 0.02 0.22 0.08]`	p = 0.2000 0.0500 0.0300 0.1000 0.3000 0.0200 0.2200 0.0800	The probabilities of emitted symbols

`>> dict = huffmandict( symbols, p)`	dict = 8×2 cell array {[1]} {[1 1]} {[2]} {[0 1 0 0 0]} {[3]} {[0 1 0 0 1 0]} {[4]} {[0 1 1]} {[5]} {[0 0]} {[6]} {[0 1 0 0 1 1]} {[7]} {[1 0]} {[8]} {[0 1 0 1]}	Huffman mapping of symbols appearing with corresponding probabilities p , to binary codewords
`>> sig = randsrc( 200, 1, [symbols; p])`	sig = 8 7 1 5 1 1 1 5 7 7 …	Generate a sequence of 200 symbols with corresponding probability of occurrence p
`>> comp = huffmanenco( sig, dict)`	comp = 0 1 0 1 1 0 1 1 0 0 …	Huffman sequence coding

`>> for i = 1:1:8`
`symbol = i`
`prob = p( i )`
`codeword = dict ( symbol, 2 )`
`fprintf ( ' x%d proba-bility : %.2f codeword :--->\n\t %d %d\n' , i , prob, codeword{1, 1})`
`end`
	symbol =
	
	1
	
	prob =
	
	0.2000
	
	codeword =
	
	1×1 cell array
	
	{[1 1]}
		Calculation and display of the occurrence probability for each symbol of the source as well as the code word assigned to it

	x1 probability : 0.20 codeword :---> 1 1 … symbol = 8 prob = 0.0800 codeword = 1×1 cell array {[0 1 0 1]} x8 probability : 0.08 codeword :--->0 1	

`>> H = myentropy ( p )`
`>> fprintf ( 'Source entropy %.2 f \ n ' , H )`
 	
`function H= myentropy (prob)`
 	
`Hi=prob.*log2( 1./ (prob+1e-9));`
`H=sum(Hi);`
	H =
	
	2.5705
	
	Entropy source 2.57
		Calculation of the entropy of the source by calling the myentropy function and displaying it in the output

`>> avgLenDict = mean ( comp )` `>> fprintf ( 'Average code word length %.2 f \ n ' , avgLenDict )`	avgLenDict = 0.4280 Average code word length 0.43	Calculation of the average code word length by calling the mean function and displaying it in the output
`>> performance = H / avgLenDict` `>> fprintf ( ' Performance source %.2f\n' , performance)`	performance = 6.0056 Source rendering 6.01	Calculation of the source performance and display in the output
`>> pleonasmos = 1 - performance` `>> fprintf ( ' Redundancy source %.2f\n' , pleonasmos )`	pleonasmos = -5.0056 Source surplus -5.01	Calculation of source redundancy and display in output

Output

x1 probability : 0.20 codeword :--->
1 1
x2 probability : 0.05 codeword :--->
0 1
x3 probability : 0.03 codeword :--->

0 1
x4 probability : 0.10 codeword :--->
0 1
x5 probability : 0.30 codeword :--->
0 0
x6 probability : 0.02 codeword :--->
0 1
x7 probability : 0.22 codeword :--->
1 0
x8 probability : 0.08 codeword :--->
0 1

EXERCISE 25

Hamming code. Hamming codes are linear block codes with $n=2^m-1$, $k=2^m-1-m$ and minimum distance $d_{min} = 3$ and which have a very simple parity check table. The parity check table which is an $m\times(2^m-1)$ table has as columns all binary sequences of length m except the zero sequence.

To create a program for the Hamming code (15,11). For this purpose use the command [h,g,n,k] = hammgen().

How many bits does the code word have and how many bits does the information message have for that particular code? What is the parity control panel and what is the generator?

Does the form of the parity control table agree with the theoretical specifications?

Encode the message

[1 0 0 1 0 1 1 1 1 1 0]

by writing the encode command: code = encode(mes,n, k, 'hamming'). Theoretically verify the resulting code word.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Output

EXERCISE 26

Marcum's Q function. When studying the effect of noise on digital signal transmission, probabilities $P(X > a)$ are of interest. The corresponding integral is not calculated analytically and for its calculation the matrix of

the Q function of Marcum is used. The value of the function Q(x) corresponds to the area of the Gaussian curve from $x \to \infty$.

In Matlab there is the built-in function qfunc(x) which calculates the value of Q(x).

Compute the values of the Q function (using qfunc) for x=[0, 1, 2, 3, 4, 5, 6, 7]. Compare with the corresponding values of Marcum's Q-function table. Then plot these values and find that Q is a rapidly decreasing function.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Graphic representations

EXERCISE 27

Noisy signal, SNR. Create a sinusoidal signal of frequency 100Hz, duration 4sec, amplitude √2 and sampling frequency 8KHz. Calculate the power of the sinusoidal signal. $P_x = \frac{1}{N}\sum_{n=1}^{N}|x[n]|^2$. Then use the wgn function to generate white Gaussian noise with SNR=15dB, (syntax : wgn(1,N,Pn,'linear'), where, Pn the noise power, will be calculated by the formula snr= 10log10(px/pn)). Add noise to the signal. Graph the noisy signal.

Code in Matlab

```
f = 100
duration = 4
A = sqrt( 2 )
fs = 8e3
Ts = 1/fs
t = 0:Ts :duration
x = A*sin(2*pi*f*t)
N = length(x)
P = 1/N*sum(abs(x.^2))

snr = 15
Pg = P/10^(snr/10)
gaussian = wgn ( 1, N, Pg, 'linear' )
addNoise = x + gaussian

plot( t, addNoise )
xlabel ( ' Time (sec)' );
ylabel ( ' Amplitude (Volt)' );
```

```
title( 'Noisy Signal SNR' );
```

Explanation of the commands used

Commands	Results	Comments
>> f = 100	f = 100	The frequency of the signal
>> duration = 4	duration = 4	The time duration of the signal
>> A = sqrt(2)	A = 1.4142	The width of the signal
>> fs = 8e3	fs = 8000	The sampling frequency
>> Ts = 1/fs	Ts = 1.2500e-04	The sampling period
>> t = 0:Ts :duration	t = Columns 1 through 8 0 0.0001 0.0003 0.0004 0.0005 0.0006 0.0008 0.0009 Columns 9 through 16 0.0010 0.0011 0.0013 0.0014 0.0015 0.0016 0.0018 0.0019 … Columns 31,993 through 32,000 3.9990 3.9991 3.9992 3.9994 3.9995 3.9996 3.9998 3.9999 Column 32.001 4,0000	The time field that defines the signal
>> x = A*sin(2*pi*f*t)	x = Columns 1 through 8 0 0.1110 0.2212 0.3301 0.4370 0.5412 0.6420 0.7389 Columns 9 through 16 0.8313 0.9185 1.0000 1.0754 1.1441 1.2058 1.2601 1.3066 …	The sinusoidal signal

	Columns 31,993 through 32,000 -0.8313 -0.7389 -0.6420 -0.5412 -0.4370 -0.3301 -0.2212 -0.1110 Column 32.001 0.0000	
>> N = length(x)	N = 32001	The signal size
>> P = 1/N*sum(abs(x.^2))	P = 1,0000	The signal strength
>> snr = 15	snr = 15	The S NR of white Gaussian noise
>> Pg = P/10^(snr/10)	Pg = 0.0316	The power of noise
>> gaussian = wgn (1, N, Pg, 'linear')	gaussian = Columns 1 through 8 -0.0877 -0.0304 -0.2824 0.1503 0.0731 -0.0827 0.1398 0.3261 Columns 9 through 16 -0.2496 -0.0287 0.0265 0.1927 -0.0648 0.2330 0.1591 -0.0431 … Columns 31,993 through 32,000 0.0230 0.2630 0.1199 0.2833 0.1259 -0.1238 0.0093 0.0811 Column 32.001 0.0229	Gaussian white noise by calling the function w gn
>> addNoise = x + gaussian	addNoise = Columns 1 through 8 -0.0877 0.0806 -0.0612 0.4805 0.5101 0.4585 0.7819 1.0650 Columns 9 through 16 0.5817 0.8898 1.0265 1.2681 1.0793 1.4388 1.4191 1.2635 … Columns 31,993 through 32,000	Adding the noise to the sinusoidal signal

	-0.8083 -0.4759 -0.5221 -0.2579 -0.3112 -0.4540 -0.2119 -0.0299 Column 32.001 0.0229	
>> plot(t, addNoise) >> xlabel (' Time (sec)'); >> ylabel (' Amplitude (Volt)'); >> title(' Noise signal SNR');	Figure 27.1	Plotting the signal-to-noise plot in the specified time domain. Add title and labels to the x and y axes

Graphic representations

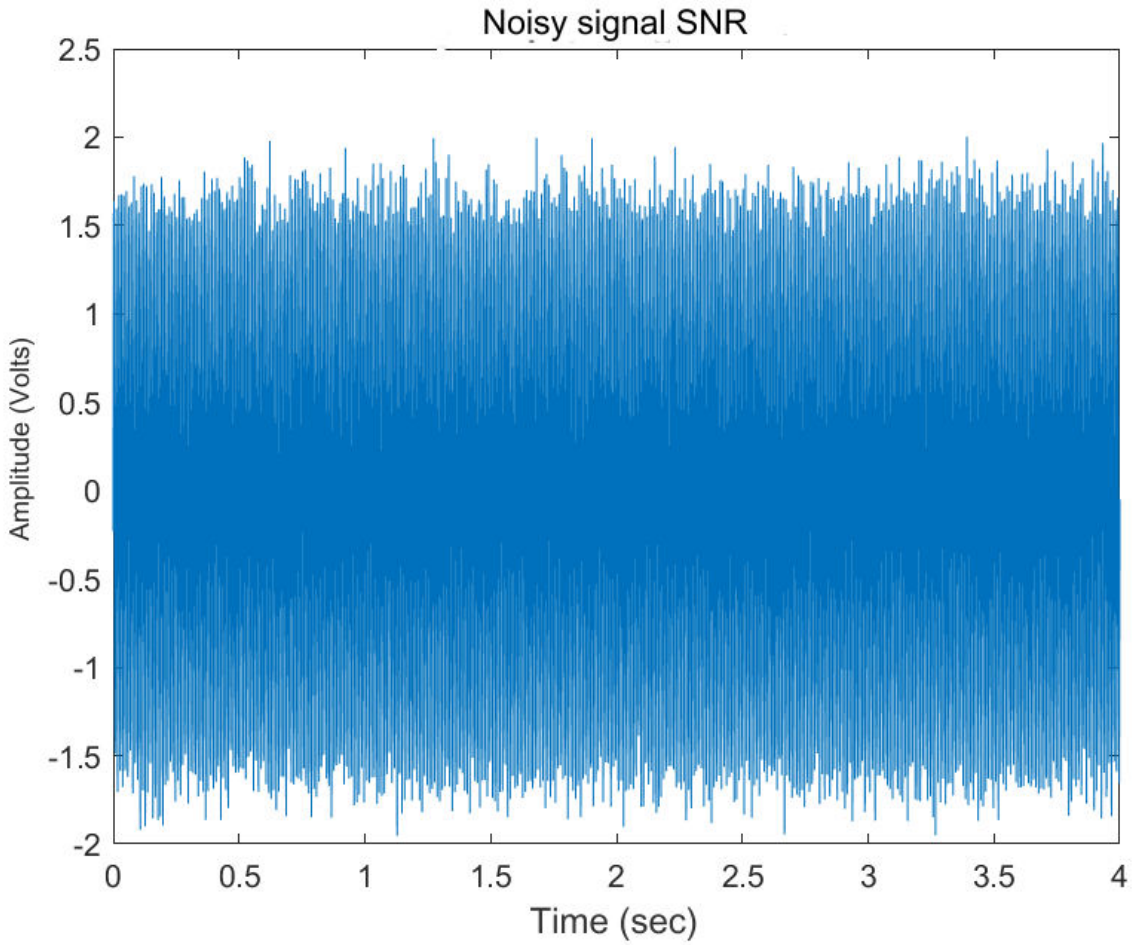

Figure 27.1. The sinusoidal signal with noise

EXERCISE 28

M-adic PSK modulation. Open Matlab's bertool (Bit Error Rate Analysis) application. Create, with the help of the application, the appropriate graph to graphically show that in M-adic PSK modulation, the noise tolerance decreases as M increases. Use theoretical analysis, AWGN channel type, PSK modulation type, channel coding none and perfect synchronization. Comment on the graph you created.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Graphic representations

EXERCISE 29

Encoding with replay code. A very simple case of block coding is the simple repetition code. In this code the code word consists of the simple repetition of 0 (or 1) n times (n odd). The decoding process is based on a majority decision: In the decoding process, an error is observed when at least (n+1)/2 of the transmitted bits have been received incorrectly. If the communication channel is a Binary Symmetric Channel, BSC, channel, which exhibits a transmitted bit error probability equal to p_{ij}, then the decoding error probability for a simple repetition code (n,k) turns out to be given by the following relation,

$$p_e = \sum_{k=(n+1)/2}^{n} \binom{n}{k} {p^k}_{ij}(1-p_{ij})^{n-k}$$

Write a program to calculate and plot the error probability pe for a repetition code, given that the error probability of a transmitted bit on the communication channel is equal to p_{ij}=0.3 and plot the decoding error probability pe as a function of length of code (block length) n. Give n all odd values from 1 to 61. What do you think we pay for reducing the error probability by increasing the code length.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Output

EXERCISE 30

Encoding with circular code. Consider circular code with n=7 and k=4. Generate a generator polynomial using the command genpoly = cyclpoly(n,k). Note that the generated polynomial has its coefficients in ascending order. Then consider the message msg=[1 1 1 0]. Use the command code=encode(msg, n, k, 'cyclic', genpoly) to encode the message. What code word comes up? Encode and theorize the message based on the generator polynomial and compare the results.

Code in Matlab

Explanation of the commands used

Commands	Results	Comments

Output

Thank you for your attention.